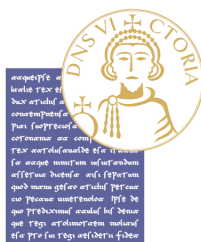


Università degli Studi del Sannio
Dipartimento di Scienze e Tecnologie
Corso di Laurea Magistrale in Biologia
Curriculum Biosanitario



Tesi di laurea in Bioinformatica

Analisi bioinformatica delle strutture comuni in piccole molecole associate al carcinoma ovarico

Relatore
Ch.mo Prof.
Francesco Napolitano

Candidata
Erica Cogliano
Mat. 506001932

Indice

1. Il carcinoma ovarico: inquadramento biologico e clinico	4
1.1 Il carcinoma ovarico come gruppo eterogeneo di neoplasie	4
1.2 Il carcinoma sieroso di alto grado	5
1.3 Diagnosi, trattamento e recidiva	8
1.4 I composti come oggetto di analisi	9
2. Chemioinformatica	11
2.1 Chemioinformatica e spazio chimico	11
2.2 Rappresentazione computazionale della struttura molecolare	12
2.2.1 Il formato MOL	14
2.2.2 Il Formato SDF	15
2.2.3 La Classe SDFset	17
2.3 Similarità molecolare	18
2.4 La rappresentazione digitale della struttura: Fingerprint molecolari	20
2.5 Coefficiente di Tanimoto: la matrice di similarità	22
2.5.1 Dalla Similarità alla Matrice di Distanza	23
2.6 Proiezione bidimensionale e analisi dei cluster con MDS e clustering k-means	24
2.7 Identificazione del core strutturale: algoritmo MCS	25
2.8 Scaffold di Bemis-Murko	27
3. Metodi	29
3.1 Obiettivo del lavoro	29
3.2 Linguaggi di programmazione e strumenti software	30
3.2.1 Database utilizzati	31
TTD	31
PubChem	32
3.3 Workflow computazionale in R	32
3.3.1 Selezione dei composti da TTD e filtro per carcinoma ovarico	32
3.3.2 Recupero delle strutture da PubChem	34
3.3.3 Unione dei file strutturali e filtraggio delle molecole	35
3.3.4 Calcolo della similarità strutturale e costruzione della matrice di distanza	36
3.3.5 Rappresentazione esplorativa delle relazioni strutturali mediante PCoA	38
3.3.6 Clustering e identificazione della MCS	38
3.4 Analisi complementare in Python: fingerprint, clustering e identificazione della Maximum Common Substructure	42
4. Risultati	46
4.1 Selezione dei composti associati al carcinoma ovarico	46
4.2 Recupero delle strutture molecolari e copertura del dataset	47
4.3 Costruzione dell'archivio strutturale e filtraggio preliminare	47
4.4 Matrice di similarità strutturale e matrice di distanza	48
4.5 Interpretazione della matrice di distanza	49

4.6 Rappresentazione esplorativa mediante PCoA	51
4.7 Dalla PCoA al clustering: visualizzazione dello spazio chimico	53
4.8 Analisi complementare in Python: fingerprint, distanza di Tanimoto e clustering	54
4.9 MDS e partizionamento in cluster nella pipeline Python	54
4.10 Identificazione delle sottostrutture comuni nei cluster	55
4.11 Confronto delle MCS con le molecole residue del file P1-05	56
4.12 Verifica in letteratura dei composti selezionati	57
4.13 Altri composti con riscontri in letteratura	59
4.14 Molecole visibili nei cluster e produzione delle immagini	60
4.15 Sintesi interpretativa dei risultati	61
5. Discussione e conclusioni	62
5.1 Sintesi dei risultati principali	62
5.2 Significato metodologico e chemioinformatico dei risultati	63
5.3 Limiti del lavoro	65
5.4 Confronto tra workflow R e pipeline Python	66
5.5 Prospettive future	67
5.6 Conclusioni	68
7. Script	76
7.1 R	76
7.2 Python	100

1. Il carcinoma ovarico: inquadramento biologico e clinico

1.1 Il carcinoma ovarico come gruppo eterogeneo di neoplasie

Il carcinoma ovarico rappresenta una delle neoplasie ginecologiche più rilevanti per impatto clinico e prognostico. A livello globale, il tumore dell'ovaio si colloca tra le principali cause di morte oncologica nella popolazione femminile; secondo le stime del Global Cancer Observatory, nel 2022 sono stati registrati circa 324.000 nuovi casi e oltre 206.000 decessi nel mondo. Il peso clinico di questa patologia non dipende soltanto dalla sua incidenza, ma soprattutto dall'elevata mortalità relativa, che riflette la frequente diagnosi in fase avanzata, l'assenza di sintomi iniziali specifici e la tendenza alla recidiva anche dopo una risposta iniziale al trattamento (Global Cancer Observatory, 2022).

Sebbene il termine “carcinoma ovarico” possa suggerire l'esistenza di una singola entità patologica, esso indica in realtà un gruppo eterogeneo di neoplasie che condividono la sede anatomica di presentazione ma differiscono profondamente per origine cellulare, caratteristiche istologiche, profilo molecolare, comportamento clinico e sensibilità terapeutica. In questo senso, il carcinoma ovarico non può essere considerato una malattia unitaria, bensì una categoria ampia che comprende sottotipi biologicamente e clinicamente distinti. Questa eterogeneità costituisce uno degli aspetti più importanti della patologia, poiché condiziona in modo diretto sia l'interpretazione biologica della malattia sia le strategie diagnostiche e terapeutiche adottate nella pratica clinica (Lisio et al., 2019).

Dal punto di vista istologico, circa il 90% dei tumori ovarici maligni appartiene al gruppo dei tumori epiteliali ovarici (epithelial ovarian cancers, EOC), mentre una quota minore è rappresentata da neoplasie derivanti da cellule germinali o da tessuti stromali. All'interno dei tumori epiteliali, i principali sottotipi riconosciuti sono il sieroso, il mucinoso, l'endometrioide e il carcinoma a cellule chiare, ai quali si aggiungono forme più rare e meno rappresentate. Tale classificazione non ha un significato puramente descrittivo, ma riflette differenze sostanziali nei meccanismi di carcinogenesi, nel grado di aggressività, nelle alterazioni molecolari e nella risposta alle terapie (Lisio et al., 2019).

L'importanza di questa classificazione risiede nel fatto che tumori apparentemente appartenenti alla stessa sede anatomica possono seguire traiettorie biologiche molto diverse. Alcuni sottotipi presentano una crescita relativamente lenta e sono associati a lesioni precursori riconoscibili, mentre altri mostrano fin dall'esordio una maggiore aggressività, una rapida diffusione e una marcata instabilità genomica. Ne deriva che il carcinoma ovarico deve essere interpretato come un insieme di malattie correlate ma non sovrapponibili, la cui comprensione richiede una distinzione accurata tra le diverse entità istopatologiche e molecolari.

Un contributo fondamentale alla comprensione di tale eterogeneità è stato fornito dal modello dualistico della carcinogenesi ovarica, che distingue i tumori epiteliali in Tipo I e Tipo II. I tumori di Tipo I comprendono neoplasie generalmente a crescita più lenta, relativamente stabili dal punto di vista genomico e spesso associate a lesioni precursori o borderline. In questo gruppo rientrano, con caratteristiche diverse, alcuni tumori sierosi di basso grado, mucinosi, endometrioidi e a cellule chiare. I tumori di Tipo II, al contrario, sono caratterizzati da una maggiore aggressività biologica, da una rapida evoluzione clinica, da elevata instabilità genomica e da una frequente diagnosi in stadio avanzato. Questo secondo gruppo include il carcinoma sieroso di alto grado, che costituisce il prototipo biologico e clinico della forma più aggressiva di carcinoma ovarico (Lisio et al., 2019).

Nella pratica clinica e nella letteratura biomedica, il sottotipo di maggiore interesse è proprio il carcinoma sieroso di alto grado (high-grade serous carcinoma, HGSC). Esso rappresenta il sottotipo predominante tra i tumori ovarici epiteliali diagnosticati in ambito clinico ed è responsabile di una quota particolarmente elevata dei decessi correlati alla malattia. La sua rilevanza dipende non solo dalla frequenza, ma anche dalla combinazione di elevata aggressività biologica, tendenza alla disseminazione peritoneale, diagnosi tardiva e marcata propensione alla recidiva dopo il trattamento iniziale. Per tali ragioni, l'HGSC costituisce oggi il principale oggetto di studio nell'ambito della ricerca biologica, clinica e traslazionale sul carcinoma ovarico (Lisio et al., 2019).

1.2 Il carcinoma sieroso di alto grado

Il carcinoma sieroso di alto grado (high-grade serous carcinoma, HGSC) costituisce il sottotipo di maggiore rilevanza biologica e clinica tra i tumori ovarici epiteliali. La sua

importanza non dipende soltanto dalla frequenza con cui viene osservato nella pratica clinica, ma soprattutto dal fatto che concentra in sé molti degli elementi che rendono il carcinoma ovarico una patologia particolarmente complessa: aggressività biologica, instabilità genomica, diffusione precoce, diagnosi spesso tardiva e marcata tendenza alla recidiva (Lisio et al., 2019).

Dal punto di vista istopatologico, l'HGSC si caratterizza per una marcata atipia nucleare di alto grado, un elevato indice mitotico e una notevole variabilità architetturale. Questi tumori possono presentare aree solide, nonché architetture papillari, ghiandolari o cribriformi, spesso accompagnate da estese zone di necrosi. Sul piano citologico, le cellule tumorali mostrano nuclei grandi, ipercromatici e pleomorfi, talvolta con multinucleazione, oltre a nucleoli prominenti e abbondanti figure mitotiche. Tali caratteristiche riflettono il comportamento aggressivo della neoplasia e la distinguono nettamente dalle forme sierose di basso grado, che oggi vengono considerate entità biologicamente differenti (Lisio et al., 2019).

Accanto ai caratteri morfologici, l'HGSC possiede un profilo molecolare altamente distintivo. Questo sottotipo è infatti associato ad alterazioni di TP53, considerate uno dei tratti più caratteristici della malattia. Le mutazioni di TP53 risultano pressoché ubiquitarie e rappresentano uno degli eventi più precoci del processo di carcinogenesi. Questo dato conferisce all'HGSC una forte specificità rispetto ad altri sottotipi di tumore ovarico epiteliale e sottolinea il ruolo cruciale della perdita dei normali meccanismi di controllo del danno cellulare.

Un altro elemento centrale è rappresentato dalla marcata instabilità genomica, che costituisce una delle principali firme molecolari dell'HGSC. Più che per la presenza di numerose mutazioni ricorrenti, questo tumore si distingue per l'elevata frequenza di alterazioni del numero di copie geniche e per il coinvolgimento di pathway deputati alla riparazione del DNA. In particolare, assumono un ruolo rilevante i difetti della ricombinazione omologa, nei quali si inserisce la rilevanza delle alterazioni di BRCA1 e BRCA2, geneticamente o funzionalmente implicate in una quota significativa dei casi. Questa configurazione molecolare contribuisce a spiegare non solo l'aggressività biologica dell'HGSC, ma anche alcune delle sue peculiarità cliniche e terapeutiche, rendendolo uno dei tumori ovarici più complessi dal punto di vista genetico.

Uno dei cambiamenti concettuali più significativi nella comprensione dell'HGSC riguarda la sua origine biologica. Per lungo tempo si è ipotizzato che questo tumore derivasse direttamente dall'epitelio di superficie ovarico, anche in relazione alla cosiddetta ipotesi della "incessante ovulazione", secondo cui i ripetuti processi di danno e riparazione associati all'ovulazione avrebbero favorito la trasformazione neoplastica. Le evidenze più recenti hanno tuttavia progressivamente spostato l'attenzione verso un'origine extra-ovarica, in particolare a partire dall'epitelio secretorio della porzione distale della tuba di Falloppio. In questo modello, lesioni precursori note come serous tubal intraepithelial carcinomas (STICs) rappresentano il più probabile stadio iniziale della trasformazione neoplastica, dal quale il tumore può successivamente diffondersi all'ovaio e alla cavità peritoneale. (Lisio et al. 2019)

Anche il pattern di diffusione dell'HGSC contribuisce in modo importante alla sua gravità clinica. A differenza di molte altre neoplasie epiteliali, esso tende a disseminarsi precocemente nella cavità peritoneale, spesso senza richiedere una metastatizzazione ematogena o linfatica classica. Le cellule tumorali possono infatti distaccarsi dal sito primitivo, sopravvivere nel liquido peritoneale e impiantarsi successivamente su superfici sierose e strutture addominali, con particolare predilezione per l'omento. Questo comportamento aiuta a spiegare sia la frequente presentazione in stadio avanzato sia la difficoltà di ottenere un controllo definitivo della malattia, anche nei casi inizialmente responsivi alla terapia (Lisio et al., 2019).

Un ulteriore elemento di criticità è rappresentato dalla scarsa specificità dei sintomi iniziali, spesso di tipo addominale o gastrointestinale. Manifestazioni come gonfiore addominale, dolore, nausea, sazietà precoce o alterazioni della funzione intestinale non favoriscono una diagnosi tempestiva e contribuiscono al fatto che molte pazienti giungano all'osservazione quando la malattia si è già estesa oltre la sede di origine. La diagnosi tardiva costituisce pertanto uno dei principali fattori responsabili della prognosi sfavorevole e contribuisce a spiegare l'elevata mortalità che caratterizza questo sottotipo (Lisio et al., 2019).

Nel complesso, il carcinoma sieroso di alto grado emerge come il nucleo biologico e clinico più rilevante del carcinoma ovarico epiteliale. La sua definizione non dipende da un solo elemento, ma dall'integrazione di caratteristiche morfologiche, molecolari e cliniche che lo rendono una neoplasia particolarmente aggressiva e complessa. Per tali ragioni,

l'HGSC costituisce il sottotipo di riferimento più appropriato per introdurre il contesto patologico entro cui si colloca il presente lavoro.

1.3 Diagnosi, trattamento e recidiva

Uno degli aspetti che contribuiscono maggiormente alla gravità clinica dell'HGSC è la difficoltà di ottenere una diagnosi precoce. I sintomi iniziali sono infatti spesso vaghi e aspecifici, prevalentemente di tipo addominale o gastrointestinale, e possono includere gonfiore addominale, dolore, nausea, sazietà precoce, alterazioni dell'alvo e aumento della frequenza urinaria. Questa scarsa specificità ostacola il riconoscimento tempestivo della malattia e contribuisce al fatto che molte pazienti giungano all'osservazione in fase avanzata.

A questa difficoltà si aggiunge l'assenza di strategie di screening realmente efficaci nella popolazione generale. Sebbene nella pratica clinica possano essere utilizzati il dosaggio del CA125, l'ecografia e altre metodiche di imaging, tali strumenti non hanno consentito di risolvere in modo soddisfacente il problema dell'identificazione precoce del tumore. Ne deriva che l'HGSC viene diagnosticato molto frequentemente quando la diffusione oltre la sede di origine è già avvenuta, con un impatto diretto sulla prognosi e sulle possibilità di trattamento (Lisio et al., 2019).

Il percorso diagnostico si basa generalmente sull'integrazione tra valutazione clinica, esame obiettivo, imaging e conferma istopatologica. Nei casi sospetti, la paziente viene sottoposta a esami radiologici, a valutazione dei marcatori sierici e, successivamente, a procedure chirurgiche utili sia per ottenere materiale biotico sia per definire con maggiore precisione l'estensione della malattia. La stadiazione rappresenta infatti un momento cruciale, poiché consente di valutare il grado di disseminazione del tumore e di impostare la strategia terapeutica più appropriata (Lisio et al., 2019).

Dal punto di vista terapeutico, il trattamento dell'HGSC si fonda tradizionalmente sulla combinazione di chirurgia citoreduttrice e chemioterapia a base di platino, associata a taxani. La chirurgia di debulking ha come obiettivo la rimozione del maggior volume possibile di malattia macroscopicamente evidente, poiché il grado di citoreduzione ottenuto rappresenta uno dei principali determinanti prognostici. I migliori esiti si osservano nei casi in cui la resezione chirurgica riesce a lasciare un residuo tumorale minimo o assente. Nei casi in cui l'estensione della malattia o le condizioni cliniche della paziente non

consentano un approccio chirurgico immediato ottimale, può essere considerata una chemioterapia neoadiuvante seguita da chirurgia intervallata (Lisio et al., 2019).

Nonostante circa il 70% dei pazienti mostri spesso una buona risposta iniziale alla chemioterapia di prima linea, questo vantaggio è spesso solo temporaneo, la maggior parte delle pazienti inizialmente responsive al trattamento va successivamente incontro a recidiva. La ricomparsa della neoplasia può essere inizialmente asintomatica e spesso viene rilevata attraverso l'aumento dei marcatori sierici o mediante indagini radiologiche di follow-up.

La recidiva si accompagna progressivamente allo sviluppo di resistenza terapeutica, che costituisce uno dei principali ostacoli alla gestione a lungo termine dell'HGSC. In particolare, sebbene molte pazienti risultino inizialmente sensibili al platino, con il tempo una parte consistente dei tumori acquisisce una ridotta responsività ai trattamenti, rendendo sempre più difficile ottenere remissioni durature. Questo fenomeno contribuisce in modo decisivo alla prognosi sfavorevole della malattia e spiega perché l'HGSC venga considerato non solo un tumore aggressivo all'esordio, ma anche una neoplasia biologicamente capace di adattarsi alla pressione terapeutica (Lisio et al., 2019).

Negli ultimi anni, il panorama terapeutico dell'HGSC si è ampliato con l'introduzione di strategie mirate, in particolare in pazienti selezionate sulla base di specifiche caratteristiche molecolari, come i difetti della ricombinazione omologa o le alterazioni di BRCA1/BRCA2. Sebbene questi approcci abbiano contribuito a migliorare il controllo della malattia in determinati contesti, il problema della recidiva e della progressiva comparsa di resistenza rimane centrale nella gestione clinica.

Nel complesso, la storia clinica dell'HGSC è fortemente condizionata da tre elementi intrecciati: diagnosi tardiva, risposta iniziale al trattamento e successiva recidiva con sviluppo di resistenza. Questa complessità non riguarda soltanto la gestione clinica della malattia, ma si riflette anche nella varietà dei composti impiegati o studiati in questo contesto, rendendo utile affiancare all'inquadramento biologico una prospettiva di analisi molecolare e computazionale.

1.4 I composti come oggetto di analisi

Il quadro biologico e clinico delineato nei paragrafi precedenti mostra come il carcinoma ovarico, e in particolare l'HGSC, rappresenti una patologia complessa non solo per le sue caratteristiche istopatologiche e molecolari, ma anche per l'articolazione del contesto terapeutico entro cui viene trattata. Accanto alla rilevanza della diagnosi tardiva, della recidiva e della resistenza ai trattamenti, emerge infatti un ulteriore elemento di interesse: la presenza di un insieme di composti associati alla malattia che non costituisce un panorama omogeneo, ma comprende agenti differenti per natura farmacologica, bersaglio biologico e caratteristiche molecolari. In questa prospettiva, il carcinoma ovarico può essere considerato non soltanto come entità clinico-patologica, ma anche come contesto entro cui si colloca una pluralità di composti di interesse terapeutico e farmacologico.

Questa osservazione suggerisce di affiancare all'inquadramento biologico e clinico un'analisi centrata sull'insieme delle molecole associate alla patologia. L'obiettivo non è discutere l'efficacia dei singoli trattamenti, ma descrivere in modo sistematico le relazioni strutturali tra i composti e il grado di eterogeneità del panorama farmacologico.

Una simile impostazione è coerente con il ruolo che l'analisi computazionale ha progressivamente assunto nello studio dei composti bioattivi. La disponibilità di database strutturati consente infatti di raccogliere insiemi di molecole associate a uno specifico contesto patologico e di considerarli come un dataset organizzato, suscettibile di analisi comparativa. In questa prospettiva, l'approccio bioinformatico non si limita a facilitare la gestione dei dati, ma permette di osservare collezioni di composti che, pur essendo accomunate dalla medesima patologia sul piano clinico, possono risultare molto eterogenee sul piano molecolare (Zhou et al., 2024).

Nella letteratura chemioinformatica, la similarità molecolare rappresenta uno dei concetti centrali per organizzare e interpretare insiemi di composti, soprattutto quando si intende esplorare un dataset chimico in modo comparativo. Ciò non implica dedurre in modo automatico proprietà farmacologiche o cliniche a partire dalla sola struttura, ma utilizzare il confronto molecolare come criterio descrittivo per comprendere la composizione interna del dataset e la distribuzione della sua variabilità strutturale (López-Pérez et al., 2024).

Nel presente lavoro, l'HGSC costituisce il principale riferimento biologico-clinico per inquadrare la rilevanza del carcinoma ovarico. Tuttavia, ai fini dell'analisi computazionale,

l'oggetto dello studio è rappresentato da un insieme più ampio di composti associati al carcinoma ovarico, considerati come dataset molecolare da esplorare sul piano della similarità strutturale. In questa prospettiva, i composti non vengono analizzati in quanto singoli trattamenti, ma come insieme organizzato di molecole confrontabili, utile per descrivere l'eterogeneità del panorama terapeutico e individuare eventuali elementi strutturali condivisi.

2. Chemioinformatica

2.1 Chemioinformatica e spazio chimico

La chemioinformatica si è affermata come una disciplina che integra chimica, informatica e analisi dei dati per raccogliere, organizzare, gestire e interpretare l'informazione chimica.

In questo ambito, il termine “informazione chimica” comprende non solo le strutture molecolari, ma anche proprietà fisico-chimiche, spettri, dati biologici e attività farmacologiche associate ai composti. La rilevanza della chemioinformatica è cresciuta parallelamente all'espansione dei database chimici e biologici e alla necessità di trattare in modo sistematico quantità di dati che non risultano più analizzabili con approcci esclusivamente manuali o descrittivi (Chetry et al., 2025; Willett, 2020).

In questo senso, la chemioinformatica non coincide semplicemente con l'uso del computer in chimica, ma rappresenta un quadro metodologico specifico per la gestione e l'interpretazione dell'informazione molecolare. Una formulazione storicamente influente la definisce come l'integrazione di risorse informative impiegate per trasformare i dati in informazione e l'informazione in conoscenza, con l'obiettivo di prendere decisioni più rapide ed efficaci nel contesto dell'identificazione e dell'organizzazione di composti di interesse farmacologico. Questa definizione evidenzia due aspetti centrali della disciplina: da un lato il legame con la scienza dell'informazione, dall'altro la sua forte applicazione all'ambito del drug discovery (Bajorath, 2024).

La funzione della chemioinformatica diventa particolarmente evidente quando si considerano collezioni ampie e strutturalmente eterogenee di molecole di interesse biologico. In tali contesti, il contributo dell'analisi computazionale consiste nella possibilità di trattare grandi collezioni di composti in modo coerente, riproducibile e sistematico, favorendo l'identificazione di pattern, relazioni e proprietà che difficilmente emergerebbero da un'osservazione non assistita da strumenti informatici. In ambito farmacologico, questo approccio risulta particolarmente utile nelle fasi iniziali del drug discovery, dove la selezione, l'organizzazione e il confronto di molecole candidate costituiscono un passaggio essenziale.

Uno dei concetti più rilevanti all'interno della chemioinformatica è quello di spazio chimico (chemical space). In termini generali, esso può essere inteso come l'insieme delle molecole

possibili descritte all'interno di uno spazio multidimensionale definito da proprietà strutturali e funzionali. In questa prospettiva, le molecole non vengono considerate soltanto come entità isolate, ma come elementi collocabili in uno spazio di relazioni, nel quale la posizione relativa dipende dal tipo di rappresentazione adottata e dai descrittori utilizzati. Sebbene il concetto possa essere applicato a diverse classi di composti, esso è stato studiato soprattutto nel contesto delle piccole molecole organiche, in particolare per applicazioni di drug discovery (Medina-Franco et al., 2022).

La rilevanza del *chemical space* dipende dal fatto che il numero di molecole effettivamente sintetizzate e catalogate rappresenta soltanto una frazione minima dell'insieme teoricamente accessibile. Per questo motivo, la chemoinformatica svolge un ruolo fondamentale nel descrivere, visualizzare ed esplorare regioni dello spazio chimico troppo estese per essere affrontate con approcci tradizionali. La letteratura recente sottolinea come l'interesse per spazi chimici ampi e ultra-ampi sia cresciuto notevolmente, anche in relazione all'aumento delle librerie virtuali e delle collezioni di composti potenzialmente sintetizzabili. In tale contesto, lo studio dello spazio chimico diventa un modo per organizzare la diversità molecolare e orientare la ricerca verso regioni di maggiore interesse biologico o farmacologico (Medina-Franco et al., 2022).

Per la ricerca biologica e farmacologica, questa impostazione è particolarmente importante perché consente di collegare la diversità delle strutture molecolari alle loro possibili implicazioni funzionali. La chemoinformatica fornisce quindi il quadro teorico e operativo per rappresentare i composti in forma digitale, confrontarli tra loro e collocarli all'interno di un contesto più ampio di relazioni strutturali. In questo senso, essa costituisce una base concettuale essenziale per tutte quelle analisi che, come nel presente lavoro, non si limitano a descrivere i composti singolarmente, ma mirano a interpretarli come elementi di un insieme molecolare organizzato e confrontabile (Medina-Franco et al., 2022).

2.2 Rappresentazione computazionale della struttura molecolare

Affinché un sistema informatico possa elaborare, archiviare o confrontare entità chimiche, è necessario tradurre la struttura molecolare — tradizionalmente descritta come un grafo costituito da atomi e legami — in un linguaggio comprensibile alla macchina. Questo processo di traduzione richiede il passaggio da una rappresentazione concettuale della molecola a forme digitali formalizzate, adatte al trattamento computazionale. In ambito

chemioinformatico, le molecole possono infatti essere rappresentate attraverso diverse forme, tra cui notazioni lineari, fingerprint, identificatori standardizzati e grafi molecolari.

Il modo più immediato per rappresentare una molecola in formato digitale è costituito dalle notazioni lineari, che descrivono la topologia della struttura attraverso una singola stringa di caratteri alfanumerici. Queste notazioni consentono di codificare in forma compatta atomi, legami, ramificazioni e cicli, rendendo la struttura molecolare facilmente leggibile da parte del computer e rapidamente gestibile all'interno di database di grandi dimensioni.

Tra queste, il formato oggi più diffuso è lo SMILES (*Simplified Molecular Input Line Entry Specification*). In questo sistema, gli atomi sono rappresentati mediante i rispettivi simboli chimici, mentre i legami possono essere impliciti oppure espressi tramite simboli specifici, come nel caso del doppio legame. Le ramificazioni vengono indicate mediante parentesi tonde, mentre i cicli sono segnalati da numeri che identificano i punti di chiusura dell'anello. Grazie a questa sintassi, lo SMILES consente di trasformare una struttura chimica complessa in una sequenza lineare compatta, idonea all'archiviazione, alla ricerca e al confronto automatico.

(Weininger, 1988).

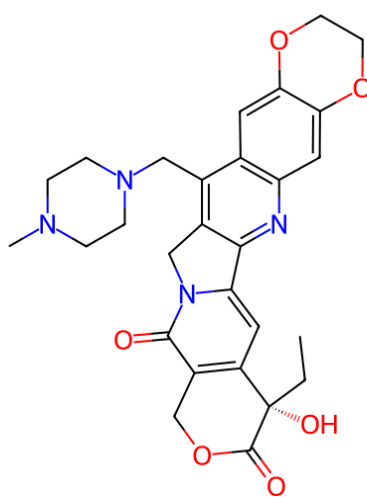
Dal punto di vista teorico, lo SMILES non costituisce soltanto una stringa di testo, ma rappresenta una formalizzazione della struttura molecolare fondata sull'applicazione della teoria dei grafi alla notazione chimica. Tale sistema interpreta la molecola come una struttura attraversabile secondo una scansione univoca, consentendo al calcolatore di ricostruirne la connettività in modo efficiente. Uno dei principali vantaggi di questa rappresentazione risiede proprio nella sua compattezza: rispetto alle tradizionali tabelle di connettività, lo SMILES permette di archiviare la medesima informazione strutturale con un ingombro notevolmente ridotto, favorendo la gestione rapida di grandi collezioni molecolari.

In ambito bioinformatico e chemioinformatico, questa proprietà rende lo SMILES particolarmente utile per l'indicizzazione, la ricerca e lo screening preliminare di database contenenti milioni di composti. Tuttavia, una stessa molecola può essere descritta da più stringhe SMILES differenti, a seconda del punto di partenza scelto per la scrittura o della convenzione adottata. Per superare questa ambiguità si ricorre allo SMILES canonico, cioè a una procedura algoritmica che assegna a ciascuna struttura una rappresentazione testuale univoca. (Weininger et al., 1989)

Nonostante la sua efficienza, lo SMILES presenta alcuni limiti intrinseci. Esso è particolarmente efficace per descrivere la connettività molecolare e, in parte, alcuni aspetti stereochimici, ma non è stato concepito per rappresentare in modo completo la geometria tridimensionale della molecola né per includere informazioni sperimentali complesse. Di conseguenza, pur risultando estremamente utile nelle fasi iniziali del trattamento computazionale delle molecole, può rivelarsi meno adatto quando l'analisi richiede una descrizione strutturale più esplicita e ricca di informazioni. In questo senso, la rappresentazione lineare costituisce soprattutto un punto di partenza per successive elaborazioni computazionali. (Weininger et al., 1989)

2.2.1 Il formato MOL

In questi casi assume particolare importanza il formato MOL, o Molfile, che costituisce una rappresentazione più esplicita della struttura chimica. A differenza delle notazioni lineari come lo SMILES, il formato MOL si basa sul concetto di Connection Table (CT), ossia una tabella di connettività che descrive la molecola come un grafo chimico in cui ogni atomo e ogni legame occupano una posizione definita all'interno di una struttura dati organizzata. In questo modo, la molecola non viene più rappresentata come una sequenza compatta di simboli, bensì come un insieme ordinato di informazioni strutturali esplicite. (Dalby, 1992)



Lurtotecan

```
CC[C@@]1(OC(=O)OCc2c1cc1n(c2=O)Cc2c-1nc1cc3c(cc1c2CN1CCN(C)CC1)OCCO3
```

SMILES della molecola Lurtotecan

Dal punto di vista organizzativo, un record MOL è articolato in più sezioni, ciascuna con una funzione specifica nell'elaborazione informatica della molecola. La Counts Line riporta il numero totale di atomi e di legami presenti nel record e fornisce al software le informazioni necessarie per interpretare correttamente la dimensione della struttura.

L'Atom Block consiste in una tabella in cui ogni riga corrisponde a un atomo e ne specifica le coordinate spaziali e l'identità chimica, permettendo così di rappresentare in modo esplicito la geometria molecolare. Il Bond Block, infine, definisce le connessioni tra le coppie di atomi, specificando il tipo di legame e, ove necessario, le informazioni stereochimiche associate. (Dalby, 1992 & CTfile)

L'importanza di questa architettura risiede nella sua capacità di fornire una descrizione strutturale esplicita della molecola, più ricca e immediatamente interpretabile rispetto alle notazioni lineari. Mentre formati come lo SMILES privilegiano compattezza e rapidità di gestione, il formato MOL offre una base più adatta per le analisi che richiedono una ricostruzione precisa della topologia e della geometria della struttura. Per questo motivo esso rappresenta un formato particolarmente rilevante nelle procedure di calcolo dei descrittori molecolari, nell'analisi della connettività e nell'identificazione di pattern strutturali complessi, come quelli ricercati mediante algoritmi di Maximum Common Substructure (MCS). In questo senso, il passaggio dalle notazioni lineari alle tabelle di connessione non rappresenta soltanto un cambiamento di formato, ma un diverso livello di descrizione computazionale della molecola, funzionale a tipi differenti di analisi.

2.2.2 Il Formato SDF

Definizione e Architettura del Formato

Il formato SDF (Structure-Data File) rappresenta uno degli standard più utilizzati per la memorizzazione e lo scambio di strutture chimiche di piccole molecole. La sua rilevanza risiede nel fatto che esso non si limita a descrivere la sola struttura molecolare, ma integra, accanto alla tabella di connessione, anche informazioni aggiuntive di natura sperimentale o calcolata. In questo senso, l'SDF può essere considerato un'estensione del formato MOL,

dal quale eredita integralmente la Connection Table, preservando quindi la descrizione esplicita degli atomi, dei legami e, quando disponibili, delle coordinate spaziali della molecola.(Dalby et al., 1992; MDL Information Systems, 1999)

```
Lurtotecan
      RDKit      2D

38 44  0  0  1  0  0  0  0  0999 V2000
    6.3349   -4.6106    0.0000 0  0  0  0  0  0  0  0  0  0  0  0  0
    [...]
    1 28  1  0
    [...]
M  END
>  <PUBCHEM_COMPOUND_CID>   (59)
60956

>  <PUBCHEM_COMPOUND_CANONICALIZED>   (59)
1

>  <PUBCHEM_CACTVS_COMPLEXITY>   (59)
1070

>  <PUBCHEM_CACTVS_HBOND_ACCEPTOR>   (59)
9

>  <PUBCHEM_CACTVS_HBOND_DONOR>   (59)
1

>  <PUBCHEM_CACTVS_ROTATABLE_BOND>   (59)
3

>  <PUBCHEM_CACTVS_SUBSKEYS>   (59)
AAADceB7uAAAAAAAAAAAAAAAAAAAAWAAAAA8WJECAAAAFix8AAAHgAACAAAEzhngYyzvMIFgCoAyX
yXAKCiCAhIiAAmCF+7NgNZvbE8JuWeCrm5hnL+Qew0PM0oEABAgACQABAgAIEAASAAAAAAAAAAAAA==

>  <PUBCHEM_IUPAC_OPENEYE_NAME>   (59)
(18S)-18-ethyl-18-hydroxy-2-[(4-methylpiperazin-1-yl)methyl]-6,9,20-trioxa-13,2
4-diazahexacyclo[12.11.0.03,12.05,10.015,24.017,22]pentacosa-1,3,5(10),11,13,15
,17(22)-heptaene-19,23-dione

>  <PUBCHEM_IUPAC_CAS_NAME>   (59)
(18S)-18-ethyl-18-hydroxy-2-[(4-methyl-1-piperazinyl)methyl]-6,9,20-trioxa-13,2
4-diazahexacyclo[12.11.0.03,12.05,10.015,24.017,22]pentacosa-1,3,5(10),11,13,15
,17(22)-heptaene-19,23-dione

>  <PUBCHEM_IUPAC_NAME_MARKUP>   (59)
(18<I>S</I>)-18-ethyl-18-hydroxy-2-[(4-methylpiperazin-1-yl)methyl]-6,9,20-trio
xa-13,24-diazahexacyclo[12.11.0.0<SUP>3,12</SUP>.0<SUP>5,10</SUP>.0<SUP>15,24</
SUP>.0<SUP>17,22</SUP>]pentacosa-1,3,5(10),11,13,15,17(22)-heptaene-19,23-dione

[...]
```

Esempio SDF

Rispetto al MOL, tuttavia, il formato SDF amplia in modo significativo le possibilità di rappresentazione del dato chimico. Esso consente infatti di associare alla struttura informazioni eterogenee, come identificatori di database, proprietà chimico-fisiche, risultati di saggi biologici o parametri derivanti da procedure computazionali. Questa caratteristica lo rende particolarmente adatto ai contesti bioinformatici e chemioinformatici, nei quali la struttura molecolare deve essere interpretata insieme al suo contenuto informativo accessorio.

Un ulteriore aspetto distintivo del SDF è la sua natura di formato multi-record. A differenza del file MOL, che descrive una singola molecola, un file SDF può contenere un numero molto elevato di strutture organizzate in sequenza all'interno dello stesso documento e separate dal delimitatore \$\$\$\$\$. Questa caratteristica lo rende particolarmente utile per la gestione di librerie molecolari, poiché consente di archiviare in un unico file intere collezioni di composti mantenendo, per ciascun record, sia la componente strutturale sia le annotazioni associate. (JRC QSAR Model Database, 2017)

Dal punto di vista organizzativo, il formato SDF mantiene la struttura gerarchica del MOL e la arricchisce con una sezione supplementare dedicata ai dati annotativi. In altri termini, ogni record SDF incorpora la struttura del Molfile — con le relative informazioni su atomi, legami e coordinate — e vi aggiunge un Data Block o Annotation Section, che rappresenta il principale elemento distintivo del formato. Questa sezione contiene campi testuali strutturati in forma di tag e valori associati, nei quali possono essere riportati dati sperimentali, quali valori di IC50, profili di tossicità, identificativi esterni come il PubChem CID, oppure proprietà fisico-chimiche ottenute per via computazionale.

Grazie a questa architettura, lo SDF rappresenta non solo una modalità di archiviazione delle strutture chimiche, ma anche un formato integrato per la gestione coordinata di struttura e informazione. In ambito teorico e applicativo, esso occupa quindi una posizione intermedia tra le rappresentazioni lineari, più compatte ma meno esplicite, e le strutture dati oggettuali utilizzate negli ambienti di analisi computazionale.

2.2.3 La Classe **SDFset**

La semplice disponibilità di file SDF, tuttavia, non è di per sé sufficiente per condurre analisi complesse su vaste librerie molecolari. Perché tali dati possano essere interrogati, validati e manipolati in modo sistematico, è necessario un ulteriore livello di astrazione che

trasformi il contenuto testuale del file in oggetti computazionali strutturati. Nell'ambiente di programmazione R, questa funzione è svolta, tra gli altri strumenti, dal package ChemmineR, che mette a disposizione la classe SDFset per l'accesso e la manipolazione delle informazioni contenute nei file SD/MOL.

(Cao et al. 2008; ChemmineR)

La classe SDFset consente di rappresentare una o più molecole importate da file SDF come una collezione di oggetti di classe SDF organizzati in una struttura di tipo list-like container. In questa forma, ciascuna molecola non è più trattata come semplice testo, ma come un'entità strutturata i cui componenti possono essere richiamati programmaticamente. La documentazione della classe indica infatti che l'oggetto contiene componenti dedicate alle informazioni strutturali e agli identificativi, e che diversi metodi di accesso permettono di recuperare separatamente atom block, bond block e data block. (ChemmineR Manual, SDFset-class)

Dal punto di vista operativo, questa astrazione risulta particolarmente importante per almeno tre ragioni. In primo luogo, garantisce modularità, poiché ogni molecola è organizzata in componenti distinte che possono essere richiamate e manipolate separatamente. In secondo luogo, permette la validazione strutturale del dato: durante l'importazione, funzioni dedicate possono identificare record corrotti, incompleti o chimicamente inconsistenti, migliorando l'affidabilità dell'analisi successiva. In terzo luogo, assicura una maggiore accessibilità computazionale, poiché rende possibile trattare dati atomici e topologici come matrici o strutture indicizzabili, facilitando il calcolo di descrittori molecolari, misure di similarità e procedure di ricerca per sottostruttura. (ChemmineR Manual)

In questo senso, la classe SDFset rappresenta un passaggio fondamentale tra il file chimico come formato di scambio e la molecola come oggetto computazionale analizzabile. La sua importanza non risiede soltanto nella comodità di accesso ai dati, ma nella possibilità di integrare rappresentazione strutturale, validazione e manipolazione algoritmica all'interno di un'unica struttura formale. Per questo motivo, strumenti di questo tipo rivestono un ruolo centrale nelle pipeline di chemoinformatica che richiedono l'elaborazione sistematica di collezioni molecolari estese.

2.3 Similarità molecolare

Il concetto di similarità molecolare costituisce uno dei fondamenti del virtual screening, una procedura computazionale in cui la somiglianza strutturale tra molecole viene tradotta in termini quantitativi al fine di ordinare i composti di una libreria rispetto a una struttura di riferimento. In questo contesto, la similarità non svolge soltanto una funzione descrittiva, ma rappresenta uno strumento operativo per prioritizzare molecole potenzialmente rilevanti nelle fasi iniziali del drug discovery. (Willett, 2003; Stumpfe & Bajorath, 2011).

La similarità molecolare può infatti essere intesa, in senso generale, come una misura quantitativa del grado di somiglianza tra due molecole, definita sulla base della loro rappresentazione strutturale e della metrica adottata per il confronto.

Come evidenziato nella letteratura del settore, questo approccio si basa sul Similar Property Principle, secondo il quale molecole strutturalmente simili tendono ad avere proprietà simili, incluse in molti casi proprietà fisico-chimiche o attività biologiche correlate. Si tratta tuttavia di un principio euristico e non assoluto, poiché la relazione tra similarità strutturale e similarità di attività può presentare eccezioni anche importanti. (O'Boyle et al., 2016; López-Pérez et al., 2024).

L'applicazione di questo principio richiede in genere la disponibilità di una struttura di riferimento nota, impiegata come query per interrogare database molecolari e identificare i composti più affini dal punto di vista strutturale. In questo contesto, l'efficacia della misura di similarità dipende dall'interazione di due elementi fondamentali: da un lato, la rappresentazione strutturale adottata per descrivere le molecole; dall'altro, il coefficiente di similarità utilizzato per quantificare il grado di somiglianza tra tali rappresentazioni. È quindi importante sottolineare che la similarità molecolare non costituisce una proprietà assoluta della molecola, ma dipende strettamente dal modo in cui la struttura viene codificata e dalla metrica scelta per il confronto. Di conseguenza, due molecole possono risultare più o meno simili a seconda della rappresentazione adottata, e la similarità deve essere interpretata come una relazione dipendente dal modello descrittivo impiegato, piuttosto che come un attributo intrinseco e invariabile della struttura chimica. (López-Pérez et al., 2024).

Nella maggior parte dei sistemi moderni, la rappresentazione strutturale è affidata ai fingerprint molecolari, cioè vettori che codificano la presenza o l'assenza di specifici frammenti o motivi sub-strutturali. Il confronto tra tali rappresentazioni viene comunemente effettuato mediante il coefficiente di Tanimoto, che costituisce una delle misure più utilizzate per il confronto di fingerprint binari e che, nella letteratura di riferimento, continua a essere indicato come metodo di scelta in molti contesti di similarity searching. (Willett, 2003).

L'interesse della similarità molecolare non risiede quindi soltanto nella quantificazione di una generica vicinanza chimica, ma nella possibilità di utilizzare il confronto strutturale come criterio per organizzare librerie molecolari, guidare procedure di screening e supportare l'identificazione di composti potenzialmente bioattivi. In questo senso, la similarità rappresenta una delle basi teoriche e operative della chemioinformatica contemporanea.

2.4 La rappresentazione digitale della struttura: Fingerprint molecolari

Per rendere possibile il confronto quantitativo tra molecole, è necessario convertire la struttura chimica in una rappresentazione computazionalmente trattabile. Un passaggio cruciale nel drug discovery computazionale consiste infatti nella traduzione della struttura chimica — tradizionalmente descritta come un grafo di atomi e legami — in una rappresentazione vettoriale che consenta un confronto rapido e quantitativo tra molecole diverse. Tale astrazione è realizzata attraverso i fingerprint molecolari, ossia descrittori digitali spesso espressi come vettori binari, nei quali viene codificata la presenza o l'assenza di specifiche caratteristiche strutturali all'interno della molecola. (Cereto-Massagué et al., 2015; Rogers & Hahn, 2010).

L'obiettivo principale di questa rappresentazione è duplice: da un lato ridurre la complessità computazionale del confronto tra grafi molecolari; dall'altro fornire una base quantitativa per la stima della somiglianza o della diversità chimica all'interno di ampie librerie di composti. In questo modo, i fingerprint permettono di convertire la struttura molecolare in una forma numerica adatta a procedure di similarità, clustering e selezione di composti. (Rogers & Hahn, 2010).

Tra le diverse metodologie di fingerprinting, i Morgan fingerprint, noti in letteratura anche come Extended-Connectivity Fingerprints (ECFP), rappresentano uno dei descrittori più utilizzati per la descrizione strutturale e la previsione dell'attività biologica. Questi descrittori derivano dall'algoritmo di Morgan del 1965, originariamente sviluppato per generare una descrizione univoca della struttura chimica, e sono stati successivamente adattati da Rogers e Hahn in forma di fingerprint circolari topologici per applicazioni di structure–activity modeling. (Morgan, 1965; Rogers & Hahn, 2010; Cereto-Massagué et al., 2015).

Natura circolare e algoritmo iterativo

La caratteristica distintiva dei Morgan fingerprint risiede nella loro natura circolare. A differenza dei fingerprint basati su percorsi lineari (path-based), che descrivono sequenze di atomi connesse lungo la molecola, l'algoritmo Morgan/ECFP esplora la struttura attraverso un processo iterativo che si espande radialmente a partire da ciascun centro atomico. (Cereto-Massagué et al., 2015; Rogers & Hahn, 2010).

In una fase iniziale, a ogni atomo viene assegnato un identificatore numerico basato su proprietà chimiche intrinseche. Nel caso degli ECFP standard, tali proprietà comprendono il numero di vicini pesanti, la valenza corretta per gli idrogeni, il numero atomico, la massa atomica, la carica atomica, il numero di idrogeni legati e l'appartenenza ad almeno un anello. Questi valori vengono poi combinati mediante hashing in un identificatore iniziale.

Successivamente, attraverso una serie di iterazioni, ciascun atomo incorpora progressivamente l'informazione relativa ai propri vicini, così che il suo identificatore finisca per rappresentare un intorno chimico via via più ampio, come un aggiornamento iterativo che genera, per ogni atomo, sottostrutture circolari sempre più estese.

Nel caso dei fingerprint Morgan, il parametro fondamentale è il raggio, che definisce l'estensione dell'intorno atomico considerato. Nella nomenclatura ECFP, il numero che segue il nome del fingerprint corrisponde al diametro effettivo della sottostruttura più grande rappresentata; di conseguenza, un raggio pari a 2 corrisponde convenzionalmente a ECFP4. Questa configurazione è tra le più utilizzate nella letteratura applicativa. (Rogers & Hahn, 2010; Cereto-Massagué et al., 2015).

Un aspetto particolarmente importante dei Morgan fingerprint è la loro natura contestuale. A differenza della ricerca per sottostruttura classica, che si limita a verificare la presenza o

l'assenza di un determinato frammento molecolare, i fingerprint Morgan descrivono il contesto chimico locale in cui quel frammento è inserito. In altre parole, non codificano semplicemente un motivo strutturale isolato, ma l'ambiente atomico che lo circonda, tenendo conto della connettività e dello stato di sostituzione. (Rogers & Hahn, 2010).

Dal punto di vista algoritmico, l'identificatore assegnato a ciascun atomo dipende infatti da proprietà che riflettono la sua connettività e la sua valenza, incluse nelle implementazioni standard anche informazioni sugli idrogeni legati. Di conseguenza, una modifica dello stato di sostituzione altera l'ambiente atomico codificato, producendo identificatori differenti e, in una fingerprint binaria finale, l'attivazione di bit diversi. Questo rende i Morgan fingerprint particolarmente adatti a catturare differenze strutturali anche sottili, ma potenzialmente rilevanti dal punto di vista chimico e biologico. (Rogers & Hahn, 2010).

Tuttavia, come tutti i fingerprint topologici, anche i Morgan fingerprint forniscono una rappresentazione semplificata della struttura molecolare e non descrivono in modo esplicito aspetti conformazionali o tridimensionali. (Cereto-Massagué et al., 2015).

2.5 Coefficiente di Tanimoto: la matrice di similarità

Una volta ottenuta la rappresentazione delle molecole sotto forma di vettori binari, come nel caso dei Morgan fingerprint, la determinazione della loro somiglianza strutturale richiede l'applicazione di una metrica quantitativa appropriata. In ambito chemioinformatico, il metodo più comunemente utilizzato per il confronto tra fingerprint binari è il coefficiente di Tanimoto.

Il coefficiente di Tanimoto consente di quantificare il grado di sovrapposizione tra due vettori binari, indicati come A e B, attraverso il rapporto tra il numero di caratteristiche condivise e il numero totale delle caratteristiche presenti in almeno uno dei due vettori. Dal punto di vista matematico, esso è espresso dalla formula:

$$T(A, B) = \frac{c}{a+b-c}$$

dove:

- a rappresenta il numero di bit impostati a "1" nel fingerprint della molecola A;
- b rappresenta il numero di bit impostati a "1" nel fingerprint della molecola B;

- c rappresenta il numero di bit comuni ad entrambi i vettori (bit attivati nella medesima posizione).

Il valore del coefficiente oscilla in un intervallo compreso tra 0, che indica assenza di caratteristiche condivise, a 1, che corrisponde all'identità delle rappresentazioni digitali considerate. In termini interpretativi, un valore elevato di Tanimoto suggerisce una maggiore similarità strutturale tra due molecole e, in accordo con il Similar Property Principle, può indicare una più alta probabilità di condividere proprietà chimico-fisiche o biologiche simili (Willett, 2006).

La diffusione del coefficiente di Tanimoto in chemoinformatica dipende dalla sua particolare adeguatezza al trattamento dei fingerprint molecolari, che sono tipicamente rappresentazioni sparse. In vettori di lunghezza pari a 1024 o 2048 bit, infatti, solo una piccola frazione delle posizioni risulta generalmente attivata, mentre la grande maggioranza è costituita da zeri. In questo contesto, metriche simmetriche come la distanza di Hamming attribuirebbero lo stesso peso sia alle corrispondenze tra bit attivi sia alle corrispondenze tra bit inattivi. Tuttavia, dal punto di vista chemioinformatico, la condivisione di assenze strutturali non possiede lo stesso significato della condivisione di caratteristiche presenti. Due molecole non possono infatti essere considerate simili soltanto perché entrambe non contengono un elevato numero di possibili frammenti chimici.

Il coefficiente di Tanimoto supera questa limitazione poiché, nel confronto tra fingerprint binari, attribuisce rilevanza alle caratteristiche effettivamente presenti, trascurando le corrispondenze 0–0. In questo modo, la misura di similarità riflette principalmente la condivisione di frammenti strutturali e gruppi funzionali, riducendo il rumore informativo derivante dalle assenze comuni.

2.5.1 Dalla Similarità alla Matrice di Distanza

Per l'applicazione di algoritmi di clustering o di riduzione della dimensionalità, il coefficiente di similarità (S) viene spesso convertito in una misura di distanza (D), calcolata come il suo complemento a uno:

$$D(A, B) = 1 - T(A, B)$$

Questa trasformazione consente di passare da una misura di somiglianza a una misura di dissimilarità, più adatta a molte procedure di analisi multivariata. Applicata all'intero

insieme di molecole, essa permette di costruire una matrice di distanza, che descrive in forma relazionale i rapporti tra tutte le coppie di composti del dataset. Tale matrice costituisce la base per successive analisi di clustering e per la proiezione dello spazio chimico in dimensioni ridotte, consentendo di evidenziare gruppi di molecole caratterizzati da maggiore prossimità strutturale.

2.6 Proiezione bidimensionale e analisi dei cluster con MDS e clustering k-means

L'analisi di insiemi molecolari ad alta dimensionalità richiede spesso strumenti capaci di rappresentare in modo sintetico le relazioni di similarità o dissimilarità tra composti. In chemoinformatica, tali relazioni sono comunemente espresse mediante fingerprint molecolari e misure di similarità, che consentono di descrivere lo spazio chimico in termini quantitativi. Tuttavia, la complessità e la dimensionalità di queste rappresentazioni rendono difficile una loro interpretazione diretta; per questo motivo, si ricorre frequentemente a tecniche di riduzione dimensionale, volte a proiettare i dati in uno spazio a due o tre dimensioni mantenendo, per quanto possibile, la struttura relazionale originaria.

Tra questi approcci, il Multidimensional Scaling (MDS) rappresenta una strategia particolarmente adatta quando i dati di partenza sono disponibili direttamente sotto forma di matrice di distanze. Con il termine *embedding* si intende una rappresentazione dei dati in uno spazio a dimensionalità ridotta che cerca di preservare le relazioni di distanza o vicinanza presenti nello spazio originale. Nel caso dell'MDS, la proiezione viene costruita in modo da mantenere, per quanto possibile, la configurazione relativa delle distanze tra gli oggetti nello spazio ridotto, rendendo il metodo utile per l'esplorazione visiva dello spazio chimico e per la preparazione di successive procedure di clustering (Hout et al., 2013; Lespinats et al., 2011).

A livello concettuale, l'MDS è particolarmente coerente con workflow nei quali il confronto tra molecole è espresso tramite una matrice di dissimilarità derivata dal coefficiente di Tanimoto. In questi casi, la tecnica consente di ottenere una rappresentazione bidimensionale dei composti senza richiedere il passaggio a descrittori ulteriormente trasformati, favorendo una lettura più immediata delle relazioni di prossimità e separazione tra gruppi molecolari. Rispetto al Sammon mapping, che rimane una possibile alternativa, l'MDS rappresenta inoltre un approccio più standard alla proiezione

di matrici di dissimilarità, mentre il Sammon mapping può essere interpretato come una variante non lineare che attribuisce maggiore peso alla conservazione delle distanze piccole (Hout et al., 2013; Lespinats et al., 2011).

È tuttavia opportuno sottolineare che l'MDS costituisce una tecnica di riduzione dimensionale e non, in senso stretto, un metodo di clustering. La sua funzione principale è infatti quella di fornire una proiezione semplificata delle relazioni di distanza tra i composti, facilitando l'individuazione visiva di eventuali raggruppamenti. Come tutte le tecniche di embedding, anche l'MDS comporta una perdita parziale di informazione, poiché la proiezione bidimensionale non può preservare integralmente tutte le relazioni presenti nello spazio originario (Hout et al., 2013).

Per la suddivisione formale dei dati in gruppi distinti può invece essere impiegato l'algoritmo K-means, uno dei metodi di partizionamento più noti nell'ambito dell'analisi dei cluster. L'obiettivo dell'algoritmo è ripartire un insieme di osservazioni in k cluster mutuamente esclusivi, assegnando ciascun elemento al gruppo appropriato sulla base della vicinanza al centro del cluster e aggiornando iterativamente tali centri nel corso del processo di ottimizzazione.

Dal punto di vista concettuale, K-means mira quindi a ridurre la variabilità interna ai cluster e a favorire la separazione tra gruppi differenti (MacQueen, 1967).

In chemoinformatica, questo approccio risulta utile per organizzare lo spazio chimico in insiemi di composti strutturalmente affini, facilitando l'identificazione di classi molecolari, la selezione di composti rappresentativi e l'interpretazione di possibili relazioni tra struttura e attività biologica. Quando applicato a una rappresentazione bidimensionale ottenuta mediante MDS, K-means consente di trasformare una distribuzione continua di punti in una suddivisione discreta in cluster, rendendo più sistematica l'analisi dei pattern osservati (MacQueen, 1967).

L'impiego combinato di MDS e K-means permette dunque di integrare due livelli complementari di analisi: da un lato, la rappresentazione bidimensionale dello spazio chimico, utile per la sua esplorazione visiva; dall'altro, la definizione di gruppi molecolari mediante una procedura di clustering formale. È tuttavia necessario considerare alcuni limiti di questo approccio: il risultato del clustering dipende dal numero di cluster scelto a priori, dalla rappresentazione dei dati e dalla qualità della proiezione bidimensionale; inoltre, la riduzione dimensionale può influenzare la geometria relativa dei punti, con

possibili effetti sulla separazione tra i gruppi identificati (MacQueen, 1967).

2.7 Identificazione del core strutturale: algoritmo MCS

L'algoritmo di Maximum Common Substructure (MCS) costituisce uno degli strumenti più rilevanti della chemoinformatica per l'identificazione di motivi strutturali condivisi tra composti diversi. Basato sulla rappresentazione delle molecole come grafi, in cui i vertici corrispondono agli atomi e gli archi ai legami chimici, l'MCS consente di individuare la più ampia sottostruttura comune a due o più molecole, fornendo così una misura della similarità strutturale a livello locale.

Dal punto di vista teorico, il termine MCS comprende due principali formulazioni matematiche del problema: il Maximum Common Induced Subgraph (MCIS), che privilegia la corrispondenza tra vertici e richiede che le relazioni di adiacenza tra gli atomi comuni siano conservate, e il Maximum Common Edge Subgraph (MCES), che si concentra invece sulla massimizzazione del numero di legami condivisi tra due strutture. In molti contesti chemioinformatici, il modello MCES risulta particolarmente utile, poiché preserva in modo più diretto la connettività dei legami covalenti, elemento fondamentale per la descrizione dell'architettura molecolare e, potenzialmente, della funzione biologica del composto. (Raymond & Willett, 2002)

L'impiego dell'MCS risulta particolarmente vantaggioso quando i metodi di confronto globale, come il coefficiente di Tanimoto calcolato su fingerprint molecolari, non riescono a descrivere in modo soddisfacente la relazione tra composti strutturalmente eterogenei. In questi casi, molecole con dimensioni differenti ma accomunate da un medesimo nucleo strutturale possono mostrare una similarità globale modesta, pur condividendo una porzione molecolare cruciale per l'interazione con il bersaglio biologico. L'analisi MCS permette quindi di spostare l'attenzione dalla molecola nella sua interezza a specifici pattern strutturali locali, evidenziando il core molecolare condiviso o scaffold comune. (Raymond & Willett, 2002)

Dal punto di vista applicativo, l'identificazione di una sottostruttura comune può supportare studi di relazione struttura-attività (SAR), il raggruppamento di composti appartenenti alla stessa famiglia chimica e l'interpretazione di similarità funzionali tra ligandi. Tuttavia, l'MCS presenta anche alcuni limiti: il risultato dipende dai criteri adottati

per il matching di atomi e legami, il problema è computazionalmente oneroso e la sottostruttura massima individuata non coincide necessariamente con la porzione strutturalmente o farmacologicamente più rilevante. Inoltre, l'analisi MCS descrive principalmente la similarità topologica della struttura molecolare e non tiene conto in modo esplicito di aspetti tridimensionali, conformazionali, stereochimici o elettronici, che possono contribuire in maniera determinante all'attività biologica.

Nel complesso, l'algoritmo MCS rappresenta quindi una strategia utile per identificare un pattern strutturale condiviso tra composti chimicamente correlati, soprattutto quando l'obiettivo è riconoscere elementi conservati che possano giustificare analogie di comportamento biologico o farmacologico.
(Raymond & Willett, 2002).

2.8 Scaffold di Bemis-Murcko

Un concetto centrale nell'analisi strutturale delle molecole bioattive è quello di molecular framework, formalizzato da Bemis e Murcko nel 1996. In questo approccio, la molecola viene scomposta nei suoi elementi strutturali principali, distinguendo tra sistemi ad anello, linker e catene laterali. Il Bemis–Murcko scaffold corrisponde al nucleo formato dall'unione dei sistemi ad anello e dei linker, mentre i sostituenti periferici vengono rimossi. In questo modo, la rappresentazione risultante conserva l'architettura centrale della molecola e consente di descriverne il “core” strutturale in forma semplificata ma chimicamente significativa. (Bemis & Murcko, 1996)

L'interesse teorico di questa rappresentazione risiede nel fatto che essa permette di ridurre la complessità della struttura molecolare senza perdere l'informazione relativa alla sua organizzazione fondamentale. Il ricorso ai Murcko scaffolds è particolarmente utile nei contesti chemioinformatici in cui sia necessario confrontare insiemi di molecole non tanto sulla base dei singoli sostituenti, quanto sul piano del loro nucleo strutturale condiviso. In questo senso, il framework molecolare può essere considerato una rappresentazione più stabile e più generale rispetto alla molecola completa, particolarmente adatta all'analisi della diversità chimica e alla classificazione di librerie di composti.

Murcko framework costituisce una rappresentazione di scaffold oggettiva e non dipendente dal dataset, poiché viene derivata dalla singola molecola e non dal confronto tra gruppi di composti. Questa caratteristica lo distingue dalla Maximum Common Substructure (MCS),

il cui risultato dipende invece dall'insieme di molecole considerate e dall'eventuale clustering preliminare. Per tale ragione, i Murcko scaffolds possono essere impiegati come livello intermedio di astrazione strutturale, utile a isolare il nucleo centrale della molecola prima di procedere a confronti più specifici tra sottostrutture comuni. (Bemis & Murcko, 1996; Langdon et al., 2011)

In ambito applicativo, i Murcko scaffolds risultano particolarmente utili per descrivere e confrontare la scaffold diversity di collezioni di composti, permettendo di valutare quante strutture condividano lo stesso framework centrale e quanto una libreria sia concentrata attorno a pochi nuclei ricorrenti oppure distribuita su scaffold più eterogenei. In questo senso, essi costituiscono uno strumento efficace per organizzare e interpretare lo spazio strutturale delle piccole molecole in modo più astratto rispetto alla singola struttura completa. (Langdon et al., 2011).

3. Metodi

Il presente capitolo descrive i dati, gli strumenti e le procedure utilizzati per costruire il dataset di composti associati al carcinoma ovarico e per analizzarne le relazioni strutturali. Dopo aver definito l'obiettivo del lavoro e le risorse software impiegate, vengono illustrati il workflow sviluppato in R per la costruzione e la prima esplorazione del dataset e la pipeline complementare implementata in Python per il clustering e l'identificazione delle sottostrutture comuni.

3.1 Obiettivo del lavoro

L'obiettivo del presente studio è stato lo sviluppo di un workflow computazionale integrato per l'analisi strutturale di composti di interesse farmacologico associati al carcinoma ovarico. In particolare, il lavoro è stato finalizzato alla selezione di molecole rilevanti in questo contesto patologico, alla loro organizzazione in un dataset strutturato e alla successiva analisi delle relazioni di similarità tra composti, con l'intento di individuare eventuali motivi strutturali comuni.

Più nello specifico, il progetto ha previsto una serie di passaggi consecutivi: identificazione dei composti di interesse a partire da database specializzati, recupero e standardizzazione delle strutture molecolari, filtraggio dei record non idonei all'analisi, confronto tra molecole mediante approcci chemioinformatici e raggruppamento dei composti in insiemi strutturalmente affini. Su tali gruppi è stata quindi effettuata una ricerca di sottostrutture condivise, al fine di evidenziare nuclei molecolari ricorrenti potenzialmente rilevanti dal punto di vista strutturale.

Un ulteriore obiettivo del lavoro è stato quello di costruire una procedura ripercorribile, capace di trasformare dati provenienti da risorse farmacologiche e chimico-strutturali in un insieme coerente e analizzabile. In coerenza con il quadro biologico-clinico delineato nel capitolo precedente, l'HGSC è stato assunto come principale riferimento per contestualizzare la rilevanza del carcinoma ovarico; sul piano operativo, tuttavia, l'analisi è stata condotta su un insieme più ampio di composti associati al carcinoma ovarico, trattati come dataset molecolare da esplorare in termini di similarità strutturale.

Nel complesso, lo studio si inserisce nell'ambito delle applicazioni computazionali all'analisi di molecole bioattive, con particolare attenzione all'identificazione di relazioni

strutturali tra composti e alla definizione di elementi chimici condivisi utili per la loro interpretazione. Le sezioni che seguono descrivono quindi il percorso con cui i dati iniziali sono stati progressivamente trasformati in un dataset strutturale analizzabile.

3.2 Linguaggi di programmazione e strumenti software

Per lo sviluppo del presente lavoro sono stati impiegati i linguaggi di programmazione R e Python, utilizzati in fasi complementari del workflow. Entrambi sono linguaggi open source ampiamente impiegati nella ricerca scientifica e dispongono di librerie dedicate alla manipolazione dei dati, alla costruzione di pipeline riproducibili e all'implementazione di procedure specialistiche.

Nel progetto, R è stato impiegato soprattutto nelle fasi di parsing dei file TTD, filtro del dataset, interrogazione iniziale di PubChem, gestione dei file SDF, calcolo della similarità tramite *fmcsR* e rappresentazione esplorativa mediante PCoA. I pacchetti *ChemmineR* e *fmcsR* hanno fornito le funzioni necessarie per leggere oggetti *SDFset*, confrontare le strutture e manipolare gli output intermedi; l'ambiente di lavoro è stato inoltre completato con pacchetti provenienti da CRAN, Bioconductor e GitHub (R Core Team; *PubChemR*; *ChemmineR*; *fmcsR*).

Python è stato invece impiegato per una pipeline complementare di clustering e identificazione della MCS. In questo contesto, *RDKit* ha consentito di operare direttamente su oggetti molecolari, calcolare Morgan fingerprints, derivare le distanze di Tanimoto, costruire i cluster e ricavare scaffold di Bemis-Murcko e Maximum Common Substructure all'interno dello stesso ambiente. Nella pipeline Python, la riduzione dimensionale è stata impostata preferenzialmente mediante MDS sulla matrice di distanze precomputata, mentre Sammon mapping è rimasto solo come opzione alternativa disponibile nel codice, in coerenza con la letteratura che descrive l'MDS come approccio standard alla rappresentazione di matrici di dissimilarità e il Sammon mapping come variante non lineare più focalizzata sulla preservazione delle distanze piccole (Hout et al., 2013; Lespinats et al., 2011). Nel progetto Python le dipendenze sono raccolte nel file `'requirements.txt'`, installabile mediante `'pip'`, rendendo più agevole la preparazione dell'ambiente di lavoro (Python Software Foundation; *PubChemPy*; *pandas*; *NumPy*; *RDKit*).

L'utilizzo integrato di R e Python ha quindi rappresentato una scelta metodologica coerente

con la natura del progetto. R ha fornito l'ambiente principale per la costruzione del dataset e per la prima esplorazione delle relazioni strutturali; Python è stato impiegato come implementazione complementare per il clustering e la ricerca delle sottostrutture comuni. Il confronto tra i due approcci ha pertanto un valore metodologico generale, pur non configurandosi come un confronto perfettamente uno-a-uno sul piano del dataset iniziale.

3.2.1 TTD

Per l'identificazione dei composti di interesse è stato utilizzato il Therapeutic Target Database (TTD), una banca dati pubblica sviluppata per raccogliere informazioni sui target terapeutici descritti in letteratura e sulle relative associazioni con patologie, pathway biologici e farmaci o ligandi diretti contro tali bersagli.

TTD è stato progettato come una risorsa integrata, in grado di collegare i target terapeutici a informazioni di interesse farmacologico e biologico, includendo anche rimandi ad altre banche dati utili per approfondire aspetti quali sequenza, struttura tridimensionale, funzione. (Wang et al., 2020).

Nel contesto del presente studio, TTD è stato impiegato come fonte primaria per la selezione dei composti associati al carcinoma ovarico. In particolare, l'estrazione dei dati è stata effettuata a partire dal file: "Drug to disease mapping with ICD identifiers" ([P1-05-Drug_disease.txt](#)), contenente per ciascun farmaco il nome del composto, un identificativo interno del database e le relative indicazioni terapeutiche associate. La sua utilità risiede nel fatto che non si limita a fornire una descrizione chimica delle molecole, ma le colloca all'interno di un contesto terapeutico più ampio, mettendole in relazione con i rispettivi bersagli biologici e con le condizioni patologiche di interesse. In questo modo, il database ha rappresentato il punto di partenza per la costruzione del dataset analizzato, consentendo di individuare in modo mirato i composti di interesse terapeutico per il contesto patologico considerato.

L'impiego di TTD in questa fase del workflow è risultato quindi particolarmente utile per collegare l'informazione farmacologica e biologica alla successiva analisi strutturale dei composti. Tale approccio ha permesso di selezionare le molecole non soltanto sulla base della loro identità chimica, ma anche in funzione della loro rilevanza terapeutica, rendendo il dataset di partenza più coerente con gli obiettivi bioinformatici e chemioinformatici del progetto. (Wang et al., 2020).

3.2.2 PubChem

Come risorsa di supporto per il controllo e il recupero delle informazioni chimiche è stato utilizzato PubChem, un database pubblico di informazione chimica sviluppato e mantenuto dal National Center for Biotechnology Information (NCBI). PubChem raccoglie e organizza dati relativi a sostanze e composti chimici, mettendo a disposizione informazioni su strutture molecolari, identificativi, sinonimi, proprietà chimico-fisiche, bioattività e collegamenti ad altre risorse biologiche e farmacologiche. Per queste caratteristiche, esso rappresenta una delle principali banche dati di riferimento per la chemioinformatica e per la gestione computazionale di dati molecolari. (Kim et al., 2023)

Nel presente studio, PubChem non è stato utilizzato come fonte primaria per la selezione dei composti, ma come risorsa di supporto e verifica rispetto ai dati inizialmente estratti da TTD. In particolare, il database è risultato utile per controllare l'identità dei composti selezionati, recuperare le relative informazioni strutturali e ottenere rappresentazioni molecolari standardizzate, necessarie per le successive analisi computazionali. Questo aspetto è particolarmente importante in workflow chemioinformatici come quello sviluppato nel presente lavoro, nei quali l'affidabilità della rappresentazione strutturale costituisce un passaggio essenziale per il confronto tra molecole.

L'utilità di PubChem nel contesto di questo progetto risiede quindi nella sua funzione di collegamento tra il livello farmacologico e terapeutico dei dati, ricavato da TTD, e il livello chimico-strutturale necessario per l'analisi molecolare. Mentre TTD ha consentito di individuare i composti rilevanti per il carcinoma ovarico, PubChem ha fornito un supporto fondamentale per il recupero e la verifica delle informazioni chimiche associate a tali molecole, contribuendo alla costruzione di un dataset più coerente e adatto alle fasi successive del workflow. (Kim et al., 2023)

3.3 Workflow computazionale in R

3.3.1 Selezione dei composti da TTD e filtro per carcinoma ovarico

La costruzione del dataset iniziale è stata effettuata a partire dal file "Drug to disease mapping with ICD identifiers" del Therapeutic Target Database (TTD), una risorsa sviluppata per collegare target terapeutici, patologie e farmaci o ligandi associati.

Il file utilizzato nel presente studio contiene una mappatura tra farmaci e indicazioni patologiche, organizzata in record testuali nei quali compaiono, per ciascun composto, un identificativo interno del database (TTDDRUID), il nome del farmaco (DRUGNAME) e una o più voci di indicazione terapeutica (INDICATI). Ogni indicazione è riportata insieme al relativo codice ICD-11 e allo stato clinico del composto, ad esempio Clinical trial o specifiche fasi di sviluppo come Phase 1. Il file mostra inoltre che uno stesso farmaco può comparire associato a più indicazioni patologiche, ciascuna riportata in righe successive dello stesso record.

In una prima fase, il file testuale grezzo è stato sottoposto a una procedura di parsing finalizzata a estrarre e organizzare in forma tabellare le informazioni rilevanti per l'analisi. In particolare, sono stati isolati l'identificativo del composto, il nome del farmaco, la patologia associata, il codice ICD-11 e lo stato clinico, così da ottenere un dataset strutturato in cui ogni riga corrispondesse a una specifica associazione tra farmaco e indicazione terapeutica. Applicando questa procedura all'intero file sono state ottenute 30.314 associazioni farmaco-indicazione, corrispondenti a 23.718 farmaci distinti. Questa trasformazione è risultata necessaria per rendere il contenuto del file interrogabile e filtrabile in modo sistematico nelle fasi successive del workflow.

Successivamente, il dataset così costruito è stato filtrato per selezionare esclusivamente i composti associati al carcinoma ovarico. Tale selezione è stata effettuata considerando sia la presenza di termini riconducibili alla patologia nel campo dell'indicazione terapeutica, sia le informazioni diagnostiche associate attraverso la classificazione ICD-11- 2C73. In questo modo è stato possibile identificare in modo mirato i farmaci rilevanti per il contesto patologico di interesse, mantenendo al tempo stesso l'informazione sul loro stato clinico e sulle relative indicazioni terapeutiche. Il dataset è stato ridotto a 207 record pertinenti, corrispondenti a 203 farmaci distinti. Tra questi, il sottoinsieme dei composti classificati come approved comprendeva 12 record, corrispondenti a 12 farmaci distinti.

Al termine di questa fase sono state ottenute una versione dettagliata del dataset, contenente tutte le associazioni pertinenti al carcinoma ovarico, e una versione aggregata a livello di farmaco, utilizzata come base per i passaggi successivi del workflow. Questo approccio ha consentito di trasformare un file testuale grezzo proveniente da TTD in un insieme di dati strutturato, coerente e adeguato alle successive analisi bioinformatiche e chemioinformatiche.

3.3.2 Recupero delle strutture da PubChem

Dopo la selezione dei composti associati al carcinoma ovarico, è stata affrontata la fase di recupero delle corrispondenti strutture molecolari in formato SDF, necessarie per le successive analisi chemioinformatiche. In un primo momento, tale recupero è stato tentato a partire dal file `P3-01-All.sdf` scaricato da TTD, che raccoglie le strutture molecolari rese disponibili dal database. Tuttavia, il confronto tra questo archivio strutturale e l'elenco dei farmaci selezionati nel passaggio precedente non ha consentito di ottenere una copertura completa del dataset di interesse: tra i composti associati al carcinoma ovarico, infatti, è stato possibile identificare direttamente solo 64 molecole. Per questo motivo si è reso necessario introdurre una fase aggiuntiva di interrogazione di PubChem, con lo scopo di ampliare il recupero delle strutture mancanti e costruire un insieme di dati più completo.

L'accesso a PubChem è stato effettuato in ambiente R mediante il pacchetto `PubChemR`, utilizzando come file di input `ttd_drug_disease_ovarian_by_drug.csv`, ottenuto nelle fasi precedenti e contenente l'elenco aggregato dei farmaci associati al carcinoma ovarico. Per ciascun composto, il nome del farmaco è stato utilizzato come chiave di interrogazione del database, con l'obiettivo di recuperare la relativa struttura molecolare in formato SDF. La procedura è stata impostata in modo sequenziale: per ogni farmaco, la ricerca è stata eseguita inizialmente nel dominio `compound`, che contiene strutture chimiche standardizzate, e successivamente, nei casi in cui non fosse disponibile un record utile, nel dominio `substance`, che contiene record inviati dai diversi contributori di PubChem e non sempre corrispondenti a una struttura standardizzata univoca. Questa doppia interrogazione è stata adottata per aumentare la probabilità di recuperare una rappresentazione strutturale utilizzabile anche per composti non immediatamente reperibili nel solo dominio `compound`.

Dal punto di vista operativo, i principali file di input di questa fase sono stati quindi: il file `ttd_drug_disease_ovarian_by_drug.csv`, contenente l'elenco dei composti da interrogare, e il file `P3-01-All.sdf`, utilizzato come fonte strutturale preliminare ma non sufficiente a coprire l'intero dataset. Gli output prodotti sono stati invece i singoli file SDF recuperati da PubChem, salvati localmente in cartelle distinte in base al dominio di provenienza (`compound` o `substance`), e il file riepilogativo `sdf_results.csv`, nel quale per ciascun farmaco sono stati registrati il nome del composto, il percorso del file SDF recuperato, il dominio di provenienza e la relativa categoria di approvazione.

Complessivamente, tramite questa procedura è stato possibile recuperare strutture molecolari per 128 dei 203 composti considerati; di queste, 68 provenivano dal dominio compound e 60 del dominio substance, mentre per i restanti 75 composti non è stato ottenuto alcun file strutturale utile.

Nei casi in cui non fosse possibile ottenere una struttura da nessuno dei due domini, il composto è stato comunque mantenuto nel file finale, lasciando mancanti i campi relativi al percorso del file e al dominio, così da conservare traccia dei recuperi non andati a buon fine.

L'interrogazione di PubChem ha quindi svolto un ruolo metodologicamente importante all'interno del workflow: non come fonte primaria per la selezione dei composti, ma come risorsa di supporto per integrare e verificare le informazioni strutturali necessarie alle analisi successive. Questo passaggio ha consentito di superare la copertura parziale del file strutturale inizialmente disponibile in TTD, rendendo il dataset più adatto alle successive fasi di unione, pulizia e confronto delle molecole. Il ricorso a TTD come fonte iniziale e a PubChem come supporto strutturale è coerente con il ruolo dei due database: TTD è infatti orientato al collegamento tra target terapeutici, patologie e farmaci, mentre PubChem fornisce un accesso esteso a informazioni chimico-strutturali standardizzate.

3.3.3 Unione dei file strutturali e filtraggio delle molecole

Dopo il recupero delle strutture molecolari da PubChem, è stato necessario riunire in un unico archivio i file SDF ottenuti, così da predisporre un dataset strutturale omogeneo per le successive analisi. A questo scopo è stato utilizzato come file di input `sdf_results.csv`, prodotto nella fase precedente, contenente per ciascun farmaco il percorso del relativo file strutturale recuperato. A partire da questo file, sono stati estratti i campi corrispondenti ai percorsi degli SDF effettivamente disponibili; complessivamente, i percorsi validi risultavano pari a 128, corrispondenti ai composti per i quali era stato possibile recuperare almeno un file strutturale da PubChem. Per ciascun percorso valido, lo script ha letto il contenuto completo del file SDF corrispondente e lo ha concatenato progressivamente in un unico archivio, salvato come `all_sdfs.sdf`. In questa fase l'unità di partenza non era ancora la singola molecola univoca, ma il contenuto integrale dei file SDF recuperati. Di conseguenza, uno stesso file poteva contenere più record

strutturali e uno stesso farmaco poteva risultare rappresentato da più strutture all'interno dell'archivio finale.

Questa scelta ha privilegiato la massima copertura strutturale disponibile nella fase iniziale del workflow, ma ha introdotto anche ridondanza e possibili duplicati riferibili allo stesso composto. Per questa ragione, il passaggio successivo ha richiesto una procedura esplicita di pulizia del dataset, finalizzata a eliminare record non informativi o completamente sovrapponibili.

Successivamente, il file `all_sdfs.sdf` è stato sottoposto a un primo filtraggio basato sulla dimensione molecolare, espressa come numero totale di atomi presenti in ciascun record strutturale. Tale operazione è stata effettuata leggendo l'insieme delle strutture come SDFset e calcolando per ogni record il numero di atomi mediante una funzione dedicata. È stata quindi applicata una soglia pari a 100 atomi, mantenendo esclusivamente i record con dimensione inferiore a tale valore. Questa scelta è stata adottata per limitare l'analisi a strutture confrontabili nel contesto delle successive procedure di similarità e clustering, escludendo record eccessivamente grandi o poco omogenei rispetto al resto del dataset.

L'output finale di questa fase è stato il file `all_sdfs_filtered.sdf`, contenente 396 record strutturali. Tale numero, superiore ai 128 percorsi inizialmente recuperati, riflette il fatto che alcuni file SDF contengono più di una struttura e che, in questa fase del workflow, i record molecolari sono stati mantenuti tutti, senza una preventiva rimozione dei duplicati. Il file così ottenuto rappresenta la base per i successivi passaggi di calcolo della similarità strutturale e di analisi esplorativa delle relazioni tra composti.

3.3.4 Calcolo della similarità strutturale e costruzione della matrice di distanza

Dopo la costruzione del file strutturale filtrato `all_sdfs_filtered.sdf`, è stata eseguita una procedura di confronto sistematico tra le molecole al fine di quantificare la loro similarità strutturale. A questo scopo, il file è stato letto come insieme di strutture molecolari (SDFset) e utilizzato come input per il calcolo di una matrice di similarità mediante il pacchetto `fmcsR`. Per ciascuna molecola è stato considerato il nome riportato nell'header del record SDF, impiegato come etichetta delle righe e delle colonne della matrice, così da mantenere la corrispondenza tra i valori numerici e le strutture analizzate.

Il confronto è stato effettuato in modo iterativo, calcolando per ogni molecola la similarità rispetto a tutte le altre presenti nel dataset tramite la funzione `fmcsBatch()`. I risultati di ciascun confronto sono stati salvati singolarmente in file RDS, così da rendere il processo più tracciabile e riutilizzabile senza dover ripetere ogni volta l'intero calcolo.

Successivamente, i valori del coefficiente di Tanimoto ottenuti dai file generati sono stati estratti e utilizzati per riempire una matrice quadrata di similarità, nella quale ciascun elemento esprime il grado di somiglianza strutturale tra una coppia di composti. L'output di questa fase è stato quindi una prima matrice di similarità strutturale costruita a partire dal confronto tra tutte le molecole del dataset.

Prima di essere utilizzata nelle analisi successive, la matrice è stata sottoposta a una serie di passaggi di pulizia. In primo luogo, sono state eliminate le righe e le colonne contenenti valori mancanti o non numerici, così da mantenere solo i confronti strutturali interpretabili. In secondo luogo, sono stati rimossi i record che mostravano una similarità pari a 1 con altre molecole al di fuori della diagonale principale, poiché tale condizione indicava la presenza di strutture identiche o completamente sovrapponibili all'interno del dataset. Infine, sono state eliminate anche eventuali righe e colonne duplicate aventi lo stesso nome. Al termine di questi passaggi è stata ottenuta una matrice finale ripulita, salvata come `matrice_similarita_fmcs.csv`. Questi passaggi aggiuntivi sono stati resi necessari soprattutto dalla natura eterogenea del dataset costruito in R, generato concatenando interi file SDF provenienti da PubChem e solo successivamente ricondotto a un sottoinsieme confrontabile.

A partire da questa matrice di similarità è stata quindi costruita la corrispondente matrice di distanza, utilizzata nelle analisi esplorative successive. A tale scopo, la matrice è stata letta dal file `matrice_similarita_fmcs.csv`, convertita in formato matriciale e sottoposta a ulteriori controlli di coerenza, verificando in particolare che fosse quadrata. Per garantire la correttezza numerica del dato, la matrice è stata resa simmetrica mediante media con la sua trasposta, gli eventuali valori mancanti sono stati sostituiti con zero, e tutti i coefficienti sono stati forzati nell'intervallo compreso tra 0 e 1. La diagonale principale è stata poi impostata a 1, coerentemente con il fatto che ogni struttura è identica a se stessa.

La conversione da similarità a distanza è stata effettuata applicando la relazione $D = 1 - S$, dove S rappresenta il coefficiente di similarità e D la distanza strutturale tra le molecole.

In questo modo, a coppie di composti molto simili corrispondono valori di distanza bassi, mentre a coppie poco simili corrispondono valori più elevati. La diagonale principale della matrice di distanza è stata quindi fissata a 0, e la matrice è stata nuovamente simmetrizzata per garantire la coerenza del risultato finale. L'output di questa fase è stato il file `matrice_distanza_fmcs.csv`, utilizzato come base per la successiva rappresentazione esplorativa mediante Principal Coordinates Analysis (PCoA).

3.3.5 Rappresentazione esplorativa delle relazioni strutturali mediante PCoA

Dopo la costruzione della matrice di distanza strutturale `matrice_distanza_fmcs.csv`, è stata effettuata un'analisi esplorativa finalizzata a rappresentare in forma semplificata le relazioni di somiglianza tra i composti. A questo scopo è stata applicata una Principal Coordinates Analysis (PCoA), implementata in ambiente R mediante la funzione `cmdscale()`, utilizzata per proiettare una matrice di distanze in uno spazio a dimensionalità ridotta. L'obiettivo di questo passaggio era ottenere una rappresentazione bidimensionale delle molecole, utile per visualizzare in modo intuitivo eventuali vicinanze strutturali, raggruppamenti e tendenze generali presenti nel dataset.

Come file di input è stata utilizzata la matrice `matrice_distanza_fmcs.csv`, ottenuta nel passaggio precedente. La riduzione dimensionale è stata quindi eseguita imponendo una proiezione su due dimensioni ($k = 2$), così da ottenere per ciascun composto una coppia di coordinate corrispondenti ai primi due assi principali della rappresentazione. Le coordinate risultanti sono state salvate nel file `pcoa_coords_2.csv`, mentre gli autovalori associati alla decomposizione e la relativa varianza spiegata sono stati registrati nel file `pcoa_eig.csv`. In questo modo è stato possibile non solo visualizzare i composti nello spazio bidimensionale, ma anche valutare quanta parte della variabilità strutturale del dataset fosse descritta dagli assi ottenuti.

Come output grafico è stato inoltre prodotto il file `pcoa_plot.png`, contenente una rappresentazione bidimensionale dei composti nello spazio definito dai primi due assi della PCoA. Tale visualizzazione ha costituito una prima sintesi esplorativa delle relazioni strutturali presenti nel dataset, utile per evidenziare possibili vicinanze tra molecole e per preparare le successive fasi di analisi e interpretazione.

3.3.6 Clustering e identificazione della MCS

Dopo la costruzione della matrice di similarità e della corrispondente matrice di distanza, è stato necessario selezionare un sottoinsieme finale di strutture pienamente coerente con i composti effettivamente mantenuti nelle analisi comparative. A questo scopo, il file `all_sdfs_filtered.sdf` è stato confrontato con i nomi riportati nella matrice finale di similarità `matrice_similarita_fmcs.csv`. Per ciascun identificativo presente nella matrice è stata ricercata la corrispondente struttura nel file SDF complessivo e, nei casi in cui fossero disponibili più record associati allo stesso nome, è stato mantenuto soltanto il primo. Sono stati inoltre esclusi i record privi di una rappresentazione SMILES valida, così da ottenere un sottoinsieme strutturale più coerente per le analisi successive. L'output di questa fase è stato il file `filtered_sdfs.sdf`, contenente 73 record strutturali, utilizzato come base per la riduzione dimensionale, il clustering e l'interpretazione strutturale finale.

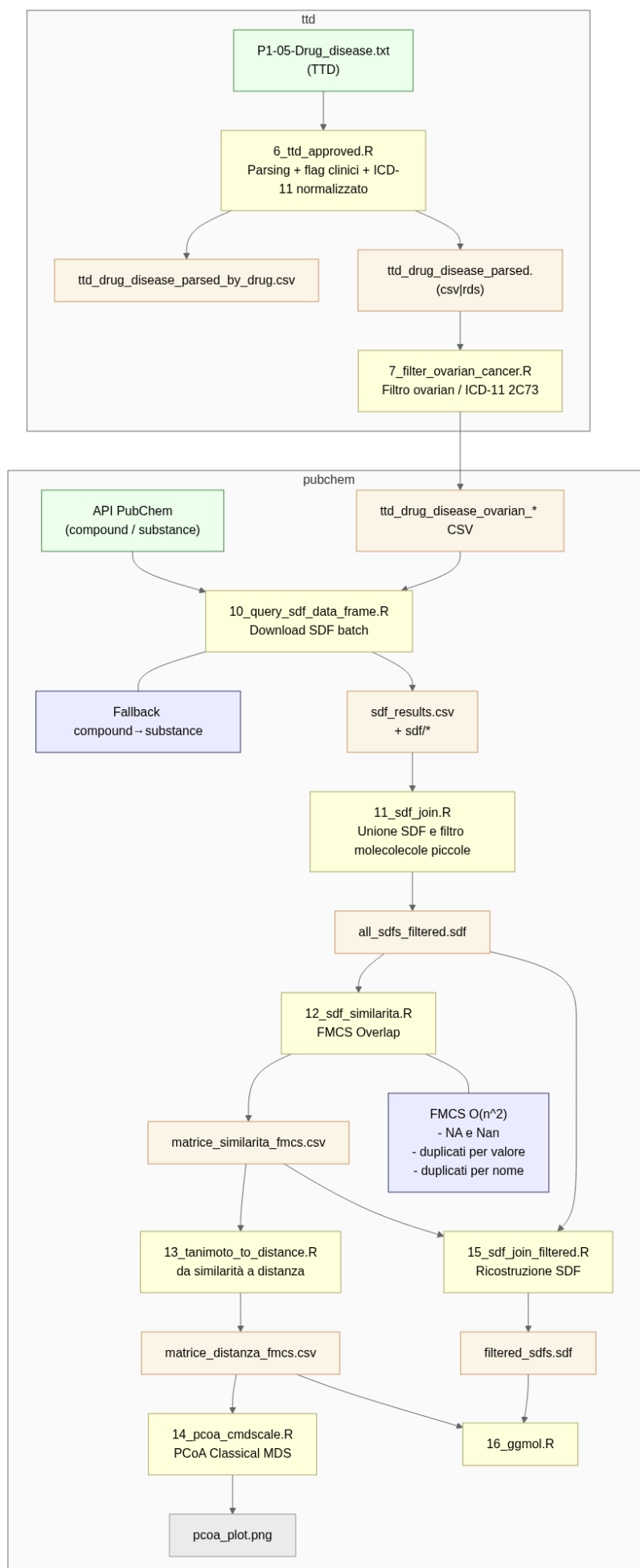
Il file `filtered_sdfs.sdf` è stato quindi letto in ambiente R e convertito in un oggetto molecolare adatto alle elaborazioni chemioinformatiche. In questa fase è stata effettuata un'ulteriore verifica delle rappresentazioni SMILES, escludendo i record contenenti stringhe vuote o non informative. Il controllo del file ha mostrato che, dei 73 record iniziali, 70 risultavano effettivamente leggibili come strutture molecolari, mentre i restanti presentavano problemi strutturali; tra le molecole valide erano inoltre presenti 3 duplicati strutturali, rimossi prima delle analisi successive. A partire da questo sottoinsieme pulito è stata quindi riletta la matrice di distanza `matrice_distanza_fmcs.csv`, ridotta alle sole strutture effettivamente mantenute.

Su tale matrice è stata applicata una procedura di riduzione dimensionale mediante Sammon mapping, utilizzata per ottenere una rappresentazione bidimensionale delle relazioni strutturali tra i composti preservando, per quanto possibile, la configurazione delle distanze originarie. Come configurazione iniziale della procedura è stata utilizzata la proiezione già ottenuta mediante `cmdscale()`, impiegata qui non come analisi autonoma, ma come punto di partenza numericamente coerente per l'ottimizzazione del Sammon mapping. Le coordinate risultanti sono state poi impiegate per il raggruppamento delle molecole tramite il metodo K-means, impostato con 10 cluster. Tale valore è stato adottato come scelta esplorativa, con l'obiettivo di ottenere una partizione sufficientemente

leggibile nella rappresentazione bidimensionale e, al tempo stesso, gruppi ancora interpretabili sul piano strutturale.

Una volta definiti i cluster, le strutture appartenenti a ciascun gruppo sono state raccolte separatamente e utilizzate per la ricerca della Maximum Common Substructure (MCS). Tale procedura è stata eseguita in modo progressivo all'interno di ciascun cluster, confrontando iterativamente le molecole appartenenti allo stesso gruppo e aggiornando di volta in volta la sottostruttura comune condivisa. In questo modo, per ciascun cluster è stata ottenuta una rappresentazione della componente strutturale maggiormente conservata tra le molecole che lo compongono. Questo passaggio ha avuto un ruolo centrale nell'interpretazione chimica dei cluster, poiché ha consentito di associare ai raggruppamenti numerici una sottostruttura comune effettivamente osservabile e confrontabile.

Infine, le coordinate bidimensionali ottenute dal Sammon mapping sono state integrate con l'assegnazione ai cluster e con le informazioni identificative dei composti, così da produrre una rappresentazione grafica finale delle molecole nello spazio ridotto. Per facilitare l'interpretazione del grafico, ai composti sono stati associati nomi o sinonimi recuperati tramite PubChem, utilizzando i sinonimi disponibili per i record identificati come CID e, nei casi non direttamente risolvibili in questo modo, interrogando il database anche a partire dagli identificativi SID. Nel complesso, il workflow in R ha consentito di trasformare un insieme iniziale di associazioni farmaco-patologia in un dataset strutturale progressivamente filtrato, adatto alla stima delle distanze, alla visualizzazione dello spazio chimico e all'identificazione di gruppi molecolari strutturalmente affini.



3.4 Analisi complementare in Python: fingerprint, clustering e identificazione della Maximum Common Substructure

Accanto al workflow sviluppato in ambiente R, è stata implementata una pipeline complementare in Python finalizzata all'analisi strutturale dei composti e all'identificazione della Maximum Common Substructure (MCS). Il ricorso a Python non è stato inteso come validazione perfettamente equivalente del workflow R, ma come implementazione alternativa orientata a una gestione più continua delle strutture molecolari nelle fasi di fingerprinting, clustering e ricerca della sottostruttura comune. Nel progetto, tale possibilità dipende soprattutto dall'uso di RDKit, che consente di trattare le strutture come oggetti molecolari nativi all'interno dello stesso ambiente di lavoro.

La pipeline Python utilizzava come file di input principale `data/compounds.sdf`. Nei casi in cui tale file non fosse disponibile, esso veniva costruito automaticamente interrogando PubChem a partire dai nomi contenuti in `data/ttd_drug_disease_ovarian_by_drug.csv`. Nel progetto Python questo file tabellare non coincide perfettamente con quello impiegato nel workflow R, poiché contiene 198 farmaci distinti anziché 203; di conseguenza, le due pipeline partivano da insiemi ampiamente sovrapponibili ma non identici. Questo scarto va quindi considerato come un limite del confronto diretto tra i due workflow, che restano comparabili soprattutto sul piano della logica metodologica generale. Durante il recupero da PubChem, quando per uno stesso nome erano disponibili più strutture, il codice selezionava la molecola con il maggior numero di atomi entro la soglia di 100 atomi, così da associare a ciascun nome una singola rappresentazione strutturale.

Una volta caricate le molecole, è stata effettuata una fase di filtraggio preliminare per eliminare i record non validi. Su questo insieme di strutture sono state quindi calcolate le Morgan fingerprints mediante RDKit, utilizzando un raggio pari a 2 e una dimensione di 2048 bit. Le fingerprint ottenute sono state impiegate per il calcolo delle distanze di Tanimoto e per la costruzione della matrice di distanza strutturale che ha costituito la base per le procedure successive.

La matrice di distanza è stata poi sottoposta a una procedura di riduzione dimensionale mediante Multidimensional Scaling (MDS), finalizzata a ottenere una rappresentazione bidimensionale delle relazioni strutturali tra le molecole preservando, per quanto possibile,

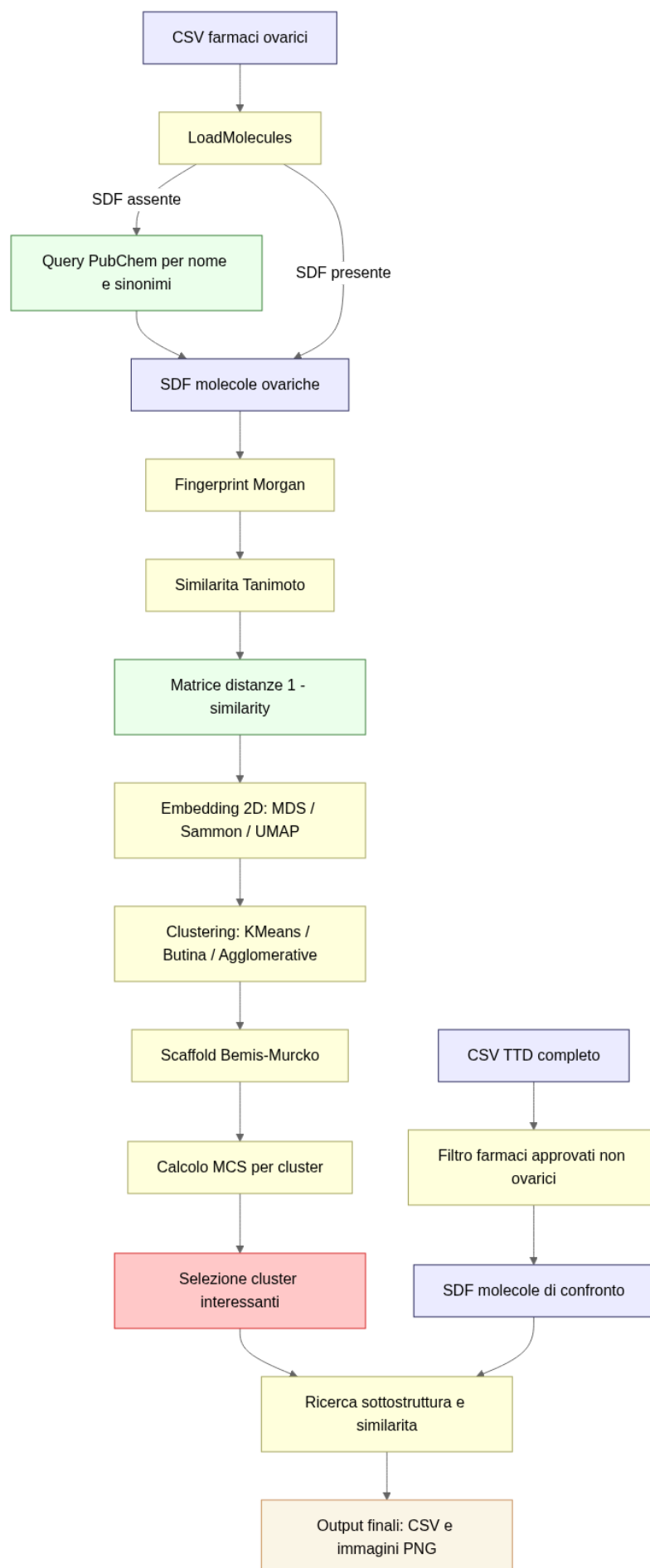
la configurazione delle distanze originarie. Questa scelta riflette l'implementazione effettivamente adottata nella pipeline principale in Python, nella quale l'MDS viene esplicitamente selezionato come strategia preferita rispetto al Sammon mapping, in linea con la distinzione teorica tra MDS come approccio generale alla proiezione di matrici di dissimilarità e Sammon mapping come sua variante non lineare orientata soprattutto alla conservazione delle distanze piccole (Hout et al., 2013; Lespinats et al., 2011). Le coordinate risultanti sono state quindi utilizzate per il raggruppamento dei composti tramite il metodo K-means, impostato nel codice con 32 cluster. Anche in questo caso il valore di **k** è stato adottato come scelta esplorativa, finalizzata a ottenere una partizione più fine dei composti e a isolare sottogruppi strutturalmente più specifici.

Ai fini dell'interpretazione chimica dei cluster, la ricerca della Maximum Common Substructure non è stata eseguita direttamente sulle molecole complete, bensì sui corrispondenti scaffold di Bemis–Murcko ottenuti mediante le funzioni dedicate di RDKit. Questa scelta metodologica è stata adottata per concentrare l'analisi sul nucleo strutturale delle molecole, riducendo l'influenza delle catene laterali e dei sostituenti periferici nella definizione della sottostruttura comune. Per ciascun cluster contenente almeno due molecole, è stata quindi calcolata la corrispondente MCS, successivamente convertita in una rappresentazione molecolare utilizzabile sia per la visualizzazione sia per i confronti con insiemi esterni di composti.

In una fase successiva della pipeline, le MCS ottenute dai cluster selezionati per l'analisi sono state utilizzate come riferimento per il confronto con le altre molecole derivate dal file 'P1-05-Drug_disease.txt', dopo avere escluso i composti corrispondenti ai farmaci già impiegati nel carcinoma ovarico. A questo scopo è stato costruito un dataset residuo di molecole approvate non associate a tale indicazione patologica, successivamente caricato come insieme di candidati per il confronto strutturale. Per ciascuna MCS sono state quindi prodotte due tipologie di output complementari: da un lato l'individuazione delle molecole del dataset filtrato che contenevano la sottostruttura comune del cluster, dall'altro una tabella di punteggi di similarità calcolati rispetto all'intero insieme delle molecole residue. In questo modo, la pipeline ha permesso di passare dall'identificazione dei motivi strutturali condivisi all'interno dei cluster a un loro impiego come criterio di interrogazione di una collezione più ampia di composti, utile per evidenziare molecole esterne al dataset iniziale ma strutturalmente affini ai core comuni emersi nell'analisi del carcinoma ovarico.

Nel complesso, la pipeline Python ha consentito di condurre nello stesso ambiente il recupero delle strutture, la selezione di una rappresentazione per composto, il calcolo delle fingerprint, il clustering e la ricerca della MCS. In questo modo è stato possibile disporre di un'implementazione complementare, utile soprattutto per la fase di interpretazione dei cluster e delle sottostrutture condivise.

Nel loro insieme, le procedure descritte in questo capitolo hanno permesso di passare da un insieme iniziale di associazioni farmaco-patologia a più livelli successivi di organizzazione del dato: un dataset tabellare filtrato, un archivio strutturale ripulito, matrici di similarità e distanza e, infine, una rappresentazione dei composti in termini di cluster e sottostrutture comuni. I risultati quantitativi e interpretativi di questi passaggi sono presentati nel capitolo successivo.



4. Risultati

Il presente capitolo riporta i risultati ottenuti applicando il workflow descritto nel capitolo precedente ai composti associati al carcinoma ovarico. I risultati sono presentati seguendo la progressione del dataset: dalla selezione iniziale dei composti, al recupero delle strutture molecolari, fino alla costruzione delle matrici di similarità e distanza, alla rappresentazione dello spazio chimico e all'identificazione di gruppi strutturalmente affini con relative sottostrutture comuni.

L'obiettivo di questo capitolo non è ripercorrere nuovamente i dettagli procedurali del workflow, già descritti nei Materiali e metodi, ma mostrare quali trasformazioni quantitative e quali pattern strutturali siano emersi nelle diverse fasi dell'analisi.

4.1 Selezione dei composti associati al carcinoma ovarico

L'applicazione della procedura di parsing al file `P1-05-Drug_disease.txt` del Therapeutic Target Database ha consentito di trasformare il contenuto testuale originario in un dataset strutturato di associazioni farmaco-indicazione. Nel complesso sono state ottenute 30.314 associazioni, corrispondenti a 23.718 farmaci distinti.

Il successivo filtraggio per il carcinoma ovarico, effettuato mediante combinazione di termini riconducibili alla patologia nel campo dell'indicazione terapeutica e del codice ICD-11 2C73, ha ridotto il dataset a 207 record pertinenti, corrispondenti a 203 farmaci distinti. All'interno di questo insieme, i composti classificati come approved risultavano 12, tutti corrispondenti a farmaci distinti.

Questo primo risultato ha permesso di passare da una raccolta molto ampia e generale di associazioni farmaco-patologia a un sottoinsieme numericamente più contenuto, ma specificamente rilevante per il contesto del carcinoma ovarico. In tal modo è stato definito il nucleo iniziale dei composti sottoposti alle successive analisi strutturali.

Un estratto della tabella aggregata per farmaco è riportato di seguito a titolo esemplificativo, per mostrare la struttura del dataset ottenuto dopo il filtraggio.

drug_id	drug_name	indications	icd11_codes	any_approved	approval_categories
D00BHL	EC20	Ovarian cancer	2C73	FALSE	phase 3
D00BMF	MK-4827	Ovarian cancer	2C73	FALSE	phase 3
D00GOV	Nivolumab	Ovarian cancer	2C73	FALSE	phase 1
D00IVN	Navicixizumab	Ovarian cancer	2C73	FALSE	phase 1
D00KGB	Abagovomab	Ovarian cancer	2C73	FALSE	phase 2/3

4.2 Recupero delle strutture molecolari e copertura del dataset

Il confronto tra l'elenco dei 203 farmaci distinti associati al carcinoma ovarico e il file strutturale P3-01-All.sdf disponibile in TTD ha mostrato una copertura soltanto parziale del dataset: solo 64 composti risultavano direttamente reperibili in formato strutturale.

Per ampliare la disponibilità di dati chimico-strutturali è stata quindi effettuata un'interrogazione supplementare di PubChem, utilizzando il nome del farmaco come chiave di ricerca e adottando in modo sequenziale i domini compound e substance. Questa procedura ha consentito di recuperare strutture molecolari per 128 dei 203 composti considerati. Tra queste, 68 strutture provenivano dal dominio compound e 60 dal dominio substance, mentre per i restanti 75 composti non è stato possibile ottenere un file strutturale utile.

Questo passaggio ha aumentato in modo sostanziale la copertura strutturale del dataset rispetto a quella ottenibile dal solo TTD. Allo stesso tempo, ha evidenziato una criticità importante del workflow: l'insieme dei composti terapeuticamente associati al carcinoma ovarico non coincide con l'insieme delle molecole immediatamente disponibili in una forma strutturale pronta per analisi chemioinformatiche. Il primo risultato rilevante del workflow consiste quindi proprio nello scarto tra rilevanza farmacologica e disponibilità di dati strutturali effettivamente utilizzabili.

4.3 Costruzione dell'archivio strutturale e filtraggio preliminare

I file SDF recuperati da PubChem sono stati successivamente riuniti in un unico archivio strutturale. A partire dai 128 percorsi validi registrati nel file riepilogativo, è stato costruito un file complessivo contenente tutti i record molecolari disponibili. Dopo il filtraggio

dimensionale basato sul numero di atomi, con esclusione delle strutture aventi almeno 100 atomi, è stato ottenuto il file `all_sdfs_filtered.sdf`, contenente 396 record strutturali.

Il numero dei record risultanti era nettamente superiore al numero iniziale dei composti per i quali era stato recuperato almeno un file SDF. Questo dato riflette il fatto che alcuni file contenevano più di una struttura e che uno stesso composto poteva essere rappresentato da record multipli. In altri termini, la fase di recupero da PubChem ha privilegiato la massima copertura strutturale possibile, ma ha introdotto una marcata ridondanza nel dataset.

Dal punto di vista dei risultati, questa fase ha avuto un duplice effetto. Da un lato ha reso disponibile un archivio chimico sufficientemente ampio per eseguire confronti strutturali sistematici; dall'altro ha mostrato che il dataset non era ancora pronto per analisi comparative dirette, rendendo necessari ulteriori passaggi di pulizia, deduplicazione e selezione. Ne emerge quindi un secondo elemento importante: la costruzione di un dataset strutturale confrontabile non coincide con il semplice recupero delle strutture, ma richiede una fase sostanziale di filtraggio.

4.4 Matrice di similarità strutturale e matrice di distanza

Il confronto strutturale sistematico tra i record molecolari disponibili ha portato alla costruzione di una matrice di similarità tra tutte le strutture mantenute nel dataset. A partire da questa matrice iniziale è stato poi possibile derivare una rappresentazione quantitativa delle relazioni di vicinanza e distanza tra i composti.

La matrice iniziale è stata successivamente sottoposta a una serie di passaggi di pulizia, comprendenti l'eliminazione di valori mancanti o non numerici, la rimozione delle strutture completamente sovrapponibili e l'esclusione di record duplicati. Al termine di questi controlli è stata ottenuta una matrice finale di similarità di dimensioni 73×73 , corrispondente a 73 record strutturali ritenuti effettivamente utilizzabili per le analisi successive.

	135428923	136245567	137226083	24958200	496739869	9952884	9811611
135428923	1.00	0.96	0.96	0.13	0.09	0.16	0.15
136245567	0.96	1.00	0.93	0.13	0.09	0.15	0.15

137226083	0.96	0.93	1.00	0.13	0.09	0.15	0.15
24958200	0.13	0.13	0.13	1.00	0.10	0.24	0.14
496739869	0.09	0.09	0.09	0.10	1.00	0.08	0.07
9952884	0.16	0.15	0.15	0.24	0.08	1.00	0.18
9811611	0.15	0.15	0.15	0.14	0.07	0.18	1.00

A partire da questa matrice è stata derivata la corrispondente matrice di distanza, ottenuta come complemento a 1 della similarità. Tale trasformazione ha consentito di esprimere i rapporti tra molecole in termini di dissimilarità strutturale, rendendo il dataset adatto alle successive procedure di ordinamento spaziale e clustering.

	135428923	136245567	137226083	24958200	496739869	9952884	9811611
135428923	0.00	0.04	0.04	0.87	0.91	0.84	0.85
136245567	0.04	0.00	0.07	0.87	0.91	0.85	0.85
137226083	0.04	0.07	0.00	0.87	0.91	0.85	0.85
24958200	0.87	0.87	0.87	0.00	0.90	0.76	0.86
496739869	0.91	0.91	0.91	0.90	0.00	0.92	0.93
9952884	0.84	0.85	0.85	0.76	0.92	0.00	0.82
9811611	0.85	0.85	0.85	0.86	0.93	0.82	0.00

Il passaggio da 396 record strutturali iniziali a 73 strutture comparabili rappresenta uno dei risultati metodologicamente più significativi del workflow. Esso mostra infatti che una parte consistente delle strutture recuperate non era immediatamente informativa per un confronto globale, a causa di ridondanze, problemi di qualità del dato o sovrapposizioni complete. La matrice finale costituisce quindi il vero nucleo analitico del dataset.

4.5 Interpretazione della matrice di distanza

La matrice di distanza ha permesso di osservare in forma quantitativa il grado di eterogeneità strutturale del dataset. I valori più bassi identificano coppie di molecole molto vicine tra loro, mentre i valori più elevati indicano composti più distanti nello spazio chimico.

Anche l'estratto riportato dalla matrice mostra chiaramente questo comportamento. Alcune coppie di molecole presentano distanze molto basse, ad esempio comprese tra 0,04 e 0,07, suggerendo una marcata somiglianza strutturale. Al contrario, altre coppie mostrano valori molto elevati, spesso compresi tra 0,76 e 0,93, indicativi di una forte dissimilarità.

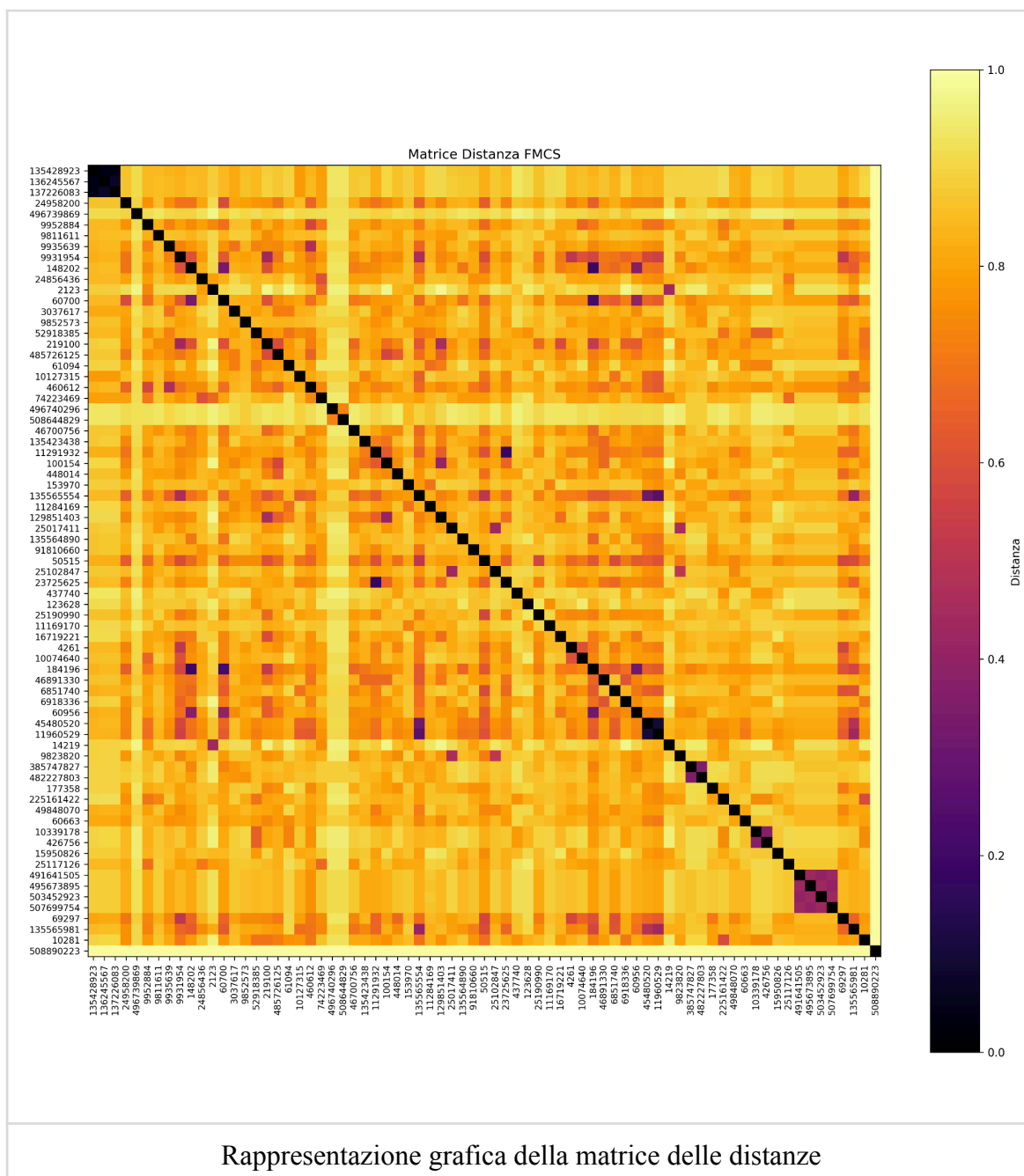
L'estratto seguente riporta alcuni valori esemplificativi della matrice di distanza e consente di visualizzare la compresenza di coppie molto vicine e di coppie fortemente dissimili all'interno dello stesso dataset.

Questo andamento suggerisce che il dataset non sia organizzato come un insieme omogeneo di composti moderatamente simili, ma piuttosto come una collezione strutturalmente eterogenea nella quale coesistono:

- piccoli gruppi di molecole strettamente affini;
- composti più isolati;
- relazioni di distanza molto ampie tra diversi sottogruppi.

Dal punto di vista interpretativo, la matrice di distanza costituisce quindi già un primo risultato biologicamente interessante: i composti associati al carcinoma ovarico non convergono tutti verso un unico tipo strutturale, ma occupano regioni diverse dello spazio chimico, con nuclei locali di maggiore similarità.

Una rappresentazione grafica della matrice consente di apprezzare visivamente questa eterogeneità, evidenziando la compresenza di aree di maggiore prossimità e aree di netta separazione tra gruppi di composti.



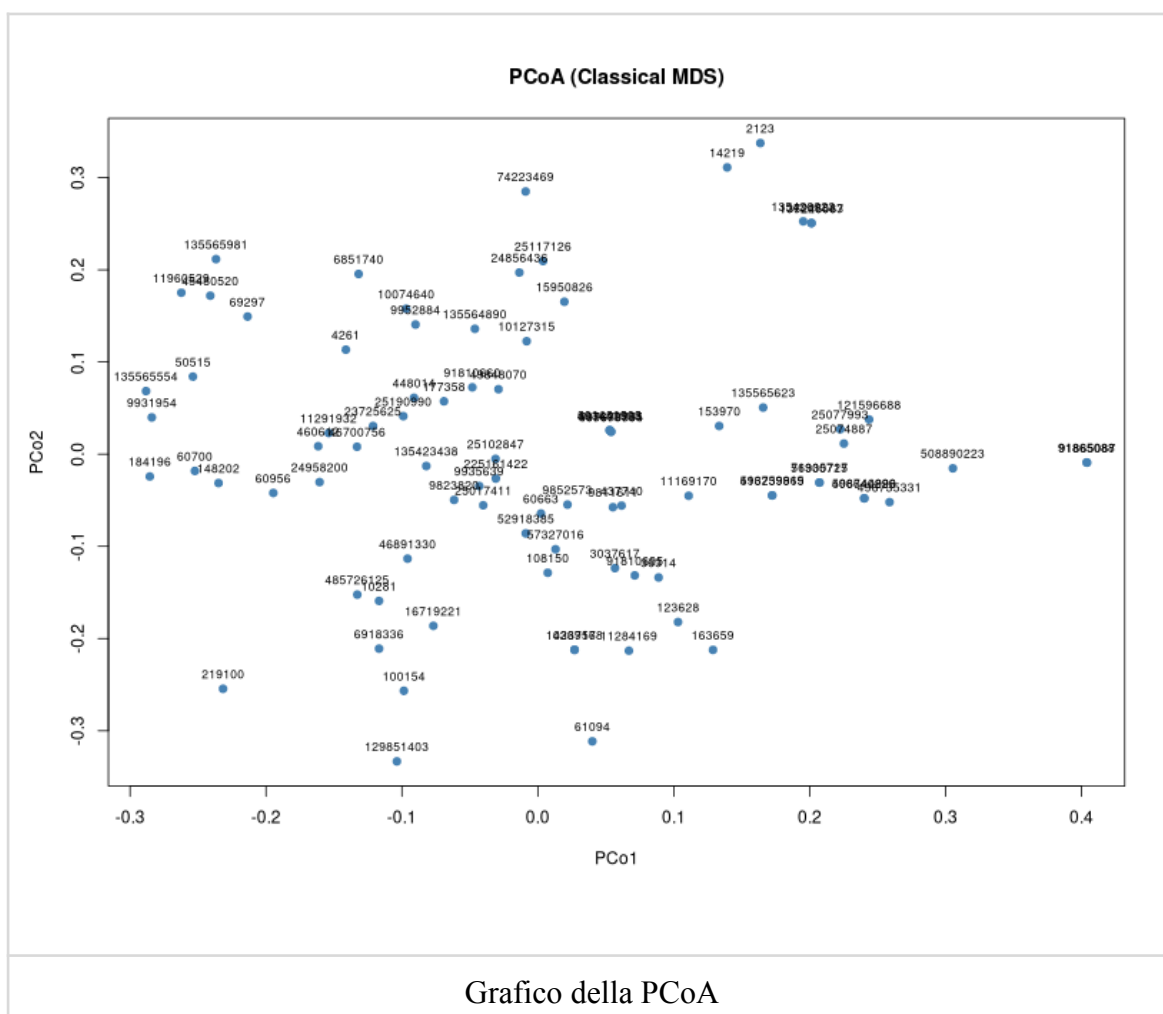
4.6 Rappresentazione esplorativa mediante PCoA

La matrice di distanza strutturale è stata utilizzata come input per una Principal Coordinates Analysis (PCoA), con l'obiettivo di ottenere una rappresentazione bidimensionale delle relazioni tra i 73 record strutturali mantenuti dopo il filtraggio.

L'analisi degli autovalori ha mostrato che il primo asse, PCo1, spiegava circa il 7,14% della variabilità complessiva, mentre il secondo asse, PCo2, ne spiegava circa il 6,57%. Complessivamente, i primi due assi descrivevano quindi circa il 13,71% della variabilità totale del dataset.

Questi valori indicano che la proiezione bidimensionale non riassume in modo esaustivo l'intera complessità del dataset, ma fornisce comunque una sintesi esplorativa utile a visualizzare la distribuzione relativa dei composti. In altri termini, la PCoA non deve essere interpretata come una rappresentazione completa dello spazio chimico, bensì come una mappa semplificata delle relazioni di prossimità tra le molecole.

Nel grafico risultante, la distribuzione dei punti non appare uniforme. Si osservano infatti addensamenti locali, interpretabili come gruppi di molecole relativamente vicine, accanto a punti più periferici o isolati, corrispondenti a composti maggiormente distanti dal resto del dataset. Questo risultato conferma quanto già suggerito dalla matrice di distanza: il panorama strutturale delle molecole associate al carcinoma ovarico risulta internamente eterogeneo e organizzato in sottoregioni di maggiore affinità. Allo stesso tempo, la bassa quota di varianza spiegata dai primi due assi suggerisce di interpretare la PCoA soprattutto come strumento di orientamento esplorativo, utile a visualizzare tendenze generali più che a descrivere in modo esaustivo l'intera struttura del dataset.



4.7 Dalla PCoA al clustering: visualizzazione dello spazio chimico

La PCoA ha fornito una prima rappresentazione globale della variabilità strutturale, ma non una partizione formale delle molecole in gruppi. Per questo motivo, nel workflow successivo è stata affiancata una procedura di riduzione dimensionale più orientata alla conservazione delle relazioni locali, seguita da un algoritmo di clustering.

Nel workflow R, la matrice di distanza è stata sottoposta a Sammon mapping, inizializzato a partire dalla proiezione ottenuta con cmdscale, e le coordinate bidimensionali risultanti sono state successivamente utilizzate per un clustering K-means con 10 cluster. In questo modo, la distribuzione continua delle molecole nello spazio ridotto è stata trasformata in una suddivisione discreta in gruppi strutturalmente affini.



Immagine Clustering workflow R

Questo passaggio ha permesso di rendere più interpretabile la struttura interna del dataset: non solo erano visibili addensamenti e separazioni nello spazio bidimensionale, ma diventava anche possibile associare ogni molecola a un cluster specifico. Il risultato più importante di questa fase non è tanto il numero dei gruppi in sé, quanto il fatto che

l'eterogeneità osservata nella matrice di distanza possa essere ricondotta a insiemi molecolari discreti, utilizzabili come base per successive analisi interpretative.

4.8 Analisi complementare in Python: fingerprint, distanza di Tanimoto e clustering

Accanto al workflow sviluppato in R, è stata implementata una pipeline complementare in Python, basata su RDKit, finalizzata a fornire una rappresentazione alternativa delle relazioni strutturali e un supporto più diretto all'interpretazione dei cluster.

Rispetto al workflow R, questa pipeline partiva da un insieme molto simile ma non perfettamente identico, costituito da 198 farmaci. I risultati ottenuti in Python devono quindi essere letti come complementari a quelli del workflow R, più che come un confronto strettamente sovrapponibile record per record.

L'analisi in Python ha prodotto una matrice di distanza basata su fingerprint di Tanimoto, offrendo così una seconda descrizione quantitativa del dataset, distinta da quella ricavata nel workflow R. Il contributo principale di questa pipeline non consiste tanto nella ripetizione degli stessi passaggi, quanto nella possibilità di disporre di una rappresentazione più direttamente integrata con le successive procedure di clustering e interpretazione strutturale.

4.9 MDS e partizionamento in cluster nella pipeline Python

Anche nella pipeline Python la matrice di distanza è stata utilizzata come base per una rappresentazione bidimensionale mediante Multidimensional Scaling (MDS), con l'obiettivo di preservare il più possibile la configurazione relazionale tra i composti nello spazio ridotto. Le coordinate così ottenute sono state successivamente impiegate per il raggruppamento tramite K-means (Hout et al., 2013; Lespinats et al., 2011).

Nel codice Python, il numero di gruppi è stato fissato a 32 cluster. Tale scelta ha prodotto una partizione più fine rispetto a quella adottata nel workflow R, permettendo di distinguere sottogruppi strutturalmente più specifici. L'adozione dell'MDS come strategia preferita rispetto al Sammon mapping rende inoltre più aderente la descrizione metodologica al comportamento effettivo della pipeline principale. Il clustering non ha quindi avuto soltanto una funzione descrittiva, ma ha costituito il passaggio chiave per

l'identificazione di insiemi di molecole sui quali ricercare una sottostruttura comune interpretabile.

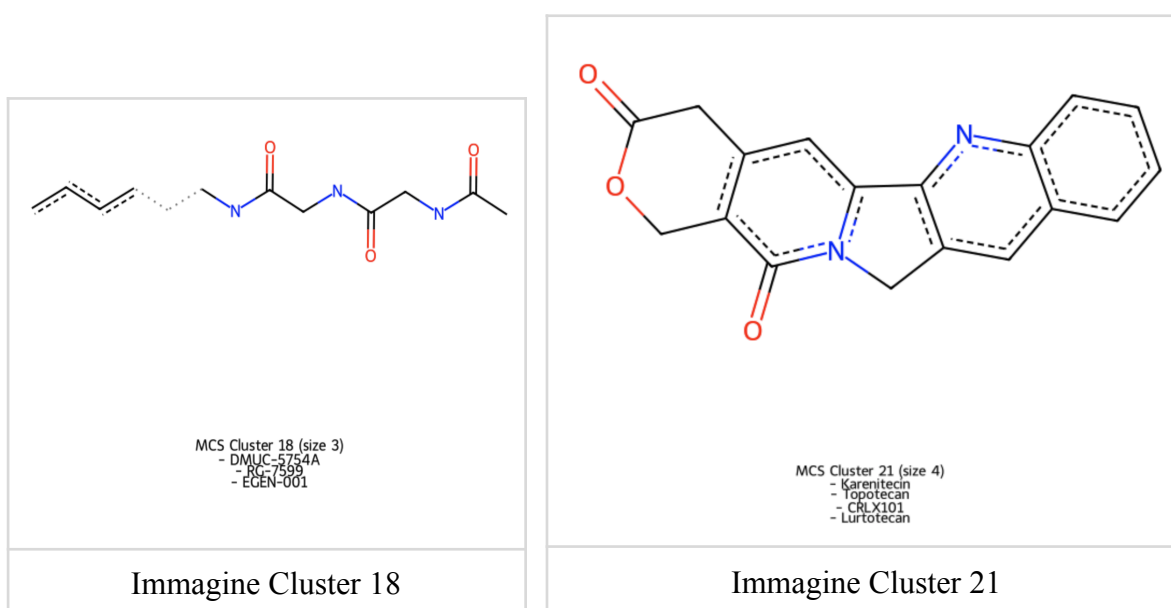
Dal punto di vista dei risultati, il clustering ha reso possibile passare da una distribuzione continua di punti nello spazio bidimensionale a una collezione di gruppi molecolari definiti. In questa fase, la pipeline Python ha contribuito soprattutto a evidenziare in modo più fine la presenza di sottogruppi strutturali distinti, rendendo più agevole la successiva lettura delle sottostrutture comuni. Questo è particolarmente utile ai fini della visualizzazione, perché consente di rappresentare:

- la posizione reciproca dei cluster nello spazio ridotto;
- le molecole appartenenti a ciascun cluster;
- la sottostruttura comune ricavata per ogni gruppo.

4.10 Identificazione delle sottostrutture comuni nei cluster

Per interpretare chimicamente i cluster ottenuti, la pipeline Python non ha calcolato la Maximum Common Substructure direttamente sulle molecole complete, ma sui corrispondenti scaffold di Bemis–Murcko. Questa scelta ha consentito di concentrarsi sul nucleo strutturale delle molecole, riducendo l'influenza delle catene laterali e dei sostituenti periferici.

Per ciascun cluster contenente almeno due molecole è stata quindi calcolata una MCS, associata al gruppo di appartenenza e alle molecole che lo compongono. Gli output prodotti mostrano 23 MCS distinte, corrispondenti ai cluster nei quali era presente un numero sufficiente di strutture per eseguire il confronto. Questo passaggio è centrale perché trasforma il cluster da semplice insieme numerico a gruppo chimicamente interpretabile. In altre parole, non ci si limita più a osservare che alcune molecole sono vicine nello spazio delle distanze, ma si tenta di identificare il motivo strutturale condiviso che può spiegare tale prossimità. In questo senso, la pipeline Python aggiunge al workflow una dimensione interpretativa più esplicita rispetto alla sola descrizione delle distanze o delle coordinate nello spazio ridotto.

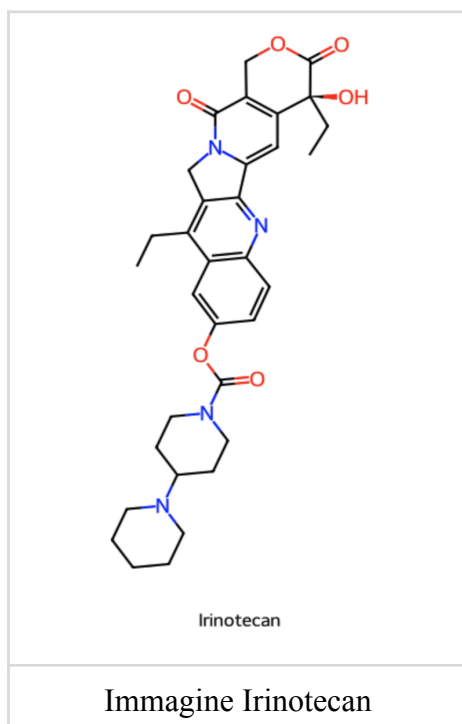


4.11 Confronto delle MCS con le molecole residue del file P1-05

La fase successiva dell'analisi è stata applicata ai cluster 18 e 21, selezionati nella pipeline Python per il confronto con le molecole esterne al dataset del carcinoma ovarico. Per ciascuno di questi cluster è stato prodotto un file tabellare di output, rispettivamente 'cluster_18_vs_approved_mols.csv' e 'cluster_21_vs_approved_mols.csv', contenente i risultati del confronto con il dataset di molecole approvate ottenuto da 'P1-05-Drug_disease.txt' dopo l'esclusione dei farmaci già associati al carcinoma ovarico. Ciascuna tabella riporta 2478 molecole effettivamente confrontate, insieme all'indicazione terapeutica annotata nel dataset filtrato e al punteggio di similarità rispetto alla MCS del cluster corrispondente.

Dal punto di vista dei risultati, questa fase non ha restituito soltanto una verifica qualitativa della presenza della sottostruttura comune, ma un profilo quantitativo di affinità strutturale esteso all'intero insieme delle molecole residue. Nel file relativo al cluster 18, il valore massimo osservato è pari a 0.222 ed è associato ad Acamprosate; seguono Acetohydroxamic Acid con 0.194 e Iodamide con 0.182. Nel file relativo al cluster 21, il valore massimo osservato è pari a 0.258 ed è associato a Irinotecan; seguono Quinolones con 0.208 e Oxcarbazepine con 0.190. Questi risultati mostrano che le MCS derivate dai cluster del carcinoma ovarico possono essere utilizzate per ordinare un insieme più ampio

di composti in base alla loro affinità strutturale, fornendo una base esplorativa per l'individuazione di molecole che condividono, almeno in parte, i motivi comuni emersi nei cluster analizzati.



4.12 Verifica in letteratura dei composti selezionati

Per valutare la rilevanza dei composti emersi dal confronto di similarità, i risultati collocati nelle posizioni più alte delle graduatorie sono stati successivamente confrontati con la letteratura scientifica disponibile sul carcinoma ovarico. In questo passaggio, Irinotecan e NAC hanno rappresentato i casi più significativi, poiché consentono di collegare in modo particolarmente chiaro l'output computazionale a dati già presenti in letteratura.

Irinotecan costituisce il riscontro più solido sul piano clinico tra i composti emersi dall'analisi, poiché coincide con il valore massimo osservato per il cluster 21 ed è stato valutato in più studi dedicati al carcinoma ovarico recidivante o resistente al platino. Un primo riferimento centrale è rappresentato dallo studio di fase II di Bodurka et al. del 2003, condotto in pazienti con carcinoma ovarico epiteliale o carcinoma peritoneale primitivo platinum-resistant o platinum-refractory, nel quale irinotecan è stato impiegato come monoterapia e ha mostrato un'attività clinica moderata, accompagnata da tossicità non

trascurabili ma senza decessi correlati al trattamento. In parallelo, lo studio di Sugiyama et al. del 1998 documenta già in una fase più precoce l'interesse per irinotecan in combinazione con cisplatino nel carcinoma ovarico refrattario o recidivante, mostrando che il composto era stato preso in considerazione come opzione terapeutica in un setting di elevata complessità clinica. Questo filone viene ulteriormente sviluppato in lavori più recenti, nei quali irinotecan compare come partner di combinazione con bevacizumab in pazienti con malattia recidivante pesantemente pretrattata e con doxorubicina liposomiale pegilata nel carcinoma ovarico platino-resistente recidivante, confermando la persistenza del suo interesse clinico anche in strategie terapeutiche più articolate (Musa et al., 2017; Shoji et al., 2017). Nel complesso, il fatto che irinotecan emerga come hit ad alta similarità e trovi al tempo stesso un supporto consistente in studi clinici dedicati al carcinoma ovarico suggerisce che il metodo adottato sia in grado di restituire composti coerenti con evidenze già consolidate, senza che ciò implichi, di per sé, una dimostrazione di efficacia terapeutica.

La N-acetilcisteina (NAC), emersa tra le primissime posizioni del ranking del cluster 18 insieme alla voce Acetylcysteine, rappresenta invece un caso particolarmente interessante sul piano biologico e preclinico. La review di Kindlon e Pidgeon del 2025 la inquadra come precursore membrana-permeabile della cisteina, capace di aumentare il pool intracellulare di questo amminoacido e di influenzare quindi la sintesi di glutatione e l'equilibrio redox cellulare; nello stesso lavoro gli autori discutono il ruolo dell'asse cisteina/glutatione nel carcinoma ovarico soprattutto in relazione alla risposta alla chemioterapia, alla tossicità del trattamento e ai meccanismi di chemioresistenza, sottolineando tuttavia che l'effetto della NAC rimane dipendente dal sottotipo tumorale, dal contesto redox e dal timing terapeutico. Un supporto sperimentale più diretto deriva dallo studio di Brum et al. del 2013 su cellule CaOV3 di carcinoma ovarico, nel quale il pretrattamento con NAC ha potenziato l'attivazione di ATM e p53 indotta da doxorubicina, in un contesto associato anche a inibizione della proliferazione e della migrazione cellulare, riorganizzazione del citoscheletro e riduzione dell'attività di mTOR. A ciò si aggiunge il lavoro di Zhang et al. del 2016, non specificamente focalizzato sul carcinoma ovarico, ma rilevante dal punto di vista meccanicistico perché mostra che NAC può ridurre i livelli di Notch3 attraverso un meccanismo di degradazione lisosomiale e attenuarne la segnalazione maligna; lo studio riporta inoltre questo effetto anche in OVCAR-3, suggerendo che il segnale osservato non sia del tutto estraneo al contesto ovarico.

Considerate insieme, queste evidenze non permettono di attribuire a NAC un ruolo antitumorale definito nel carcinoma ovarico, ma indicano che la sua comparsa tra gli hit principali dell'analisi è compatibile con una letteratura che la colloca come modulatore plausibile di processi biologici rilevanti per la sensibilità al trattamento.

4.13 Altri composti con riscontri in letteratura

Accanto ai casi di irinotecan e NAC, altri composti emersi dall'analisi presentano un riscontro significativo nel carcinoma ovarico pur non coincidendo con i risultati più forti in termini di similarità. Ordinando questi casi per rilevanza complessiva nel contesto della patologia, il collegamento più diretto riguarda treosulfan, la cui presenza in letteratura è ampia e continuativa nel carcinoma ovarico recidivante o platino-resistente: gli studi disponibili ne documentano un'attività moderata come seconda linea o trattamento palliativo, un impiego anche in combinazione con gemcitabina e, allo stesso tempo, limiti comparativi rispetto a topotecan negli studi randomizzati (Keldsen et al., 1998; Meier et al., 2009; Hilman et al., 2010; Olesen et al., 2021). Un profilo di interesse clinico, seppur più legato a strategie di combinazione, emerge anche per vorinostat, per il quale la letteratura mostra una attività minima in monoterapia nelle pazienti non selezionate, una combinazione con carboplatino e gemcitabina resa problematica dalla tossicità ematologica e risultati più favorevoli quando associato a carboplatino e paclitaxel nel recidivante platino-sensibile (Modesitt et al., 2008; Matulonis et al., 2015; Meteran et al., 2024). A un livello successivo può essere collocata thalidomide, studiata sia come singolo agente sia in combinazione con topotecan nel carcinoma ovarico recidivante, con un'attività clinica complessivamente moderata ma con segnali più convincenti nei regimi combinati, da interpretare comunque alla luce del profilo di tossicità della molecola (Chan et al., 2006; Downs et al., 2008). Su un piano prevalentemente preclinico si colloca invece disulfiram, che appare particolarmente rilevante come candidato di drug repurposing per il suo effetto sulla frazione ALDH⁺ stem-like, sull'aumento dei ROS e sul potenziamento dell'apoptosi indotta da cisplatino, oltre che per l'attività osservata in combinazione con inibitori di PARP attraverso un incremento dell'apoptosi e della risposta al danno del DNA (Guo et al., 2019; Tang et al., 2023). Infine, la melatonina si distingue per una letteratura coerente nel potenziamento dell'effetto del cisplatino, mediato dalla modulazione di pathway come

ERK/p90RSK/HSP27 e PI3K/Akt e da meccanismi che non sembrano dipendere esclusivamente dai recettori MT1 (Kim et al., 2012; Zemła et al., 2017; Baghal-Sadriforoush et al., 2022). Nel loro insieme, questi composti non coincidono necessariamente con gli hit principali del ranking di similarità, ma ampliano il quadro dei risultati mostrando che l'area chimico-biologica intercettata dall'analisi comprende molecole che la letteratura ha già esplorato, con diversi livelli di maturità clinica e preclinica, nel contesto del carcinoma ovarico.

4.14 Molecole visibili nei cluster e produzione delle immagini

Un risultato particolarmente importante riguarda la possibilità di associare a ogni cluster non solo una MCS, ma anche l'elenco esplicito delle molecole che lo compongono. Le immagini prodotte per i cluster riportano infatti:

- l'identificativo del cluster;
- la dimensione del cluster;
- i nomi delle molecole appartenenti al gruppo.

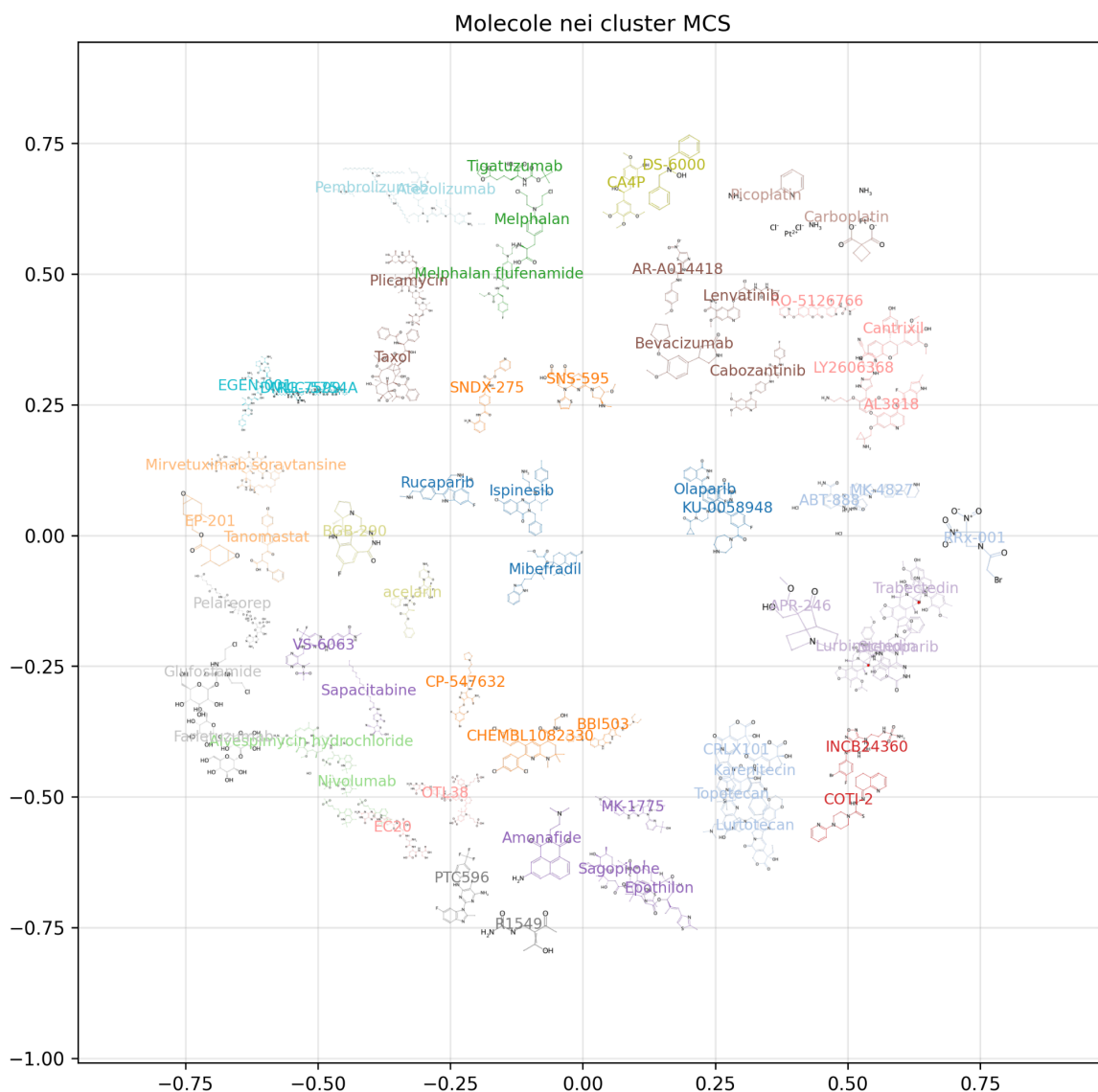
Questo significa che il clustering non produce soltanto una suddivisione astratta del dataset, ma genera un output direttamente utilizzabile per la documentazione dei risultati. Ogni immagine di cluster può essere letta su due livelli:

1. come rappresentazione della sottostruttura comune del gruppo;
2. come riepilogo delle molecole che rendono visibile quel pattern strutturale.

Dal punto di vista della tesi, questo è molto utile perché permette di costruire figure in cui il lettore può collegare immediatamente:

- la posizione di un gruppo nello spazio ridotto;
- le molecole che ne fanno parte;
- il core strutturale condiviso.

In questo senso, le molecole visibili nei cluster costituiscono il ponte tra analisi numerica e interpretazione chimica.



4.15 Sintesi interpretativa dei risultati

Nel loro insieme, i risultati mostrano che i composti associati al carcinoma ovarico non costituiscono un insieme strutturalmente omogeneo, ma una collezione ampia ed eterogenea di molecole distribuite in regioni differenti dello spazio chimico. Il passaggio da oltre trentamila associazioni farmaco-indicazione iniziali a 203 farmaci pertinenti e, successivamente, a 73 strutture effettivamente comparabili evidenzia che la costruzione del dataset analitico rappresenta essa stessa un risultato metodologicamente rilevante del lavoro.

La matrice di distanza e la PCoA hanno mostrato la presenza di nuclei locali di maggiore similarità all'interno di un panorama strutturale complessivamente disperso. Questo dato

suggerisce che, pur in assenza di un unico profilo chimico dominante, il dataset contenga gruppi di composti che condividono relazioni strutturali sufficientemente forti da giustificare analisi di clustering e ricerca di sottostrutture comuni.

L'integrazione tra workflow R e pipeline Python ha infine permesso di articolare due livelli complementari di lettura dei dati: da un lato una descrizione quantitativa dell'eterogeneità del dataset, dall'altro una prima interpretazione chimica dei gruppi molecolari identificati. In questo modo, il risultato finale del capitolo non consiste soltanto nella produzione di matrici, coordinate o cluster, ma nella costruzione di un quadro strutturale più ordinato dei composti associati al carcinoma ovarico, utile come base per la discussione generale del lavoro.

5. Discussione e conclusioni

Il presente capitolo ha lo scopo di interpretare criticamente i risultati ottenuti, chiarire il contributo effettivo del lavoro e delimitarne i principali limiti. L'obiettivo non è ripetere i passaggi metodologici o riassumere in forma descrittiva il Capitolo 4, ma discutere che cosa i risultati mostrino rispetto all'analisi strutturale dei composti associati al carcinoma ovarico e quale valore abbiano, sul piano chemioinformatico, le procedure adottate.

5.1 Sintesi dei risultati principali

Il risultato più evidente emerso dal lavoro è che i composti associati al carcinoma ovarico non costituiscono un insieme strutturalmente uniforme. Al contrario, il dataset analizzato mostra una marcata eterogeneità, con la compresenza di molecole molto distanti tra loro e di piccoli gruppi più compatti, caratterizzati da livelli più elevati di similarità strutturale. Questa osservazione è coerente con il fatto che il carcinoma ovarico rappresenti non una singola entità terapeutica, ma un contesto clinico nel quale convergono molecole appartenenti a classi farmacologiche, fasi di sviluppo e strategie di intervento differenti.

Un secondo risultato rilevante riguarda la costruzione stessa del dataset analitico. Il passaggio dalle 30.314 associazioni iniziali ai 203 farmaci distinti pertinenti e, successivamente, alle 73 strutture effettivamente comparabili evidenzia che la selezione delle molecole e la loro trasformazione in un insieme strutturale interrogabile costituiscono una parte sostanziale del contributo del lavoro. In altre parole, il risultato non risiede soltanto nell'analisi finale delle distanze o dei cluster, ma anche nella definizione di un percorso riproducibile attraverso cui dati eterogenei, incompleti e ridondanti vengono progressivamente convertiti in un dataset confrontabile.

Un terzo elemento significativo consiste nell'identificazione di nuclei locali di similarità all'interno di un panorama globale disperso. La matrice di distanza, la PCoA e le procedure di clustering hanno infatti mostrato che, pur in assenza di un unico profilo chimico dominante, esistono gruppi di composti sufficientemente affini da giustificare analisi più mirate delle sottostrutture comuni. Questo punto è particolarmente importante perché suggerisce che l'eterogeneità del dataset non escluda la presenza di organizzazioni locali leggibili e potenzialmente utili per successive interpretazioni.

Un quarto risultato rilevante emerge dalla fase successiva di confronto tra le MCS dei cluster selezionati e le molecole residue derivate dal file 'P1-05-Drug_disease.txt', dopo esclusione dei farmaci già associati al carcinoma ovarico. Questo passaggio ha mostrato che i motivi strutturali ricavati dal clustering possono essere utilizzati non solo per descrivere il dataset iniziale, ma anche per interrogare un insieme più ampio di composti e produrre graduatorie di affinità strutturale. In tal modo, la pipeline non si limita a identificare sottostrutture condivise all'interno del dataset di partenza, ma genera anche un criterio operativo per evidenziare molecole esterne strutturalmente affini ai core emersi dall'analisi.

Un quinto elemento di interesse è rappresentato dal confronto con la letteratura scientifica. I casi di Irinotecan e N-acetilcisteina hanno mostrato che alcuni tra i composti emersi con maggiore rilievo dall'analisi trovano un riscontro in studi già pubblicati sul carcinoma ovarico, rispettivamente su un piano clinico e su un piano biologico-preclinico. Accanto a questi, la presenza in letteratura di composti come treosulfan, vorinostat, thalidomide, disulfiram e melatonina rafforza ulteriormente l'idea che l'area chimico-biologica intercettata dal workflow non sia arbitraria, ma intercetti molecole che sono state effettivamente esplorate, con livelli diversi di maturità, nel contesto della patologia.

5.2 Significato metodologico e chemioinformatico dei risultati

Dal punto di vista chemioinformatico, il lavoro mostra che un insieme di composti selezionati sulla base della loro associazione terapeutica con una patologia può essere trattato come uno spazio chimico da esplorare in termini di similarità, distanza e organizzazione interna. In questo senso, il contributo della tesi non consiste nell'identificazione di una nuova relazione farmacologica o di un nuovo bersaglio biologico, ma nella costruzione di un quadro strutturale ordinato di molecole già rilevanti nel contesto del carcinoma ovarico.

L'eterogeneità osservata ha un significato metodologico preciso. Essa indica che la categoria "composti associati al carcinoma ovarico" non coincide con una famiglia chimicamente coesa, ma aggrega molecole selezionate per rilevanza terapeutica o sperimentale piuttosto che per vicinanza strutturale. Da questo punto di vista, il risultato è coerente con la natura stessa della fonte utilizzata: il Therapeutic Target Database organizza infatti le molecole in base al rapporto con indicazioni e target, non in base a

criteri di similarità chimica. L'analisi strutturale consente quindi di aggiungere un livello di lettura diverso rispetto a quello originariamente fornito dal database.

Allo stesso tempo, la presenza di cluster e di sottostrutture comuni mostra che l'eterogeneità globale non è sinonimo di completa disorganizzazione. Alcuni gruppi di molecole condividono infatti tratti strutturali sufficientemente conservati da rendere plausibile una loro lettura come sottoregioni dello spazio chimico. Il significato di questo risultato va mantenuto entro limiti rigorosi: la tesi non dimostra di per sé una comune attività biologica o un'identica modalità d'azione dei composti raggruppati, ma mostra che esistono relazioni strutturali locali che possono costituire un punto di partenza per analisi successive.

In questo quadro, la ricerca della Maximum Common Substructure assume un ruolo interpretativo importante. Il passaggio dal semplice raggruppamento numerico alla definizione di un nucleo strutturale condiviso rende infatti i cluster maggiormente leggibili sul piano chimico. La MCS non deve essere intesa come descrizione esaustiva della funzione delle molecole considerate, ma come rappresentazione compatta del tratto strutturale che accomuna i composti di uno stesso gruppo. Ciò rende i risultati più facilmente documentabili e potenzialmente più utili per confronti futuri con altre collezioni di molecole.

La successiva applicazione delle MCS a un dataset esterno filtrato aggiunge un ulteriore livello metodologico al lavoro. In questo caso, la sottostruttura comune non viene utilizzata soltanto come strumento descrittivo dei cluster, ma come riferimento per una procedura di interrogazione di una collezione più ampia di composti. Il valore chemioinformatico di questo passaggio risiede proprio nel collegamento tra due livelli di analisi: da un lato la sintesi strutturale interna ai cluster, dall'altro la possibilità di usare tale sintesi per ordinare molecole esterne in base alla loro affinità con i motivi comuni identificati.

Un aspetto particolarmente significativo è che questa procedura non è rimasta confinata al solo piano computazionale. Il confronto tra alcuni degli hit selezionati e la letteratura disponibile ha mostrato che il workflow è capace di restituire composti compatibili con evidenze già presenti nel carcinoma ovarico. Ciò non implica che la similarità strutturale sia sufficiente a dedurre efficacia farmacologica o rilevanza clinica, ma suggerisce che il metodo possa funzionare come strumento di prioritizzazione plausibile, utile per

selezionare molecole da sottoporre a verifiche successive di natura bibliografica, biologica o farmacologica.

5.3 Limiti del lavoro

Il primo limite riguarda la natura del dataset iniziale. La selezione dei composti è stata effettuata a partire dalle annotazioni di TTD relative al carcinoma ovarico, utilizzando quindi un criterio basato sull'associazione terapeutica riportata nel database. Questo approccio ha il vantaggio di essere sistematico e riproducibile, ma dipende dalla qualità, completezza e granularità delle annotazioni disponibili. Di conseguenza, il dataset costruito non rappresenta l'intero universo dei composti rilevanti per il carcinoma ovarico, bensì il sottoinsieme recuperabile e interrogabile a partire dalla specifica risorsa utilizzata.

Un secondo limite, particolarmente importante, riguarda lo scarto tra composti selezionati e composti effettivamente confrontabili sul piano strutturale. Non tutti i farmaci identificati come pertinenti disponevano infatti di una rappresentazione strutturale pronta all'uso, e una parte dei record recuperati ha richiesto ulteriori filtri per ridondanza, problemi di qualità o non confrontabilità. Ne deriva che il dataset finale analizzato è più ristretto del dataset farmacologicamente rilevante di partenza. Questo passaggio non invalida l'analisi, ma deve essere dichiarato con chiarezza perché condiziona l'ampiezza e la rappresentatività delle conclusioni.

Un terzo limite riguarda il rapporto tra cornice biologico-clinica e oggetto operativo del workflow. Nel Capitolo 1 l'HGSC è stato assunto come riferimento principale per motivare la rilevanza del problema clinico; sul piano computazionale, tuttavia, l'analisi è stata condotta su composti associati al carcinoma ovarico in senso più ampio. Questa scelta è metodologicamente giustificabile, in quanto consente di lavorare su un dataset sufficientemente esteso, ma impone cautela nell'interpretazione biologica dei risultati: i pattern strutturali osservati non devono essere letti come specifici dell'HGSC, bensì come riferiti al più ampio insieme di composti annotati per il carcinoma ovarico.

Un quarto limite riguarda le scelte esplorative adottate nelle fasi di riduzione dimensionale e clustering. Sia nel workflow R sia nella pipeline Python, il numero di cluster è stato fissato secondo criteri pragmatici di leggibilità e interpretabilità, non come esito di una procedura di ottimizzazione univoca. Anche la scelta di lavorare su scaffold di Bemis-Murcko nella ricerca della MCS, pur essendo metodologicamente sensata ai fini

interpretativi, privilegia una specifica lettura delle molecole e ne trascura altre componenti strutturali potenzialmente rilevanti. I risultati ottenuti devono quindi essere considerati come esplorativi e orientativi, più che come classificazione definitiva del dataset.

Un quinto limite riguarda l'interpretazione della fase di confronto con il dataset residuo e della successiva verifica in letteratura. Il fatto che alcuni composti ad alta affinità strutturale trovino un riscontro in studi pubblicati non implica una validazione sperimentale del metodo né dimostra una loro efficacia nel carcinoma ovarico. La letteratura richiamata comprende infatti livelli diversi di evidenza, che spaziano da studi preclinici in linee cellulari a trial clinici in setting selezionati di malattia recidivante o resistente. Di conseguenza, il valore di questa fase deve essere interpretato in termini di coerenza e plausibilità dei risultati, non come prova diretta di rilevanza terapeutica.

5.4 Confronto tra workflow R e pipeline Python

Il lavoro ha impiegato due implementazioni distinte ma complementari. Il workflow in R ha avuto un ruolo centrale nella costruzione del dataset, nella gestione dei file strutturali, nel filtraggio progressivo dei record e nella prima descrizione quantitativa delle relazioni di similarità e distanza. La pipeline in Python, invece, ha permesso di integrare più direttamente fingerprinting, riduzione dimensionale mediante MDS, clustering e identificazione della Maximum Common Substructure nello stesso ambiente, rendendo più lineare la fase interpretativa dei gruppi molecolari (Hout et al., 2013; Lespinats et al., 2011).

Il rapporto tra i due workflow va però definito con precisione. La pipeline Python non costituisce una validazione strettamente equivalente del workflow R, perché i due percorsi non partono da dataset perfettamente identici e adottano in parte scelte operative differenti. Il confronto tra R e Python deve quindi essere inteso soprattutto come confronto tra approcci metodologici generali e non come verifica uno-a-uno di ciascun record o di ciascun cluster ottenuto.

Nonostante questo limite, l'integrazione tra i due ambienti rappresenta uno degli aspetti più interessanti del lavoro. R si è dimostrato particolarmente adatto alla costruzione trasparente del dataset e alla sua esplorazione progressiva; Python, attraverso RDKit e l'uso dell'MDS come strategia preferita di embedding, ha offerto una gestione più continua degli oggetti molecolari e una maggiore immediatezza nella generazione di cluster e immagini delle

sottostrutture comuni. Considerati insieme, i due workflow non si sostituiscono a vicenda, ma consentono una lettura più completa del dataset: il primo orientato alla costruzione e alla descrizione quantitativa dello spazio chimico, il secondo alla sua interpretazione strutturale.

5.5 Prospettive future

Un primo sviluppo futuro riguarda la definizione di un dataset iniziale più controllato e omogeneo. Sarebbe utile verificare in modo più sistematico la corrispondenza tra i composti annotati nei database farmacologici e le strutture effettivamente disponibili, introducendo procedure di standardizzazione più rigorose e criteri espliciti per la gestione di sinonimi, stereochimica, sali e forme multiple dello stesso composto.

Un secondo sviluppo potrebbe consistere nell'estendere l'analisi con criteri di clustering e validazione più formalizzati. L'adozione di misure aggiuntive per la scelta del numero di cluster, il confronto tra algoritmi differenti e l'inclusione di metriche di qualità della partizione permetterebbero di rendere più robusta la lettura dei gruppi molecolari identificati.

Un terzo ambito di sviluppo riguarda l'integrazione con informazioni biologiche e farmacologiche esterne. I cluster strutturali e le MCS individuate potrebbero essere messi in relazione con classi terapeutiche, target molecolari, meccanismi d'azione o profili di sensibilità farmacologica, così da verificare se i pattern strutturali osservati corrispondano anche a organizzazioni funzionali più definite.

Un quarto sviluppo futuro riguarda l'estensione sistematica della fase di prioritizzazione successiva al clustering. La procedura impiegata in questo lavoro, basata sul confronto tra MCS di cluster e molecole residue del dataset TTD filtrato, potrebbe essere ampliata integrando criteri aggiuntivi di selezione, quali robustezza del match di sottostruttura, collegamento con specifici target, presenza di evidenze precliniche o cliniche pregresse e livello di novità rispetto ai composti già impiegati nel carcinoma ovarico. In questo modo, il workflow potrebbe evolvere da strumento descrittivo dello spazio chimico a procedura più esplicitamente orientata alla prioritizzazione di candidati per studi successivi.

Infine, il workflow potrebbe essere esteso a sottoinsiemi più specifici della patologia, inclusi dataset più strettamente focalizzati sull'HGSC o su altri sottotipi del carcinoma

ovarico. Questo permetterebbe di valutare se la maggiore specificità clinica del dataset si traduca anche in una diversa organizzazione dello spazio chimico e in pattern strutturali più omogenei.

5.6 Conclusioni

Nel complesso, il presente lavoro ha mostrato che i composti associati al carcinoma ovarico possono essere organizzati in un dataset strutturale analizzabile e interrogati mediante strumenti chemioinformatici per descriverne l'eterogeneità e individuare nuclei locali di similarità. Il contributo principale della tesi consiste nella costruzione di un workflow ripercorribile che collega selezione dei composti, recupero delle strutture, filtraggio del dataset, analisi delle distanze e interpretazione dei cluster.

I risultati ottenuti non consentono di definire un unico profilo chimico dei composti associati al carcinoma ovarico, ma mostrano che all'interno di un insieme complessivamente eterogeneo emergono gruppi strutturalmente più coerenti e sottostrutture condivise interpretabili. Inoltre, la successiva applicazione delle MCS a un dataset esterno filtrato ha mostrato che tali motivi comuni possono essere utilizzati per ordinare altre molecole in base alla loro affinità strutturale, estendendo il valore del workflow oltre la sola descrizione del dataset iniziale.

Il confronto con la letteratura ha infine aggiunto un livello di verifica qualitativa ai risultati computazionali. Il riscontro di composti come irinotecan e N-acetilcisteina, insieme alla presenza in letteratura di ulteriori molecole emerse dall'analisi quali treosulfan, vorinostat, thalidomide, disulfiram e melatonina, suggerisce che il metodo adottato sia capace di intercettare regioni dello spazio chimico effettivamente pertinenti al carcinoma ovarico. In questo senso, la tesi fornisce non soltanto una descrizione strutturale del dataset, ma anche una base metodologica per future analisi di prioritizzazione e approfondimento, confermando il valore di un approccio integrato tra costruzione del dato, esplorazione dello spazio chimico, interpretazione strutturale e verifica bibliografica dei composti selezionati.

6. Bibliografía

Global cancer observatory: <https://gco.iarc.who.int/en>

Lisio, M.-A., Fu, L., Goyeneche, A., Gao, Z.-H., & Telleria, C. (2019). High-Grade Serous Ovarian Cancer: Basic Sciences, Clinical and Therapeutic Standpoints. *International Journal of Molecular Sciences*, 20(4)

www.cancer.gov/types/ovarian/hp/ovarian-prevention

López-Pérez, K., Avellaneda-Tamayo, J. F., Chen, L., López-López, E., Concha, A., Torres-Méndez, F., Molina-García, V. M., Prieto-Martínez, F. D., Medina-Franco, J. L., & Quintana, J. C. (2024). Molecular similarity: Theory, applications, and perspectives. *Artificial Intelligence Chemistry*, 2(2), 100077.

Zhou, Y., Zhang, Y., Zhao, D., Yu, X., Shen, X., Zhou, Y., Wang, S. S., Qiu, Y. Q., Chen, Y. Z., & Zhu, F. (2024). TTD: Therapeutic Target Database describing target druggability information. *Nucleic Acids Research*, 52(D1), D1465–D1477.

Chetry, A. B., et al. (2025). From molecules to data: the emerging impact of chemoinformatics in chemistry. *Frontiers in Chemistry*, 13, 1601456.

Willett, P. (2020). The Literature of Chemoinformatics: 1978–2018. *Journal of Cheminformatics*, 12, 46.

Bajorath, J. (2024). Milestones in chemoinformatics: global view of the field. *Drug Discovery Today*, 29(11), 104154.

Medina-Franco, J. L., et al. (2022). Chemical Multiverse: An Expanded View of Chemical Space. *Molecular Informatics*, 42(2), e2200116.

Weininger, D. (1988). SMILES, a chemical language and information system. 1. Introduction to methodology and encoding rules. *Journal of Chemical Information and Computer Sciences*, 28(1), 31–36.

Weininger, D., Weininger, A., & Weininger, J. L. (1989). SMILES. 2. Algorithm for generation of unique SMILES notation. *Journal of Chemical Information and Computer*

Sciences, 29(2), 97–10.

Dalby, A., Nourse, J. G., Hounshell, W. D., Gushurst, A. K. I., Grier, D. L., Leland, B. A., & Laufer, J. (1992). Description of several chemical structure file formats used by computer programs developed at Molecular Design Limited. *Journal of Chemical Information and Computer Sciences*, 32(3), 244–255.

MDL Information Systems, Inc. (1999). CTfile Formats (December 1999). MDL Information Systems, Inc.

Cao, Y., Charisi, A., Cheng, L.-C., Jiang, T., & Girke, T. (2008). ChemmineR: a compound mining framework for R. *Bioinformatics*, 24(15), 1733–1734.

ChemmineR. SDFset-class: Class “SDFset”. Bioconductor / rdr documentation.

ChemmineR Manual. ChemmineR: Cheminformatics Toolkit for R. Bioconductor manual.

López-Pérez, K., Avellaneda-Tamayo, J. F., Chen, L., López-López, E., Juárez-Mercado, K. E., Medina-Franco, J. L., & Miranda-Quintana, R. A. (2024). Molecular similarity: Theory, applications, and perspectives. *Artificial Intelligence Chemistry*, 2, 100077.

O’Boyle, N. M., Boström, J., Sayle, R. A., & Gill, A. (2016). Comparing structural fingerprints using a literature-based similarity benchmark. *Journal of Cheminformatics*, 8, 36.

Willett, P. (2003). Similarity-based approaches to virtual screening. *Biochemical Society Transactions*, 31(3).

Cereto-Massagué, A., Ojeda, M. J., Valls, C., Mulero, M., Garcia-Vallvé, S., & Pujadas, G. (2015). Molecular fingerprint similarity search in virtual screening.

Rogers, D., & Hahn, M. (2010). Extended-connectivity fingerprints. *Journal of Chemical Information and Modeling*, 50(5), 742–754.

Willett, P. (2006). Similarity-based virtual screening using 2D fingerprints. *Drug Discovery Today*, 11(23–24), 1046–1053. <https://doi.org/10.1016/j.drudis.2006.10.005>.

MacQueen, J. B. (1967). Some methods for classification and analysis of multivariate observations. In Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability

Sammon, J. W. Jr. (1969). A nonlinear mapping for data structure analysis. IEEE Transactions on Computers, 18(5), 401–409.

Hout MC, Papesh MH, Goldinger SD. Multidimensional scaling. Wiley Interdisciplinary Reviews: Cognitive Science. 2013;4(1):93–103. doi:10.1002/wcs.1203.

Raymond, J. W., & Willett, P. (2002). Maximum common subgraph isomorphism algorithms for the matching of chemical structures. Journal of Computer-Aided Molecular Design, 16(7), 521–533.

Bemis, G. W., & Murcko, M. A. (1996). The properties of known drugs. 1. Molecular frameworks. Journal of Medicinal Chemistry, 39(15), 2887–2893.

Langdon, S. R., Brown, N., & Blagg, J. (2011). Scaffold Diversity of Exemplified Medicinal Chemistry Space. Journal of Chemical Information and Modeling, 51, 2174–2185.

R Core Team. R: A language and environment for statistical computing. R Foundation for Statistical Computing, <https://www.r-project.org/>

Python Software Foundation. Python documentation / What is Python?
<https://docs.python.org/>

PubChemR. Interface to the ‘PubChem’ Database for Chemical Data Retrieval. CRAN package documentation. <https://cran.r-project.org/web/packages/PubChemR/PubChemR>

PubChemPy. PubChemPy documentation, <https://docs.pubchempy.org/en/latest/>

fmcsR. fmcsR package documentation / vignette. Bioconductor.

pandas documentation, https://pandas.pydata.org/docs/user_guide/index.html

RDKit documentation, <https://www.rdkit.org/docs/GettingStartedInPython.html>

NumPy documentation, <https://numpy.org/doc/stable/>

TTD documentation, <https://ttd.idrblab.cn>

Wang, Y., et al. (2020). Therapeutic target database 2020: enriched resource for facilitating research and early development of targeted therapeutics. *Nucleic Acids Research*, 48(D1), D1031–D1041.

Kim, S., Chen, J., Cheng, T., Gindulyte, A., He, J., He, S., Li, Q., Shoemaker, B. A., Thiessen, P. A., Yu, B., Zaslavsky, L., Zhang, J., & Bolton, E. E. (2023). PubChem 2023 update.

Baghal-Sadriforoush, S., Bagheri, M., Abdi Rad, I., & Sotoodehnejadmatalahi, F. (2022). Melatonin Sensitizes OVCAR-3 Cells to Cisplatin through Suppression of PI3K/Akt Pathway. *Cellular and Molecular Biology*, 68(4), 158–169.

Bodurka, D. C., Levenback, C., Wolf, J. K., Gano, J., Wharton, J. T., Kavanagh, J. J., & Gershenson, D. M. (2003). Phase II trial of irinotecan in patients with metastatic epithelial ovarian cancer or peritoneal cancer. *Journal of Clinical Oncology*, 21(2), 291–297.

Brum, G., Carbone, T., Still, E., & Correia, V. (2013). N-acetylcysteine potentiates doxorubicin-induced ATM and p53 activation in ovarian cancer cells. *International Journal of Oncology*, 42(1), 211–218.

Chan, J. K., Manuel, M. R., Ciaravino, G., Cheung, M. K., Husain, A., & Teng, N. N. H. (2006). Safety and efficacy of thalidomide in recurrent epithelial ovarian and peritoneal carcinoma. *Gynecologic Oncology*, 103(3), 919–923.

Downs, L. S., Jr., Judson, P. L., Argenta, P. A., Ghebre, R., Geller, M. A., Bliss, R. L., Boente, M. P., Nahhas, W. A., Abu-Ghazaleh, S., Chen, M. D., & Carson, L. F. (2008). A prospective randomized trial of thalidomide with topotecan compared with topotecan alone in women with recurrent epithelial ovarian carcinoma. *Cancer*, 112(2), 331–339.

Guo, F., Yang, Z., Kulbe, H., Albers, A. E., Sehouli, J., & Kaufmann, A. M. (2019). Inhibitory effect on ovarian cancer ALDH⁺ stem-like cells by Disulfiram and Copper treatment through ALDH and ROS modulation. *Biomedicine & Pharmacotherapy*, 118, 109371.

Hilman, S., Koh, P. K., Collins, S., & Allerton, R. (2010). The use of treosulfan and gemcitabine in the treatment of platinum-resistant ovarian cancer. *Oncology Letters*, 1(1), 209–213.

- Keldsen, N., Madsen, E. L., Havsteen, H., Kamby, C., Laursen, L., & Sandberg, E. (1998). Oral treosulfan as second-line treatment in platinum-resistant ovarian cancer: A phase II study. The Danish Ovarian Cancer Study Group. *Gynecologic Oncology*, 69(2), 100–102.
- Kim, J.-H., Jeong, S.-J., Kim, B., Yun, S.-M., Choi, D. Y., & Kim, S.-H. (2012). Melatonin synergistically enhances cisplatin-induced apoptosis via the dephosphorylation of ERK/p90 ribosomal S6 kinase/heat shock protein 27 in SK-OV-3 cells.
- Kindlon, E. A., & Pidgeon, G. P. (2025). N-Acetyl Cysteine as a promising therapeutic approach in ovarian cancer: potential and perspectives. *Academic Oncology*, 2(2), 10.20935/acadonco7784.
- Matulonis, U., Berlin, S., Lee, H., Whalen, C., Obermayer, E., Penson, R., Liu, J., Campos, S., Krasner, C., & Horowitz, N. (2015). Phase I study of combination of vorinostat, carboplatin, and gemcitabine in women with recurrent, platinum-sensitive epithelial ovarian, fallopian tube, or peritoneal cancer. *Cancer Chemotherapy and Pharmacology*, 76(2), 417–423.
- Meier, W., du Bois, A., Reuss, A., Kuhn, W., Olbricht, S., Gropp, M., Richter, B., Lück, H.-J., Kimmig, R., & Pfisterer, J. (2009). Topotecan versus treosulfan, an alkylating agent, in patients with epithelial ovarian cancer and relapse within 12 months following 1st-line platinum/paclitaxel chemotherapy. A prospectively randomized phase III trial by the Arbeitsgemeinschaft Gynaekologische Onkologie Ovarian Cancer Study Group (AGO-OVAR). *Gynecologic Oncology*, 114(2), 199–205.
- Meteran, H., Knudsen, A. Ø., Jørgensen, T. L., Nielsen, D., & Herrstedt, J. (2024). Carboplatin plus Paclitaxel in Combination with the Histone Deacetylase Inhibitor, Vorinostat, in Patients with Recurrent Platinum-Sensitive Ovarian Cancer. *Journal of Clinical Medicine*, 13(3), 897.
- Modesitt, S. C., Sill, M., Hoffman, J. S., & Bender, D. P. (2008). A phase II study of vorinostat in the treatment of persistent or recurrent epithelial ovarian or primary peritoneal carcinoma: A Gynecologic Oncology Group study. *Gynecologic Oncology*, 109(2), 182–186.
- Musa, F., Pothuri, B., Blank, S. V., Ling, H. T., Speyer, J. L., Curtin, J., Boyd, L., Li, X., Goldberg, J. D., Muggia, F., & Tiersten, A. (2017). Phase II study of irinotecan in

combination with bevacizumab in recurrent ovarian cancer. *Gynecologic Oncology*, 144(2), 279–284.

Olesen, K. D., Larsen, A. T. R., Jensen, L. H., Steffensen, K. D., & Søndergaard, S. R. (2021). Treosulfan in platinum-resistant ovarian cancer. *International Journal of Gynecological Cancer*, 31(7), 1045–1051.

Shoji, T., Takatori, E., Omi, H., Kagabu, M., Honda, T., Futagami, M., Yokoyama, Y., Kaiho, M., Tokunaga, H., Otsuki, T., Takano, T., Yaegashi, N., Kojimahara, T., Ohta, T., Nagase, S., Soeda, S., Watanabe, T., Nishiyama, H., & Sugiyama, T. (2017). A phase II study of irinotecan and pegylated liposomal doxorubicin in platinum-resistant recurrent ovarian cancer (Tohoku Gynecologic Cancer Unit 104 study). *Cancer Chemotherapy and Pharmacology*, 80(2), 355–361.

Sugiyama, T., Yakushiji, M., Nishida, T., Ushijima, K., Okura, N., Kigawa, J., & Terakawa, N. (1998). Irinotecan (CPT-11) combined with cisplatin in patients with refractory or recurrent ovarian cancer. *Cancer Letters*, 128(2), 211–218.

Tang, B., Wu, M., Zhang, L., Jian, S., Lv, S., Lin, T., Zhu, S., Liu, L., Wang, Y., Yi, Z., & Jiang, F. (2023). Combined treatment of disulfiram with PARP inhibitors suppresses ovarian cancer. *Frontiers in Oncology*, 13, 1154073.

Zemła, A., Grzegorek, I., Dzięgiel, P., & Jabłońska, K. (2017). Melatonin Synergizes the Chemotherapeutic Effect of Cisplatin in Ovarian Cancer Cells Independently of MT1 Melatonin Receptors. *In Vivo*, 31(5), 801–809.

Zhang, X., Wang, Y.-N., Zhu, J.-J., Liu, X.-X., You, H., Gong, M.-Y., Zou, M., Cheng, W.-H., & Zhu, J.-H. (2016). N-acetylcysteine negatively regulates Notch3 and its malignant signaling. *Oncotarget*, 7(21), 30855–30866.

7. Script

7.1 R

ttd_approved.R

```
## - Scopo: parse di P1-05-Drug_disease.txt (TTD) per mappare farmaco-malattia
con ICD-11
##   e stato clinico; aggiunge flag/categorie di approvazione e salva i
risultati.
## - Input: P1-05-Drug_disease.txt, file testuale, con drug_id, e drug_name,
gli
## id non sono gli stessi in pubchem, quindi mi prendo i drug_name per trovare
gli sdf in pubchem
## - Output:
##   - ttd_drug_disease_parsed.csv/.rds (righe farmaco×indicazione):
##     ovvero un file csv, con all'interno drug_id, drug_name e
indicazioni/approvazioni
##   - ttd_drug_disease_by_drug.csv (vista aggregata per farmaco)
## ogni farmaco viene associato a multiple patologie, quindi lo script riduce
il
## nome del farmaco ad una sola riga con le varie patologie associate.
## - Uso: Rscript query_pubchem/6_ttd_approved.R
## - Nota: categorie/regioni di approvazione derivate da keyword (euristico).

normalize_icd_field <- function(x) {
  if (is.na(x) || !nzchar(x)) return(NA_character_)
  x <- trimws(gsub("^ICD-11:\\s*", "", x))
  if (!nzchar(x) || x %in% c("N.A.", "#N/A", "N/A", "NA")) return(NA_character_)
  x
}

is_valid_drug_id <- function(x) {
  is.character(x) && nzchar(x) && grepl("[A-Z0-9]{6}$", x)
}

parse_ttd_drug_disease <- function(file_path) {
  lines <- readLines(file_path, warn = FALSE)
  rows <- vector("list", length(lines))
  r <- 0L
  drug_id <- NA_character_
  drug_name <- NA_character_

  for (ln in lines) {
```

```

if (!nzchar(trimws(ln))) next

parts <- strsplit(ln, "\t", fixed = TRUE)[[1]]
if (length(parts) < 1) next
key <- trimws(parts[1])

# Skip non-data sections quickly
if (key %in% c("TTD - Therapeutic Targets Database Full Data Download File",
              "Abbreviations:", "Title - Drug to disease mapping with ICD
identifiers")) next

if (identical(key, "TTDDRUID")) {
  val <- if (length(parts) >= 2) trimws(parts[2]) else NA_character_
  if (is_valid_drug_id(val)) {
    drug_id <- val
  }
  next
}

if (identical(key, "DRUGNAME")) {
  val <- if (length(parts) >= 2) trimws(parts[2]) else NA_character_
  # Skip header line "Drug Name"
  if (!identical(tolower(val), "drug name") && nzchar(val)) {
    drug_name <- val
  }
  next
}

if (identical(key, "INDICATI")) {
  # Skip header line for abbreviations
  if (length(parts) >= 2 && tolower(trimws(parts[2])) %in% c("indication",
"disease entry")) next

  indication <- if (length(parts) >= 2) trimws(parts[2]) else NA_character_
  if (!is.na(indication) && nzchar(indication)) {
    low <- tolower(indication)
    if (low %in% c("#n/a", "n.a.", "n/a", "na", "#na")) indication <-
NA_character_
  }
  icd_raw <- if (length(parts) >= 3) trimws(parts[3]) else NA_character_
  status <- if (length(parts) >= 4) trimws(parts[4]) else NA_character_

  # There could be extra tab-separated columns; glue them back into status
  if (length(parts) > 4) {
    status <- paste(c(status, trimws(parts[5:length(parts)])), collapse = "

```

```

")
  status <- trimws(status)
}

icd11 <- normalize_icd_field(icd_raw)

# Only add rows when we have a current valid drug id and name
if (is_valid_drug_id(drug_id) && nzchar(drug_name)) {
  r <- r + 1L
  rows[[r]] <- data.frame(
    drug_id = drug_id,
    drug_name = drug_name,
    indication = indication,
    icd11 = icd11,
    clinical_status = status,
    stringsAsFactors = FALSE
  )
}
next
}

# ignore other keys and header lines
}

if (r == 0L) return(data.frame(drug_id=character(), drug_name=character(),
indication=character(), icd11=character(), clinical_status=character(),
stringsAsFactors = FALSE))
df <- do.call(rbind, rows[seq_len(r)])

# Basic cleanup: drop rows without both identifiers
df <- df[!is.na(df$drug_id) & nzchar(df$drug_id) & !is.na(df$drug_name) &
nzchar(df$drug_name), ]
rownames(df) <- NULL
df
}

add_approval_flags <- function(df) {
  if (!nrow(df)) return(transform(df, approved = logical(0), approval_category =
character(0), approval_region = character(0)))
  s <- tolower(df$clinical_status)

  approved <- grepl("^approved", s)

  approval_category <- ifelse(grepl("^approved", s), "approved",
                             ifelse(grepl("phase\\s*4", s), "phase 4",

```

```

        ifelse(grepl("phase\\s*3", s), "phase 3",
        ifelse(grepl("phase\\s*2/\\s*3|phase\\s*2/3", s), "phase
2/3",

        ifelse(grepl("phase\\s*2", s), "phase 2",
        ifelse(grepl("phase\\s*1", s), "phase 1",
        ifelse(grepl("phase\\s*0", s), "phase 0",
        ifelse(grepl("preclinical", s), "preclinical",
        ifelse(grepl("investigative", s), "investigative",
        ifelse(grepl("discontinued", s), "discontinued",
NA_character_)))))))))

# Regions
approval_region <- ifelse(grepl("china", s), "China",
        ifelse(grepl("\\beu\\b|europe|european", s), "EU",
        ifelse(grepl("\\bus\\b|usa|united states|fda", s), "US",
        ifelse(grepl("uk|united kingdom|mhra", s), "UK",
        ifelse(grepl("japan|pmda", s), "Japan",
        ifelse(grepl("australia|tga", s), "Australia",
NA_character_))))))

df$approved <- approved
df$approval_category <- approval_category
df$approval_region <- approval_region
df
}

get_input_path <- function() {
  candidates <- c("P1-05-Drug_disease.txt")
  for (p in candidates) if (!is.na(p) && file.exists(p)) return(p)
  return(NA_character_)
}

if (sys.nframe() == 0L) {
  input_path <- get_input_path()
  if (is.na(input_path)) {
    # Try absolute known path within this project
    maybe <- "c:/Users/Erica
Cogliano/OneDrive/Documenti/tesi/ttd-drugs-approved/P1-05-Drug_disease.txt"
    if (file.exists(maybe)) input_path <- maybe
  }
  if (is.na(input_path)) stop("Input file P1-05-Drug_disease.txt not found; run
from the project root or oc-gdsc-unito folder.")

  message("Reading: ", input_path)
  df <- parse_ttd_drug_disease(input_path)

```

```

message("Parsed rows: ", nrow(df))
message("Unique drugs: ", length(unique(df$drug_id)))

df <- add_approval_flags(df)

message("\nPreview (first 10):")
print(utils::head(df, 10))

message("\nUnique clinical_status examples:")
print(utils::head(sort(unique(df$clinical_status)), 20))

# Save outputs next to input
root_dir <- normalizePath(file.path(dirname(normalizePath(input_path)), ".."))
out_dir <- '.'
if (!dir.exists(out_dir)) dir.create(out_dir, recursive = TRUE)
out_csv <- file.path(out_dir, "ttd_drug_disease_parsed.csv")
out_rds <- file.path(out_dir, "ttd_drug_disease_parsed.rds")
utils::write.csv(df, out_csv, row.names = FALSE, fileEncoding = "UTF-8")
saveRDS(df, out_rds)
message("\nSaved: ", out_csv)
message("Saved: ", out_rds)

# Aggregated view: one row per drug with concatenated indications and ICD
codes
agg <- do.call(rbind, lapply(split(df, df$drug_id), function(dd) {
  data.frame(
    drug_id = dd$drug_id[1],
    drug_name = dd$drug_name[1],
    indications = paste(sort(unique(na.omit(dd$indication))), collapse = ";
"),
    icd11_codes = paste(sort(unique(na.omit(dd$icd11))), collapse = "; "),
    any_approved = any(dd$approved, na.rm = TRUE),
    approval_categories = paste(sort(unique(na.omit(dd$approval_category))),
collapse = "; "),
    approval_regions = paste(sort(unique(na.omit(dd$approval_region))),
collapse = "; "),
    stringsAsFactors = FALSE
  )
}))
out_csv_agg <- file.path(out_dir, "ttd_drug_disease_by_drug.csv")
utils::write.csv(agg, out_csv_agg, row.names = FALSE, fileEncoding = "UTF-8")
message("Saved: ", out_csv_agg)
}

```

filter_ovarian_cancer.R

```
## lo script prende in input l output dello script 6 ovvero il file
ttd_drug_disease_parsed.rds
## ottengo in output: ttd_drug_disease_ovarian_by_drug.csv,
## ttd_drug_disease_ovarian_by_drug-approved.csv,
ttd_drug_disease_ovarian_only.csv
## in questo script si crea un filtro per il file in input (che contiene tutti i
farmaci)
## e li filtra per ovarian cancer
get_parsed_df <- function() {
  candidates <- c(
    "ttd_drug_disease_parsed.rds",
    "ttd_drug_disease_parsed.csv"
  )
  path <- NA_character_
  for (p in candidates) if (file.exists(p)) { path <- p; break }
  if (is.na(path)) stop("Parsed file not found. Run 6_ttd_approved.R first to
generate parsed outputs.")

  if (grepl("\\.rds$", tolower(path))) {
    df <- readRDS(path)
  } else {
    df <- utils::read.csv(path, stringsAsFactors = FALSE)
  }
  list(df = df, parsed_path = path)
}

ensure_approval_flags <- function(df) {
  if (!all(c("approved", "approval_category", "approval_region") %in% names(df)))
  {
    s <- tolower(df$clinical_status)
    df$approved <- grepl("^approved", s)
    df$approval_category <- ifelse(grepl("^approved", s), "approved",
                                   ifelse(grepl("phase\\s*4", s), "phase 4",
                                   ifelse(grepl("phase\\s*3", s), "phase 3",
                                   ifelse(grepl("phase\\s*2/\\s*3|phase\\s*2/3", s),
"phase 2/3",
                                   ifelse(grepl("phase\\s*2", s), "phase 2",
                                   ifelse(grepl("phase\\s*1", s), "phase 1",
                                   ifelse(grepl("phase\\s*0", s), "phase 0",
                                   ifelse(grepl("preclinical", s), "preclinical",
                                   ifelse(grepl("investigative", s), "investigative",
                                   ifelse(grepl("discontinued", s), "discontinued",
NA_character_))))))))))
  }
```

```

df$approval_region <- ifelse(grepl("china", s), "China",
                              ifelse(grepl("\\beu\\b|europe|european", s), "EU",
                                        ifelse(grepl("\\bus\\b|usa|united states|fda", s),
"US",
                                        ifelse(grepl("uk|united kingdom|mhra", s), "UK",
                                                ifelse(grepl("japan|pmda", s), "Japan",
                                                        ifelse(grepl("australia|tga", s), "Australia",
NA_character_))))))
}
df
}

filter_ovarian <- function(df) {
  s <- tolower(df$indication)
  mask_ind <- !is.na(s) & grepl("\\bovarian\\b|\\bovary\\b", s)
  mask_icd <- !is.na(df$icd11) & grepl("^2C73", df$icd11)
  df[mask_ind | mask_icd, , drop = FALSE]
}

aggregate_by_drug <- function(df) {
  if (!nrow(df)) return(df)
  do.call(rbind, lapply(split(df, df$drug_id), function(dd) {
    data.frame(
      drug_id = dd$drug_id[1],
      drug_name = dd$drug_name[1],
      indications = paste(sort(unique(na.omit(dd$indication))), collapse = ";
"),
      icd11_codes = paste(sort(unique(na.omit(dd$icd11))), collapse = "; "),
      any_approved = any(dd$approved, na.rm = TRUE),
      approval_categories = paste(sort(unique(na.omit(dd$approval_category))),
collapse = "; "),
      approval_regions = paste(sort(unique(na.omit(dd$approval_region))),
collapse = "; "),
      stringsAsFactors = FALSE
    )
  }))
}

if (sys.nframe() == 0L) {
  df_info <- get_parsed_df()
  df <- ensure_approval_flags(df_info$df)
  message("Total rows: ", nrow(df), " | Unique drugs: ",
length(unique(df$drug_id)))

  ov <- filter_ovarian(df)

```

```

message("Ovarian-related rows: ", nrow(ov), " | Unique drugs: ",
length(unique(ov$drug_id)))

# Save detailed rows and aggregated views next to the parsed dataset
out_dir <- dirname(normalizePath(df_info$parsed_path))
out_rows <- file.path(out_dir, "ttd_drug_disease_ovarian_only.csv")
out_rows_approved <- file.path(out_dir,
"ttd_drug_disease_ovarian_only_approved.csv")
utils::write.csv(ov, out_rows, row.names = FALSE, fileEncoding = "UTF-8")

ov_agg <- aggregate_by_drug(ov)
out_agg <- file.path(out_dir, "ttd_drug_disease_ovarian_by_drug.csv")
utils::write.csv(ov_agg, out_agg, row.names = FALSE, fileEncoding = "UTF-8")

ov_approved <- subset(ov, approved)
ov_agg_approved <- aggregate_by_drug(ov_approved)
utils::write.csv(ov_approved, out_rows_approved, row.names = FALSE,
fileEncoding = "UTF-8")
out_agg_approved <- file.path(out_dir,
"ttd_drug_disease_ovarian_by_drug_approved.csv")
utils::write.csv(ov_agg_approved, out_agg_approved, row.names = FALSE,
fileEncoding = "UTF-8")

message("Saved: ", out_rows)
message("Saved: ", out_rows_approved)
message("Saved: ", out_agg)
message("Saved: ", out_agg_approved)

message("\nPreview ovarian (first 10):")
print(utils::head(ov, 10))
}

```

query_sdf_data_frame.R

```

## prendiamo i drug_name da ttd_drug_disease_ovarian_by_drug.csv
## e proviamo a cercare gli sdf su pubchem attraverso la funzione get_sdf()
## usando il nome dei farmaci come query e sia substance che compound come
dominio
## output: sdf_results.csv, ovvero un dataframe con i percorsi ai file sdf che
## sono stati creati da get_sdf

# Installa i pacchetti solo se mancanti
if (!requireNamespace("devtools", quietly = TRUE)) {

```

```

install.packages("devtools")
}
library(devtools)
if (!requireNamespace("PubChemR", quietly = TRUE)) {
  devtools::install_github("selcukorkmaz/PubChemR")
}
library(PubChemR)

# Funzione per processare il dataframe e cercare gli SDF
# Prende in input un dataframe con almeno le colonne "drug_name" e
"approval_categories"
# Restituisce un dataframe con i risultati delle ricerche SDF
# Nel contempo, salva i file SDF trovati nelle cartelle sdf/compound e
sdf/substance
# Per ogni farmaco, cerca prima tra i compound e poi tra le substance
# Se non trova il farmaco in nessuno dei due domini, lascia NA nei campi
sdf_path e domain
process_drug_data <- function(df_ovarian_drugs, sdf_output_dir = "sdf") {
  # Cartella dove voglio salvare i singoli file SDF
  cartella_compound <- file.path(sdf_output_dir, "compound")
  if (!dir.exists(cartella_compound)) {
    dir.create(cartella_compound, recursive = TRUE)
  }
  cartella_substance <- file.path(sdf_output_dir, "substance")
  if (!dir.exists(cartella_substance)) {
    dir.create(cartella_substance, recursive = TRUE)
  }

  # Validazione colonne richieste
  required_cols <- c("drug_name", "approval_categories")
  missing_cols <- setdiff(required_cols, colnames(df_ovarian_drugs))
  if (length(missing_cols) > 0) {
    stop(sprintf(
      "Colonne mancanti nel file di input: %s",
      paste(missing_cols, collapse = ", ")
    ))
  }

  # Inizializza un data frame to salvare i risultati
  # e altre informazioni utili
  results <- data.frame(
    drug_name = character(),
    sdf_path = character(),
    domain = character(),
    approval_categories = character(),

```

```

stringsAsFactors = FALSE
)

sleep_seconds <- 0.5

# Itera per riga usando l'indice
for (i in seq_len(nrow(df_ovarian_drugs))) {
  iter_row <- df_ovarian_drugs[i, , drop = FALSE]
  drug_name <- iter_row$drug_name
  approval_cat <- iter_row$approval_categories

  # Crea una nuova riga nel data frame dei risultati
  new_row <- data.frame(
    drug_name = drug_name,
    sdf_path = NA,
    domain = NA,
    approval_categories = approval_cat,
    stringsAsFactors = FALSE
  )

  cat("Cerco", drug_name, "nei compound..\n")
  Sys.sleep(sleep_seconds)
  sdf_path <- tryCatch(
    get_sdf(
      drug_name,
      namespace = "name",
      domain = "compound",
      path = cartella_compound
    ),
    error = function(e) {
      message("Errore get_sdf (compound) per ", drug_name, ": ", e$message)
      NULL
    }
  )

  if (!is.null(sdf_path)) {
    # Se il file SDF e' stato trovato, aggiorna il data frame dei risultati
    new_row$sdf_path <- sdf_path
    new_row$domain <- "compound"
  }

  # Se invece non e' stato trovato tra i compound, cerco tra le substance
  if (is.null(sdf_path)) {
    cat("Cerco", drug_name, "nelle substance..\n")
    Sys.sleep(sleep_seconds)
  }
}

```

```

sdf_path <- tryCatch(
  get_sdf(
    drug_name,
    namespace = "name",
    domain = "substance",
    path = cartella_substance
  ),
  error = function(e) {
    message("Errore get_sdf (substance) per ", drug_name, ": ", e$message)
    NULL
  }
)

if (!is.null(sdf_path)) {
  # Se il file SDF e' stato trovato, aggiorna il data frame dei risultati
  new_row$sdf_path <- sdf_path
  new_row$domain <- "substance"
}

# Se il file SDF non e' stato trovato in nessuno dei due domini
if (is.null(sdf_path)) {
  cat("Drug non trovato:", drug_name, "\n")
}

# Aggiungi la nuova riga al data frame dei risultati
results <- rbind(results, new_row)
}

results
}

if (sys.nframe() == 0L) {
  # Percorso del file di input (relativo alla root del progetto "tesi")
  file_input <- "ttd_drug_disease_ovarian_by_drug.csv"
  # leggo il file importandolo come dataframe (stringhe come caratteri)
  df_ovarian_drugs <- read.csv(file_input, stringsAsFactors = FALSE)

  results <- process_drug_data(df_ovarian_drugs)

  # Salva i risultati in un file CSV
  output_file <- "sdf_results.csv"
  write.csv(results, output_file, row.names = FALSE)
}

```

sdf_join.R

```
# input: sdf_result.csv
# - dal file sdf_result.csv ho i percorsi degli SDF (non l'intero sdf per drug)
# devo leggere ciascun percorso ed ottenere il contenuto del file
corrispondente
# output:
# - all_sdfs.sdf - Tutti gli sdf cosi' letti li unisco in un unico file
all_sdfs.sdf
# - all_sdfs_filtered.sdf - Filtro gli sdf in base alla grandezza (numero di
atomi)

setwd("query_pubchem")
source("sdf_helper.R")

sdf_results <- read.csv("sdf_results.csv")

list_path_sdf <- sdf_results$sdf_path

# Inizializziamo il file di output
if (file.exists("all_sdfs.sdf")) {
  file.remove("all_sdfs.sdf")
}

for (path in list_path_sdf) {
  if (!is.na(path) && file.exists(path)) {
    # Leggiamo il contenuto del file sdf come stringa
    sdf_read <- readLines(path)
    # Concateniamo il contenuto del file appena letto al file all_sdfs.sdf
    write(sdf_read, file = "all_sdfs.sdf", append = TRUE)
  }
}

cat("Tutti gli SDF sono stati uniti in all_sdfs.sdf\n")

# Filtro per grandezza della molecola
atom_max <- 100

all_sdfs <- read.SDFset("all_sdfs.sdf")
filtered_sdf_list <- list()
filtered_names <- character()

for (i in seq_along(all_sdfs)) {
  current_sdf <- all_sdfs[[i]]
  atom_count <- sdf_atom_count(current_sdf)
```

```

if (atom_count < atom_max) {
  filtered_sdf_list <- c(filtered_sdf_list, current_sdf)
  molecule_name <- sdf_molecule_name(current_sdf)
  filtered_names <- c(filtered_names, molecule_name)
}
}

filtered_sdfs <- SDFset(filtered_sdf_list, filtered_names)
cid(filtered_sdfs) <- makeUnique(cid(filtered_sdfs))
cat("Salvo", length(filtered_sdfs), "SDF con meno di", atom_max, "atomi\n")
write.SDF(filtered_sdfs, "all_sdfs_filtered.sdf")

```

sdf_similarita.R

```

# input: all_sdfs_filtered.sdf contiene le molecole piccola da noi filtrate
#        nello script precedente
# obiettivo: similarita' strutturale,
# utilizzo fmcs per comparare i compound e generare una matrice di similarita'

source("sdf_helper.R")

if (!requireNamespace("fmcsR", quietly = TRUE)) {
  cat("Pacchetto 'fmcsR' non trovato. Provo ad installarlo...\n")
  BiocManager::install("fmcsR")
}

setwd("query_pubchem")

# Leggiamo gli SDF
sdfset <- read.SDFset("all_sdfs_filtered.sdf")

# Verifichiamo che abbiamo caricato almeno qualche composto
cat("Composti caricati:", length(sdfset), "\n")

# Matrice di similarita' con metodo iterativo
n <- length(sdfset)
mat_sim <- matrix(nrow = n, ncol = n)

# Inizializziamo i nomi di righe e colonne con vettori di
# lunghezza n con stringhe vuote
colnames(mat_sim) <- character(n)
rownames(mat_sim) <- character(n)

```

```

for (i in 1:n) {
  # Siccome non ho un singolo nome per ogni sdf, utilizzo il molecule name
  # fornito dal sdf stesso come nomi di riga e colonne
  name <- header(sdfset[[i]])["Molecule_Name"]
  colnames(mat_sim)[i] <- name
  rownames(mat_sim)[i] <- name
}

cartella_fmcs<-"fmcs"
if (!dir.exists(cartella_fmcs)) {
  dir.create(cartella_fmcs)
}

# Calcore l'MCS tra tutte le molecole e salvare il risultato in un file rds.
for (i in 1:n) {
  name <- header(sdfset[[i]])["Molecule_Name"]
  # Ogni molecola avra' un suo file dove salviamo il risultato della fmcs batch
  current_fmcs_path = paste(cartella_fmcs, "/", i , "-fmcs-", name, ".rds",
  sep="")

  # Se il file esiste gia', saltiamo e passa al prossimo
  if (!file.exists(current_fmcs_path)) {
    # Il file non esiste, quindi calcoliamo e salviamo.
    cat(i, ": fmcsBatch: ", name, "\n")
    result <- fmcsBatch(sdfset[i], sdfset[1:n])
    # Salviamo il risultato delle varie fmcsBatch in un file
    cat("Saving to: ", current_fmcs_path, "\n")
    saveRDS(result, current_fmcs_path)
  }
}

files <- list.files(path=cartella_fmcs)
# prendo i tanimoto coefficient e salvo nella matrice di similarita',
# ogni file contiene la colonna numero 4 degli fmcs che salviamo come riga nella
# matrice di similarita'
for (file_name in files) {
  nstr<-strsplit(file_name, split="-")[[1]][1]
  riga<-as.integer(nstr)
  file_path <- paste(cartella_fmcs, file_name, sep="/")
  cur_fmcs<-readRDS(file_path)
  temporanea<-cur_fmcs[,4]
  mat_sim[riga,] <- temporanea
}

```

```

# Filtro 1
# puliamo, creiamo un vettore booleano che ci dice quali righe hanno valori NA o
NAN
righe_brutte<-apply(mat_sim,1, function(riga) any(is.na(riga)) ||
any(is.nan(riga)))
mat_sim_pulita <- mat_sim[!righe_brutte, !righe_brutte, drop=TRUE]

# Filtro 2
# Andare a vedere le righe che hanno piu di un uno,
# per ogni riga n controllare se ci sono 1 in posizioni non n
# quindi eliminare tutti quei valori 1 che non si trovano nella diagonale.
# L'idea e' che se 1 non appare nella diagonale, vuol dire che le due molecole
# non sono la stessa (diverso ID) ma sono identiche tra loro (similarita'
massima)

numero_di_righe <- length(mat_sim_pulita[1,])
righe_ripetute = rep(FALSE, numero_di_righe)

for (i in 1:numero_di_righe) {
  for (j in i:numero_di_righe) {
    if (i != j) {
      righe_ripetute[i] = righe_ripetute[i] | mat_sim_pulita[i,j] == 1.0
    }
  }
}

mat_sim_no_repeat <- mat_sim_pulita[!righe_ripetute, !righe_ripetute,
drop=TRUE]

# Filtro 3
# Rimuovere dalla matrice le righe e colonne duplicate (che hanno lo stesso
nome)
duplicati <- which(duplicated(rownames(mat_sim_no_repeat)))
if (length(duplicati) > 0) {
  mat_sim_no_repeat <- mat_sim_no_repeat[-duplicati, -duplicati, drop=FALSE]
}

## Salviamo anche la matrice finale senza duplicati per uso nello script 14
output_path <- "matrice_similarity_fmcs.csv"
write.csv(mat_sim_no_repeat, output_path)
cat("Salvata matrice in:", output_path , "\n")

```

```

## Script: 13_tanimoto_to_distance_cmdscale.R
## Scopo: partire da una matrice di similarità (coefficiente di Tanimoto in
[0,1])
## Script : da matrice di similarità (Tanimoto) a matrice di distanza
## Input fisso: 'matrice_similarita_fmcs.csv' generato dallo script 12
## Output: 'matrice_distanza_fmcs.csv'

## 1) Leggi la matrice di similarità dal file CSV
file_sim <- "matrice_similarita_fmcs.csv"
if (!file.exists(file_sim)) {
  stop("File di similarità non trovato: ", file_sim,
       "\nAssicurati di aver eseguito lo script 13 che crea
mat_sim_no_repeat.csv")
}

cat("Leggo la matrice di similarità da: ", file_sim, "\n", sep = "")
sim_df <- read.csv(file_sim, row.names = 1, check.names = FALSE)
sim <- as.matrix(sim_df)

## 2) Controlli di base
if (nrow(sim) != ncol(sim)) {
  stop("La matrice deve essere quadrata (stesso numero di righe e colonne)")
}

## 3) Sistemiamo la matrice di similarità in modo semplice
## - Rendiamo simmetrica facendo la media con la trasposta
sim <- (sim + t(sim)) / 2
## - Rimpiazziamo eventuali NA con 0 (nessuna similarità)
sim[is.na(sim)] <- 0
## - Limitiamo i valori al range [0,1]
sim[sim < 0] <- 0
sim[sim > 1] <- 1
## - Mettiamo la diagonale a 1 (ogni elemento è uguale a se stesso)
diag(sim) <- 1

## 4) Converti da similarità a distanza:  $D = 1 - S$ 
dist_mat <- 1 - sim
## - Metti la diagonale a 0 (distanza con se stesso è zero)
diag(dist_mat) <- 0
## - Simmetrizza per sicurezza
dist_mat <- (dist_mat + t(dist_mat)) / 2

## 5) Salva la matrice di distanza
output_file <- "matrice_distanza_fmcs.csv"
write.csv(dist_mat, output_file)

```

```
cat("Salvata matrice di distanza in: ", output_file, "\n", sep = "")
```

pcoa_cmdscale.R

```
# Script: 14_pcoa_cmdscale.R
# Scopo: leggere una matrice di distanza da file CSV e calcolare la PCoA
(Classical MDS)
# Input:  matrice_distanza_fmcs.csv  (prodotta dallo script 13)
# Output: pcoa_coords_2.csv (coordinate dei campioni su 2 assi)
#         pcoa_eig.csv          (autovalori e varianza spiegata)
#         pcoa_plot.png         (grafico semplice 2D)

# 1) nome del file di input
file_dist <- "matrice_distanza_fmcs.csv"

# 2) Controllo che il file esista
if (!file.exists(file_dist)) {
  stop("File di distanza non trovato: ", file_dist,
       "\nAssicurati di aver eseguito prima lo script 14 che crea questo file.")
}

cat("Leggo la matrice di distanza da: ", file_dist, "\n", sep = "")

# 3) Leggo il CSV. La prima colonna contiene i nomi delle righe.
dist_df <- read.csv(file_dist, row.names = 1, check.names = FALSE)

# 4) Converto in matrice e in numerico
#     (se qualche colonna è in formato carattere, forzo a numerico)
dist_mat <- as.matrix(dist_df)
storage.mode(dist_mat) <- "numeric"

# 5) Controlli semplici
n <- nrow(dist_mat)
if (n != ncol(dist_mat)) {
  stop("La matrice deve essere quadrata: righe=", n, ", colonne=",
       ncol(dist_mat))
}

# Rendo la matrice simmetrica e metto la diagonale a 0
# (piccole differenze numeriche vengono corrette)
diag(dist_mat) <- 0
dist_mat <- (dist_mat + t(dist_mat)) / 2
```

```

# Verifico presenza di NA
na_count <- sum(is.na(dist_mat))
if (na_count > 0) {
  stop("La matrice contiene ", na_count, " valori NA.\n",
       "Rimuovi/Imputa i NA nello script 14 e riprova.")
}

# Verifico che le distanze siano >= 0
if (any(dist_mat < 0, na.rm = TRUE)) {
  cat("Avviso: trovate distanze negative. Le imposto a 0 per sicurezza.\n")
  dist_mat[dist_mat < 0] <- 0
}

# 6) Calcolo PCoA (Classical MDS) su 2 dimensioni
k <- 2
cat("Eseguo cmdscale con k=", k, "...\\n", sep="")
fit <- cmdscale(as.dist(dist_mat), k = k, eig = TRUE)

# 7) Coordinate
coords <- as.data.frame(fit$points)
colnames(coords) <- c("PCo1", "PCo2")

# 8) Autovalori e varianza spiegata (solo quelli positivi)
positive_eig <- fit$eig[fit$eig > 0]
var_prop <- positive_eig / sum(positive_eig)
eig_df <- data.frame(Eigenvalue = positive_eig,
                    VarExplained = var_prop,
                    Cumulative = cumsum(var_prop))

# 9) Salvataggi
write.csv(coords, "pcoa_coords_2.csv")
write.csv(eig_df, "pcoa_eig.csv", row.names = FALSE)
cat("Salvate coordinate in: pcoa_coords_2.csv\\n")
cat("Salvati autovalori/varianza in: pcoa_eig.csv\\n")

# 10) Grafico semplice e salvataggio PNG
png("pcoa_plot.png", width = 800, height = 600)
plot(coords$PCo1, coords$PCo2,
     xlab = "PCo1", ylab = "PCo2",
     main = "PCoA (Classical MDS)", pch = 19, col = "steelblue")
text(coords$PCo1, coords$PCo2, labels = rownames(coords), pos = 3, cex = 0.7)
dev.off()
cat("Salvato grafico in: pcoa_plot.png\\n")

# 11) Messaggio finale

```

```

if (any(fit$eig < 0)) {
  cat("Nota: sono presenti autovalori negativi (distanza non perfettamente
euclidea).\n")
  cat("      L'analisi è comunque valida per visualizzazione; per
correzioni,\n")
  cat("      si può usare ape::pcoa in uno script avanzato.\n")
}

```

sdf_join_filtered.R

```

# script preparativo per lo script 16

# Tutti gli sdf, sono in un unico file: all_sdfs_filtered.sdf # script 11

#prendo tutta la lista di percorsi, li inserisce in un solo file all_sdf
#aggiunge pero' anche quei file che non vogliamo, quindi dobbiamo filtrare in
# base alla matrice di similarita (71 farmaci)
# ha righe e colonne e ci sono gli id delle molecole che vogliamo

setwd("query_pubchem")
source("sdf_helper.R")

#sdf_results <- read.csv("sdf_results.csv")
mat_sim<-read.csv("matrice_similarita_fmcs.csv", row.names = 1)
# In mat sim ci dovrebbero essere 73 elementi

# Questa e' la lista di nomi di molecole filtrate
mat_sim_rownames = rownames(mat_sim)

all_sdfs = read.SDFset("all_sdfs_filtered.sdf")

filtered_sdf_list = list()
filtered_names = character()
all_smiles <- sdf2smiles(all_sdfs)

for (molecule_name in mat_sim_rownames) {
  current_sdf = grepSDFset(molecule_name, all_sdfs)
  current_sdfset = SDFset(current_sdf)
  smiles_set <- sdf2smiles(current_sdfset)
  # Se ne trova piu' di uno, prendi solo il primo
  current_sdf <- current_sdf[[1]]
  smiles <- as.character(smiles_set[[1]])
  if (is.na(smiles) || is.null(smiles)) {

```

```

    cat("Attenzione: la molecola", molecule_name,
        "non ha un SMILES associato. Viene saltata.\n", datablock(current_sdf),
        "\n")
    next
  }
  filtered_sdf_list <- c(filtered_sdf_list, current_sdf)
  filtered_names <- c(filtered_names, molecule_name)
}

filtered_sdfs = SDFset(filtered_sdf_list, filtered_names)

cat("Salvo", length(filtered_sdfs), "SDF filtrati\n")
write.SDF(filtered_sdfs, "filtered_sdfs.sdf")

```

ggmol.R

```

# Script: 16_ggmol.R
# Scopo: usare il pacchetto ggmol da GitHub

if (!require("BiocManager", quietly = TRUE)) {
  install.packages("BiocManager")
}

if (!require("ChemmineOB", quietly = TRUE)) {
  BiocManager::install("ChemmineOB")
}

if (!requireNamespace("devtools", quietly = TRUE)) {
  install.packages("devtools")
}

if (!requireNamespace("ggmol", quietly=TRUE)) {
  devtools::install_github("franapoli/ggmol")
}

message("ggmol installato e caricato.")

if (!requireNamespace("ggrepel", quietly=TRUE)) {
  install.packages("ggrepel")
}

if (!requireNamespace("webchem", quietly=TRUE)) {
  install.packages("webchem")
}

if (!requireNamespace("ChemmineR", quietly = TRUE)) {
  cat("Pacchetto 'ChemmineR' non trovato. Provo ad installarlo...\n")
  BiocManager::install("ChemmineR")
}

```

```

}
if (!requireNamespace("ggforce", quietly = TRUE)) install.packages("ggforce")
library(ggforce)
library(ggplot2)
library(ggrepel)
library(webchem)

if (!requireNamespace("MASS", quietly = TRUE)) {
  install.packages("MASS")
}
library(ChemmineR)
library(ggmol)
library(MASS)
library(devtools)
library(fmcsR)

source("sdf_helper.R")
setwd("4-query_pubchem")

#Se il file chemms.rds esiste nella cartella, allora posso evitare di calcolare
# con Chemmol, e caricare direttamente l'oggetto.
chemms_file_name <- "chemms.rds"
if (file.exists(chemms_file_name)) {
  chemms <- readRDS(chemms_file_name)
} else {
  #a chemmol passo gli sdf
  sdfset <- read.SDFset("filtered_sdfs.sdf")
  chemms <- Chemmol(sdfset)
  # Salva il risultato in un file, per il futuro, così da evitare di ricostruire
  # l'oggetto con Chemmol, che impiega tempo.
  saveRDS(chemms, chemms_file_name)
}

smiles_list <- character()
smiles_list_count <- length(chemms@smiles)
smiles_mask <- c()
for (i in 1:smiles_list_count) {
  smiles <- chemms@smiles@smilist[[i]]@smilist[[1]]
  smiles_list <- c(smiles_list, smiles)
  # smile assenti in chemms numero 45 e 71 (" e "*")
  # Vogliamo escluderli
  if (smiles == "" || smiles == "*") {
    smiles_mask <- c(smiles_mask, FALSE)
  }
}

```

```

    cat("Escludo smiles", i, "\n")
  } else {
    smiles_mask <- c(smiles_mask, TRUE)
  }
}
# Ora devo escludere i duplicati altrimenti ggplot non funziona.

smiles_duplicati <- which(duplicated(smiles_list))
# Aggiorna la maschera
for (i in smiles_duplicati) {
  smiles_mask[i] = FALSE
}
smiles_list <- smiles_list[smiles_mask]

#Sammon prende in input le matrici di distance, risulta un duplicato che
elimino
#stress: misura quanto le distanze nella mappa a bassa dimensione differiscono
#dalle distanze originali. Valori più bassi = migliore fedeltà della riduzione
dimensionale.
distance_df <- read.csv("matrice_distanza_fmcs.csv", row.names = 1)

distance_matrix = as.matrix(distance_df)
# Rimuovi righe corrispondenti agli smiles mancanti
distance_matrix <- distance_matrix[smiles_mask, smiles_mask]
stress_count <- 8
init <- cmdscale(distance_matrix, k=stress_count)
init_duplicates <- duplicated(init)
init <- init[!init_duplicates, , drop=FALSE]
distance_matrix <- distance_matrix[!init_duplicates, !init_duplicates,
drop=FALSE]

#SAMMON
fprints_2d <- sammon(distance_matrix, init, stress_count)$points

#Il codice costruisce una tabella (data.frame) per visualizzare i risultati di
Sammon + clustering.
#x/y: coordinate 2D (prime due colonne) prodotte da Sammon.
#name: vettore di nomi degli oggetti (deve avere la stessa lunghezza delle
righe di coords).
#smiles: colonna placeholder (qui NA).
#Cluster: fattore derivato da km$cluster (risultato di kmeans).
cluster_count <- 10
clusters <- kmeans(fprints_2d, centers=cluster_count)$cluster

# In clusted ho delle associazioni Molecule_Name - Cluster_Number.

```

```

# Idea: posso creare N SDFset (dove N=cluster_count nel nostro caso), e per
ogni
# SDFsetN costruisco l'insieme di SDF appartenenti al cluster N.

# Iniziamo col creare cluster_count SDFlists vuote
clustered_sdflists <- replicate(10, list(), simplify=FALSE)
# Assicuriamoci di lavorare sulla stessa lista di SDFs come abbiamo fatto con
gli SMILES
masked_sdfs <- chemms@sdf[smiles_mask]

# Ora in clustered_sdflists ho cluster_count SDFlist vuote.
# Popoliamole.
for (i in seq_along(clusters)) {
  # Qui i rappresenta l'i-esimo SDF di chemms@sdf
  # mentre clusters[i] rappresenta il suo cluster number
  # quindi con chemms@sdf[[i]] posso prendere l'SDF
  # e con i posso aggiungerlo alla clustered_sdflist corrispondente
  cluster_number = clusters[i]
  len <- length(clustered_sdflists[cluster_number])
  clustered_sdflists[[cluster_number]] <-
append(clustered_sdflists[[cluster_number]], masked_sdfs[[i]])
}

for (i in seq_along(clustered_sdflists)) {
  cat("Nel cluster", i, "ho", length(clustered_sdflists[[i]]), "SDFs\n")
}

# Adesso in clustered_sdfsets ho 10 SDFsets, ognugno che rappresenta un cluster

sdf_mcs_clusters <- list()

for (cluster_num in 1:cluster_count) {
  sdf_mcs_accumulato<-NULL
  mcs_accumulato<-NULL

  for (sdf_i in 1:length(clustered_sdflists[[cluster_num]])) {
    next_sdf <- clustered_sdflists[[cluster_num]][[sdf_i]]
    if (cluster_num == 4 && sdf_i == 1) {
      # La molecola 1 del cluster n 4 manda in errore mcs2sdfset, quindi la
salto
      next
    }
    if(is.null(sdf_mcs_accumulato)) {
      sdf_mcs_accumulato <- next_sdf
    } else {

```

```

    mcs_accumulato <- fmcs(sdf_mcs_accumulato, next_sdf)
    cat("cluster", cluster_num, "sdf", sdf_i, "\n")
    sdf_mcs_accumulato <- mcs2sdfset(mcs_accumulato)[[1]][[1]]
  }
}

sdf_mcs_clusters[[cluster_num]] <- sdf_mcs_accumulato
}

for (i in seq_along(sdf_mcs_clusters)) {
  plot(sdf_mcs_clusters[[i]])
}

# Collezione sinonimi a partire dai CID delle molecole
sdf_names <- rownames(distance_matrix)
alt_names <- pc_synonyms(sdf_names, from="cid")
synonyms <- c()
for (alt_name in alt_names) {
  synonyms <- c(synonyms, alt_name[1])
}

tab <- data.frame(
  x = fprints_2d[, 1],
  y = fprints_2d[, 2],
  name = synonyms,
  smiles = smiles_list,
  Cluster = as.factor(clusters)
)

indices <- c(1:14, 16:17, 19:38, 40:67)
filtered_sdfs <- chemms@sdf[indices]

# Alcuni id non sono CID (compound ID), ma sono SID (substance ID).
# Purtroppo pc_synonyms non funziona bene usando i SID, quindi
# abbiamo usato ChatGPT per creare una funzione che interroga
# PubCHEM direttamente usando HTTP GET, e dato un SID ne ottiene il primo
sinonimo.
#script supporto vedi helper
for (i in seq_along(tab$name)) {
  if (is.na(tab$name[i])) {
    id <- rownames(tab)[i]
    syn <- pc_sid_synonyms(id)
    tab$name[i] <- syn
  }
}

```

```

}

cat("Plotting\n")
ggplot(tab[indices,], aes(x, y, smiles=smiles, colour=Cluster,label=name))+
  geom_mol(show_atoms=F, resolution=700)+
  geom_label_repel(size=2.5, point.padding=17)

```

7.2 Python

helper.py

```

from enum import Enum
import io
import os
from matplotlib.offsetbox import OffsetImage, AnnotationBbox
import pubchempy as pc
import pandas as pd
import numpy as np
from sammon.sammon import sammon
from sklearn.manifold import MDS
from sklearn.cluster import AgglomerativeClustering
from tqdm import tqdm
import matplotlib.pyplot as plt
import umap
from PIL import Image

from loguru import logger

from rdkit.Chem import (
    ForwardSDMolSupplier,
    Mol,
    SDWriter,
    rdFMCS,
    rdFingerprintGenerator,
    Draw,
    rdDepictor,
    SDMolSupplier,
    DataStructs,
    MolFromSmarts,
    FastFindRings,
)

from rdkit.Chem.Draw import rdMolDraw2D
from rdkit.ML.Cluster import Butina

```

```

from sklearn.cluster import KMeans
from rdkit.Chem.Scaffolds import MurckoScaffold

def InitLogger():
    # Prendi il livello dalla variabile d'ambiente,
    # ma usa "INFO" come paracadute se non è impostata
    current_level = os.getenv("LOGURU_LEVEL", "INFO")
    logger.remove() # Rimuove il logger predefinito
    logger.add(
        lambda msg: tqdm.write(msg, end=""), colorize=True, level=current_level
    ) # Aggiunge un nuovo logger con il livello specificato

def LoadMolecules(file_path, fallback_path, include_substances=False) ->
SDMolSupplier:
    """
    Questa funzione carica le molecole da un file SDF in `file_path`. Se il file
    non esiste,
    scarica i dati da PubChem usando i nomi dei farmaci trovati in file csv in
    `fallback_path`
    e salva le MOL nel file SDF in `file_path` per usi futuri.

    - `file_path`: percorso del file SDF da cui caricare le molecole
    - `fallback_path`: percorso del file CSV da cui caricare i nomi dei farmaci
    se il file SDF non esiste
    - `include_substances`: se True, include anche le sostanze (substance) nella
    ricerca su PubChem,
    altrimenti cerca solo nei composti (compound). Nota che includere le
    sostanze può aumentare
    significativamente il tempo di ricerca e non garantisce risultati
    aggiuntivi,
    quindi è disabilitato per default.
    """
    compounds_sdf_file_name = file_path
    if not os.path.exists(compounds_sdf_file_name):
        df = pd.read_csv(fallback_path)
        # Prendi solo i nomi delle molecole nella colonna "drug_name"
        molecule_names = df["drug_name"].tolist()
        logger.info(
            "Caricati {} nomi di molecole dal file {}".format(
                len(molecule_names), fallback_path
            )
        )
    )

```

```

        logger.info(
            "Il file {} non esiste. Scaricando i dati da PubChem...".format(
                compounds_sdf_file_name
            )
        )
        mols = FindMolsForNames(molecule_names,
include_substances=include_substances)
        SaveMolsToSDF(mols, compounds_sdf_file_name)
    else:
        mols = SDMolSupplier(compounds_sdf_file_name)

    return mols

def GetListFromSDMolSupplier(supplier: SDMolSupplier) -> list[Mol]:
    """Transforma un SDMolSupplier in una lista di Mol, filtrando eventuali None
che indicano molecole non caricate correttamente"""
    return [m for m in supplier if m is not None]

def FindMolsForNames(names, include_substances=True) -> list[Mol]:
    mols = []
    for name in tqdm(names, "Cercando molecole per nomi"):
        mol = FindMolByNameWithSynonyms(name, include_substances)
        if mol is not None:
            mols.append(mol)
    logger.info(f"Trovate {len(mols)} molecole per {len(names)} nomi")
    return mols

def FindMolByName(name: str) -> Mol:
    """
    Questa funzione cerca una molecola su PubChem usando il nome e restituisce
un oggetto Mol.
    Se la ricerca fallisce o non viene trovato nulla, restituisce None.
    """
    logger.debug(f"Cercando la molecola '{name}' su PubChem")
    namespace = PubChemNamespace.CID
    domain = PubChemDomain.COMPOUND
    try:
        ids = pcp.get_cids(name, namespace="name")
    except Exception as e:
        logger.error(f"Connessione fallita per '{name}': {e}")
    return None

```

```

if ids is None or len(ids) == 0:
    logger.warning(f"Nessun ID trovato per '{name}'")
    namespace = PubChemNamespace.SID
    domain = PubChemDomain.SUBSTANCE
    ids = FindSidsBySubstanceName(name)

    # una volta ottenuti gli ID, possiamo ottenere la migliore molecola
    # associata a quegli ID
    return GetBestMol(ids, name, namespace=namespace, domain=domain)

def FindSidsBySubstanceName(name: str) -> list[int]:
    """
    Questa funzione cerca una molecola su PubChem usando il nome come sostanza e
    restituisce una lista di SID associati a quella sostanza.
    Se la ricerca fallisce o non viene trovato nulla, restituisce None.
    """
    logger.debug(f"Cercando la molecola '{name}' su PubChem come sostanza")
    try:
        sids = pcp.get_sids(name, namespace="name", domain="substance")
    except Exception as e:
        logger.error(f"Connessione fallita per '{name}' come sostanza: {e}")
        return None
    if sids is None or len(sids) == 0:
        logger.warning(f"Nessuna substance trovata per '{name}'")
        return None

    return sids

def FindCidsBySubstanceName(name: str) -> Mol:
    """
    Questa funzione cerca una molecola su PubChem usando il nome come sostanza e
    restituisce una lista di CID associati a quella sostanza.
    Se la ricerca fallisce o non viene trovato nulla, restituisce None.
    """
    sids = FindSidsBySubstanceName(name)
    # Le substance potrebbero essere composte da piu' compound, quindi
    # prendiamo tutti i SID trovati e cerchiamo i CID a loro associati.
    return FindCidsBySids(sids, name)

def FindCidsBySids(sids, name):
    """
    Questa funzione prende una lista di SIDs e restituisce una lista di CID

```

```

associati a quei SIDs.
"""
# Cerca i CID associati a ciascun SID.
cids_dict = pcp.get_cids(sids, namespace="sid", domain="substance")

# cids_dict e' un array di dizionari [ {...}, {...}, .. ]
# dove ogni entry e' un dizionario con attributi SID e CID,
# ovvero ha la forma { "SID": 12345, "CID": [ 67890, 67891 ] }
# Si puo' notare anche che CID e' una lista, perche' una substance puo'
essere associata a piu' CID.
cids = []
for entry in cids_dict:
    if "CID" in entry:
        cids.extend(entry["CID"])
if len(cids) == 0:
    logger.warning(f"Nessun CID trovato per '{name}' come substance")
    return None
# Deduplica i CID
cids = list(set(cids))
return cids

def FindMolByNameWithSynonyms(name: str, include_substances=True) -> Mol:
    """
    Questa funzione cerca una molecola su PubChem usando i sinonimi
    e restituisce il primo Mol trovato.
    Se la ricerca fallisce o non viene trovato nulla, restituisce None.
    """
    logger.debug(f"Cercando la molecola '{name}' su PubChem come sinonimo")
    synonyms = []
    try:
        synonyms = pcp.get_synonyms(name, namespace="name", domain="compound")
    except Exception as e:
        logger.error(f"Connessione fallita per '{name}' come sinonimo: {e}")
    if synonyms is None or len(synonyms) == 0:
        if include_substances:
            try:
                synonyms = pcp.get_synonyms(name, namespace="name",
domain="substance")
            except Exception as e:
                logger.error(
                    f"Connessione fallita per '{name}' come sinonimo di
sostanza: {e}"
                )
    if synonyms is None or len(synonyms) == 0:

```

```

        logger.warning(f"Nessun sinonimo trovato per '{name}'")

        # Synonyms e' una lista di dizionari, dove ogni dizionario ha la forma {
"CID": 12345, "Synonym": ["...", "..."] }
        # Appiattiamo la lista di sinonimi in un'unica lista di stringhe
        flat_synonyms = []
        for entry in synonyms:
            if "Synonym" in entry:
                flat_synonyms.extend(entry["Synonym"])

        # Prova a cercare la molecola usando i sinonimi trovati
        for synonym in flat_synonyms:
            mol = FindMolByName(synonym)
            # Se troviamo una molecola valida usando un sinonimo, restituiamo quella
molecola
            if mol is not None:
                SetMolName(mol, name)
                return mol

        logger.warning(f"Nessuna molecola trovata per '{name}' usando sinonimi")
        return None

class PubChemNamespace(Enum):
    CID = "cid"
    SID = "sid"

class PubChemDomain(Enum):
    COMPOUND = "compound"
    SUBSTANCE = "substance"

def GetBestMol(
    ids,
    name: str,
    namespace: PubChemNamespace.COMPOUND,
    domain: PubChemDomain.COMPOUND,
) -> Mol:
    """
    Questa funzione prende una lista di compound ID e restituisce la molecola
    con il maggior numero di atomi.
    Inoltre associa il nome alla molecola usando la proprietà `_Name`.
    Se il testo SDF è vuoto o non valido, restituisce None.
    """

```

```

if ids is None or len(ids) == 0:
    logger.warning(f"Nessun ID valido trovato per '{name}'")
    return None

# Otteniamo il testo SDF per gli ID dati
try:
    sdf_text = pcp.get_sdf(ids, namespace=namespace.value,
domain=domain.value)
except Exception as e:
    logger.error(f"Errore durante il recupero dello SDF per '{name}': {e}")
    return None

if sdf_text is None or sdf_text.strip() == "":
    logger.warning(f"SDF vuoto o non valido per '{name}'")
    return None

# Trasformare il testo SDF in un oggetto richiede una particolare
attenzione.
# Bisogna creare uno stream di byte a partire dal testo SDF, e poi passarlo
a ForwardSDMolSupplier.
sdf_stream = io.BytesIO(sdf_text.encode("utf-8"))
supplier = ForwardSDMolSupplier(sdf_stream)

# Se ci sono più molecole, prendiamo quella con il maggior numero di atomi
best_mol = GetBestMolFromSupplier(supplier)

# Assicuriamoci di associare name al risultato
if best_mol is not None:
    logger.success(
        f"Trovata molecola per '{name}' con {best_mol.GetNumAtoms()} atomi"
    )
    best_mol.SetProp("_Name", name)
return best_mol

def GetBestMolFromSupplier(supplier: ForwardSDMolSupplier) -> Mol:
    """
    Questa funzione prende un ForwardSDMolSupplier e restituisce la molecola con
    il maggior numero di atomi.
    """
    ATOM_COUNT_MAX = 100 # Imposta un limite massimo di atomi per evitare di
processare molecole troppo grandi
    ATOM_COUNT_MIN = 2 # Imposta un limite minimo di atomi per evitare di
considerare molecole troppo semplici o non valide

```

```

best_mol = None
max_atoms = 0
for mol in supplier:
    if mol is not None:
        num_atoms = mol.GetNumAtoms()
        if num_atoms > max_atoms and num_atoms <= ATOM_COUNT_MAX and
num_atoms >= ATOM_COUNT_MIN:
            max_atoms = num_atoms
            best_mol = mol
return best_mol

def GetMolName(mol):
    return mol.GetProp("_Name") if mol.HasProp("_Name") else "Unknown"

def SetMolName(mol, name):
    mol.SetProp("_Name", name)

def GetMolProperty(mol: Mol, property_name: str):
    if mol.HasProp(property_name):
        return mol.GetProp(property_name)
    else:
        return None

def SetMolProperty(mol: Mol, property_name: str, value):
    mol.SetProp(property_name, value)

def SaveMolsToSDF(mols, file_path):
    writer = SDWriter(file_path)
    for mol in mols:
        writer.write(mol)
    writer.flush()
    writer.close()
    logger.info(f"Salvate {len(mols)} molecule in {file_path}")

def PrintBiggestMol(mols):
    biggest_mol = None
    max_atoms = 0
    for mol in mols:
        if mol is not None:

```

```

        num_atoms = mol.GetNumAtoms()
        if num_atoms > max_atoms:
            max_atoms = num_atoms
            biggest_mol = mol
    if biggest_mol is not None:
        logger.info(
            f"La molecola più grande è '{GetMolName(biggest_mol)}' con
{max_atoms} atomi"
        )
    else:
        logger.warning("Nessuna molecola valida trovata")

class FingerprintStrategy(Enum):
    MORGAN = "morgan"
    RDKIT = "rdkit"

# Definiamo una funzione che ci trasforma le molecole di un SDF in un vettore
di fingerprint
# @input: SDMol
# @output: Vettore di fingerprint
def GetFingerprintFromSDMol(mols: SDMolSupplier, strategy: FingerprintStrategy
= FingerprintStrategy.MORGAN) -> list:
    if strategy == FingerprintStrategy.MORGAN:
        gen = rdFingerprintGenerator.GetMorganGenerator(radius=2, fpSize=2048)
        msg = "Generando fingerprint Morgan"
    elif strategy == FingerprintStrategy.RDKit:
        gen = rdFingerprintGenerator.GetRDKitFPGenerator()
        msg = "Generando fingerprint RDKit"
    fps = []
    for mol in tqdm(mols, msg):
        fp = gen.GetFingerprint(mol)
        fps.append(fp)
    return fps

def GetFingerprint(mol: Mol):
    """
    Restituisce la fingerprint della molecola passata come argomento in `mol`
    """
    gen = rdFingerprintGenerator.GetMorganGenerator(radius=2, fpSize=2048)
    return gen.GetFingerprint(mol)

```

```

def GetFingerprintsFromMols(mols: list[Mol]) -> list:
    """
    Restituisce la lista delle fingerprint associate alle molecole della lista
    `mols`
    """
    gen = rdFingerprintGenerator.GetMorganGenerator(radius=2, fpSize=2048)
    fps = []
    for mol in tqdm(mols, "Generando fingerprint"):
        fp = gen.GetFingerprint(mol)
        fps.append(fp)
    return fps

# Definiamo una funzione che generi la matrice di similarita' di tanimoto a
# partire da fingerprints
# input: fingerprints
# output: matrice di similarita' in formato adatto a Butina.ClusterData
def GetSimilarityMatrixFromFingerprints(fingerprints):
    sim_mat = []
    n_fps = len(fingerprints)

    for i in tqdm(range(1, n_fps), "Calcolando similarita'"):
        # Per ogni fingerprint, calcolo la sua similarita' con tutte le altre
        # fingerprint
        logger.debug("Calcolando similarita' per fingerprint {}".format(i))
        current_fp = fingerprints[i]
        similarita_con_altre = DataStructs.BulkTanimotoSimilarity(
            current_fp, fingerprints[:i]
        )
        sim_mat.extend(similarita_con_altre)

    return sim_mat

# Definiamo una funzione che generi la matrice delle distanze a partire da
# fingerprints
# input: vettore di fingerprint
# output: matrice di distanze in formato adatto a Butina.ClusterData
def GetDistanceMatrixFromFingerprints(fingerprints):
    dist_mat = []
    sim_mat = GetSimilarityMatrixFromFingerprints(fingerprints)

    for sim_value in sim_mat:
        # Per ogni valore della matrice di similarita'
        # Trasformo il valore di similarita' in un valore di distanza con la

```

```

seguente formula
    dist_value = 1.0 - sim_value
    dist_mat.append(dist_value)

    return dist_mat

# Definiamo una funzione che genera i cluster con Butina a partire da una
matrice delle distanze
# input:
# - distance_matrix: matrice delle distanze in formato adatto a
Butina.ClusterData
# - n: Numero di righe e colonne della matrice
# output: clusters
def GetButinaClustersFromDistanceMatrix(distance_matrix, n, dist_tresh):
    return Butina.ClusterData(
        distance_matrix, n, distThresh=dist_tresh, isDistData=True
    )

# Questa funzione espande una distance matrix da un formato Butina ad un
formato pieno NxN,
# adatto ai metodi che lavorano su matrici di distanza complete.
# input:
# - compressed_distance_matrix: distance matrix adatto a Butina.ClusterData
# - n: Numero di righe e colonne della matrice
# output: matrice completa
def ExpandDistanceMatrix(compressed_distance_matrix, n):
    expanded_matrix = []

    for i in range(n):
        new_row = []
        for j in range(n):
            if i == j:
                new_row.append(0.0)
            elif i < j:
                # Butina distance matrix is compressed, so we need to calculate
the index in the compressed format
                index = int((j * (j - 1)) / 2 + i)
                new_row.append(compressed_distance_matrix[index])
            else:
                # The distance matrix is symmetric, so we can use the value from
the upper triangle
                index = int((i * (i - 1)) / 2 + j)
                new_row.append(compressed_distance_matrix[index])

```

```

        expanded_matrix.append(new_row)

    return expanded_matrix

def CompressDistanceMatrix(expanded_distance_matrix):
    """
    Converte una matrice delle distanze NxN nella forma compressa richiesta da
    Butina.
    """
    compressed_matrix = []
    n = len(expanded_distance_matrix)

    for j in range(1, n):
        for i in range(j):
            compressed_matrix.append(expanded_distance_matrix[i][j])

    return compressed_matrix

class MappingStrategy(Enum):
    """
    Strategie di riduzione della dimensionalità per generare coordinate 2D a
    partire da una matrice di distanze.

    - SAMMON: utilizza il Sammon mapping, che è una tecnica di riduzione
    della dimensionalità non lineare che cerca di preservare le piccole distanze.

    - MDS: utilizza il Multiple Dimensional Scaling, che è la tecnica
    standard di riduzione della dimensionalità lineare.

    - UMAP: utilizza il Uniform Manifold Approximation and Projection, che è
    un'ottima tecnica per la visualizzazione di cluster.
    """
    SAMMON = "sammon"
    MDS = "mds"
    UMAP = "umap"

class ClusteringStrategy(Enum):
    """
    Strategie di clustering per generare cluster a partire da coordinate 2D o da
    una matrice di distanze.

    - KMEANS: utilizza il clustering KMeans, che è la tecnica standard di
    clustering.

    - BUTINA: utilizza il clustering Butina, che è una tecnica di clustering
    basata sulla distanza.

    - AGGLOMERATIVE: utilizza il clustering agglomerativo, che è una tecnica

```

```

di clustering gerarchico.
"""
KMEANS = "kmeans"
BUTINA = "butina"
AGGLOMERATIVE = "agglomerative"

def GetClustersFromDistanceMatrix(
    mols: list[Mol],
    distance_matrix,
    cluster_count,
    random_state,
    mapping_strategy: MappingStrategy = MappingStrategy.SAMMON,
    cluster_strategy: ClusteringStrategy = ClusteringStrategy.KMEANS,
) -> list[ClusteredMol]:
    """
    Genera coordinate 2D e cluster a partire da una matrice delle distanze.

    input:
        - mols: lista di molecole su cui vogliamo generare i cluster
        - distance_matrix: matrice delle distanze completa NxN
        - cluster_count: numero di cluster da generare

    output:
        - Lista di ClusteredMol, che contiene le molecole clusterizzate e le loro
coordinate 2D
    """
    clustered_mols = []
    coords_2d = []

    # Creiamo un array numpy dalla matrice di distanze
    np_distance_matrix = np.array(distance_matrix)

    if mapping_strategy == MappingStrategy.SAMMON:
        logger.info(
            "Eseguendo Sammon mapping per ridurre la matrice di distanze a 2
dimensioni"
        )
        np.random.seed(random_state)
        sammon_result = sammon(np_distance_matrix, inputdist="distance",
init="")
        coords_2d = sammon_result[0]
    elif mapping_strategy == MappingStrategy.MDS:
        logger.info("Eseguendo MDS per ridurre la matrice di distanze a 2
dimensioni")

```

```

mds = MDS(
    n_components=2,
    metric="precomputed",
    random_state=random_state,
    normalized_stress="auto",
    n_init=4,
    init="random",
)
coords_2d = mds.fit_transform(np_distance_matrix)
elif mapping_strategy == MappingStrategy.UMAP:
    logger.info("Eseguendo UMAP per ridurre la matrice di distanze a 2
dimensioni")
    reducer = umap.UMAP(
        n_components=2, metric="precomputed", random_state=random_state
    )
    coords_2d = reducer.fit_transform(np_distance_matrix)
else:
    raise ValueError(
        f"Strategia di riduzione della dimensionalità '{mapping_strategy}'
non supportata"
    )

clusters = []
if cluster_strategy == ClusteringStrategy.KMEANS:
    logger.info("Eseguendo KMeans per generare {}
cluster".format(cluster_count))
    km = KMeans(cluster_count, random_state=random_state)
    # KMeans fit predict restituisce un vettore di cluster a cui ogni
molecola appartiene,
    # ad esempio [0, 0, 1, 1, 2] significa che le prime due molecole
appartengono al cluster 0,
    # le successive due al cluster 1 e l'ultima al cluster 2
    clusters = km.fit_predict(coords_2d)
elif cluster_strategy == ClusteringStrategy.BUTINA:
    logger.info("Eseguendo Butina per generare {}
cluster".format(cluster_count))
    logger.warning(
        "Questa strategia ignora il numero di cluster specificato e utilizza
una threshold di distanza per generare i cluster"
    )
    # Butina utilizza una threshold di distanza per generare i cluster,
quindi non
    # possiamo specificare direttamente il numero di cluster che vogliamo,
    # ma possiamo giocare con la threshold per cercare di ottenere un numero
di cluster simile a quello desiderato.

```

```

dist_tresh = 0.85
compressed_distance_matrix = CompressDistanceMatrix(distance_matrix)
clusters = Butina.ClusterData(
    compressed_distance_matrix, len(mols), dist_tresh, isDistData=True
)
logger.info(
    f"Butina ha generato {len(clusters)} cluster con threshold
{dist_tresh}"
)
# Butina restituisce una tupla di tuple, dove ogni tupla interna
contiene gli indici delle molecole che appartengono a quel cluster.
# Convertiamo la tupla di tuple in un vettore di cluster a cui ogni
molecola appartiene
cluster_vector = [0] * len(mols)
for cluster_id, cluster in enumerate(clusters):
    for mol_index in cluster:
        cluster_vector[mol_index] = cluster_id
clusters = cluster_vector
elif cluster_strategy == ClusteringStrategy.AGGLOMERATIVE:
    logger.info(
        "Eseguendo Agglomerative Clustering per generare {} cluster".format(
            cluster_count
        )
    )
    agg = AgglomerativeClustering(
        n_clusters=cluster_count, metric="precomputed", linkage="average"
    )
    clusters = agg.fit_predict(np_distance_matrix)
else:
    raise ValueError(f"Strategia di clustering '{cluster_strategy}' non
supportata")

# Ora abbiamo un vettore di cluster e un vettore di coordinate 2D per ogni
molecola,
# vogliamo combinarli in un unico contenitore che ci permetta di accedere
facilmente a tutte queste informazioni.
for i, (mol, cluster_id) in enumerate(zip(mols, clusters)):
    x, y = coords_2d[i]
    clustered_mols.append(
        ClusteredMol(mol=mol, cluster_id=int(cluster_id), x=float(x),
y=float(y))
    )

# Verifichiamo che clustered_mols e mols siano compatibili
assert len(clustered_mols) == len(

```

```

        mols
    ), "Il numero di molecole clusterizzate non corrisponde al numero di
molecole originali"
    for i in range(len(mols)):
        assert (
            clustered_mols[i].mol == mols[i]
        ), f"La molecola clusterizzata alla posizione {i} non corrisponde alla
molecola originale"

    return clustered_mols

class ClusteredMol:
    """Classe che rappresenta una molecola clusterizzata, con informazioni utili
per l'analisi e la visualizzazione"""

    @property
    def mol(self) -> Mol:
        """mol e' la molecola originale"""
        return self._mol

    @property
    def cluster_id(self) -> int:
        """cluster_id e' l'id del cluster a cui appartiene questa molecola"""
        return self._cluster_id

    @property
    def x(self) -> float:
        """x e' la coordinata x della molecola nello spazio 2D dell'embedding"""
        return self._x

    @property
    def y(self) -> float:
        """y e' la coordinata y della molecola nello spazio 2D dell'embedding"""
        return self._y

    def __init__(self, mol, cluster_id, x, y):
        self._mol = mol
        self._cluster_id = cluster_id
        self._x = x
        self._y = y

    def GetName(self):
        return GetMolName(self._mol)

```

```

class ClusterMCS:
    """Classe che rappresenta il risultato dell'MCS di un cluster, con
    informazioni utili per l'analisi e la visualizzazione"""

    @property
    def cluster_id(self) -> int:
        """cluster_id e' l'id del cluster a cui si riferisce questo MCS"""
        return self._cluster_id

    @property
    def mcs_smarts(self) -> str:
        """mcs_smarts e' la rappresentazione SMARTS della sottostruttura comune,
        che puo' essere usata per cercare questa sottostruttura in altre molecole"""
        return self._mcs_smarts

    @property
    def mcs_mol(self) -> Mol:
        """mcs_mol e' la rappresentazione Mol della sottostruttura comune"""
        return self._mcs_mol

    @property
    def mols_in_cluster(self) -> list[ClusteredMol]:
        """mols_in_cluster e' la lista di molecole che appartengono al cluster a
        cui si riferisce questo MCS"""
        return self._mols_in_cluster

    def __init__(self, cluster_id, mcs_smarts, mcs_mol, mols_in_cluster):
        self._cluster_id = cluster_id
        self._mcs_smarts = mcs_smarts
        self._mcs_mol = mcs_mol
        self._mols_in_cluster = mols_in_cluster

    def GetSize(self):
        """Restituisce il numero di molecole che appartengono al cluster a cui
        si riferisce questo MCS"""
        return len(self._mols_in_cluster)

    def RenderColoredMolImage(
        mol: Mol, color, size: tuple[int, int] = (220, 220)
    ) -> np.ndarray:
        """
        Disegna la molecola come immagine RGBA con sfondo trasparente.
        Il colore viene applicato alla molecola stessa, non al frame.

```

```

"""
mol_to_draw = Draw.PrepareMolForDrawing(mol)
drawer = rdMolDraw2D.MolDraw2DCairo(size[0], size[1])
options = drawer.drawOptions()

options.clearBackground = False
options.useBWAtomPalette()
options.setSymbolColour(tuple(color[:3]))
options.setLegendColour(tuple(color[:3]))
options.fillHighlights = False
options.continuousHighlight = False

atom_ids = list(range(mol_to_draw.GetNumAtoms()))
bond_ids = list(range(mol_to_draw.GetNumBonds()))
atom_colors = {atom_id: tuple(color[:3]) for atom_id in atom_ids}
bond_colors = {bond_id: tuple(color[:3]) for bond_id in bond_ids}
atom_radii = {atom_id: 0.01 for atom_id in atom_ids}
drawer.DrawMolecule(
    mol_to_draw,
    atom_ids,
    bond_ids,
    atom_colors,
    bond_colors,
    atom_radii,
    -1,
    "",
)
drawer.FinishDrawing()

png_bytes = drawer.GetDrawingText()
return np.asarray(Image.open(io.BytesIO(png_bytes)).convert("RGBA"))

def GetClusterCount(clustered_mols: list[ClusteredMol]) -> int:
    """Funzione che calcola il numero di cluster presenti in una lista di
ClusteredMol"""
    cluster_ids = set([cm.cluster_id for cm in clustered_mols])
    return len(cluster_ids)

def GetClustersMCS(
    clustered_mols: list[ClusteredMol], mols: list[Mol]
) -> list[ClusterMCS]:
    """
    Funzione che genera MCSs a partire dai clusters

```

```

input:
    - clustered_mols: una lista di ClusteredMol, che contiene le molecole
clusterizzate e le loro coordinate 2D
    - mols: contenitore di molecole su cui vogliamo calcolare gli MCS, che puo'
essere la lista originale di molecole o
    la lista dei murcko scaffold generici, a seconda di cosa vogliamo
analizzare

output:
    - Lista di ClusterMCS
"""
cluster_results = []
cluster_count = GetClusterCount(clustered_mols)
for cluster_id in tqdm(
    range(cluster_count),
    total=cluster_count,
    desc="Calcolando MCS per ogni cluster",
):
    # Otteniamo gli indici delle molecole che appartengono al cluster i,
    filtrando la lista di ClusteredMol per cluster_id
    mol_indices_in_cluster = []
    clustered_mols_in_cluster = []
    for i, clustered_mol in enumerate(clustered_mols):
        if clustered_mol.cluster_id == cluster_id:
            mol_indices_in_cluster.append(i)
            clustered_mols_in_cluster.append(clustered_mol)

    if len(mol_indices_in_cluster) < 2:
        # salta i cluster con una sola molecola
        continue

    # trova la MCS per questo specifico cluster
    logger.debug(
        "Calcolando MCS per cluster {} con {} molecole".format(
            cluster_id, len(mol_indices_in_cluster)
        )
    )

    # otteniamo le molecole originali usando gli indici ottenuti e la lista
    di molecole passata come argomento
    mols_in_cluster = [mols[i] for i in mol_indices_in_cluster]
    mcs_res = rdFMCS.FindMCS(mols_in_cluster)

    # converti il risultato MCS in una molecola visualizzabile

```

```

mcs_mol = MolFromSmarts(mcs_res.smartsString)

# La mol ottenuto da MolFromSmarts non ha determinati valori che sono
richiesti per il calcolo delle fingerprints
# quindi, invochiamo `UpdatePropertyCache` e `FastFindRings` per forzare
l'aggiornamento di questi valori.
mcs_mol.UpdatePropertyCache(strict=False)
FastFindRings(mcs_mol)

# Nel ClusterMCS, salviamo anche la lista di ClusteredMol che
appartengono a quel cluster,
# in modo da poter accedere facilmente a tutte le informazioni su quelle
molecole quando analizziamo o visualizziamo l'MCS
cluster_results.append(
    ClusterMCS(
        cluster_id=cluster_id,
        mcs_smarts=mcs_res.smartsString,
        mcs_mol=mcs_mol,
        mols_in_cluster=clustered_mols_in_cluster,
    )
)
return cluster_results

def DrawClusterMCS(cluster_mcs: ClusterMCS, out_dir: str = "out/mcs"):
    """
    Funzione che disegna l'MCS di un cluster e lo salva in out/mcs come file
    PNG.
    Il nome del file e' cluster_{id}_mcs.png, dove {id} e' l'id del cluster.
    input: cluster_mcs - un oggetto ClusterMCS, ovvero il risultato della
    funzione GetClustersMCS per un singolo cluster
    """
    # Se la cartella out/mcs non esiste, creala
    if not os.path.exists(out_dir):
        os.makedirs(out_dir)

    mcs = cluster_mcs.mcs_mol
    id = cluster_mcs.cluster_id
    cluster_size = cluster_mcs.GetSize()
    legend = f"MCS Cluster {id} (size {cluster_size})"
    for clustered_mol in cluster_mcs.mols_in_cluster:
        legend += f"\n- {clustered_mol.GetName()}"

    DrawMol(mcs, out_dir, name_prefix=f"cluster_{id}_mcs", legend=legend)

```

```

def PlotClustersMCS(clusters_mcs: list[ClusterMCS], out_dir: str = "out/mcs"):
    """
    Funzione che disegna un grafico di tutte le molecole che appartengono a quel
    cluster,
    usando le coordinate 2D dell'embedding e colorando le molecole in base al
    cluster.
    Il grafico viene salvato in `out_dir` come file PNG con nome
    clusters_mcs.png
    """

    # Se la cartella out/mcs non esiste, creala
    if not os.path.exists(out_dir):
        os.makedirs(out_dir)

    fig, ax = plt.subplots(figsize=(10, 10))
    cmap = plt.get_cmap("tab20")
    xs = []
    ys = []

    for cluster_mcs in clusters_mcs:
        for clustered_mol in cluster_mcs.mols_in_cluster:
            color = cmap(clustered_mol.cluster_id % cmap.N)

            mol_img = RenderColoredMolImage(clustered_mol.mol, color, size=(180,
180))

            image_box = OffsetImage(mol_img, zoom=0.25)
            ab = AnnotationBbox(
                image_box,
                (clustered_mol.x, clustered_mol.y),
                frameon=False,
                pad=0.0,
                zorder=2,
            )
            ax.add_artist(ab)

            xs.append(clustered_mol.x)
            ys.append(clustered_mol.y)

    ax.text(
        clustered_mol.x,
        clustered_mol.y,
        clustered_mol.GetName(),
        fontsize=8,
        ha="center",

```

```

        va="bottom",
        color=color,
        zorder=3,
    )

x_margin = max((max(xs) - min(xs)) * 0.1, 0.25) if xs else 1.0
y_margin = max((max(ys) - min(ys)) * 0.1, 0.25) if ys else 1.0

if xs and ys:
    ax.set_xlim(min(xs) - x_margin, max(xs) + x_margin)
    ax.set_ylim(min(ys) - y_margin, max(ys) + y_margin)

ax.set_title("Molecole nei cluster MCS")
ax.grid(True, alpha=0.3)
plt.savefig(f"{out_dir}/clusters_mcs.png", dpi=300, bbox_inches="tight")
plt.close()

def DrawClustersMCS(clusters_mcs: list[ClusterMCS], out_dir: str = "out/mcs"):
    """
    Funzione che disegna gli MCS di ogni cluster e li salva nella cartella
    specificata come file PNG.
    Il nome del file e' cluster_{id}_mcs.png, dove {id} e' l'id del cluster.

    Inoltre, questa funzione disegna un grafico di tutte le molecole che
    appartengono a quel cluster,
    usando le coordinate 2D dell'embedding e colorando le molecole in base al
    cluster.

    - input: `clusters_mcs` - risultato della funzione GetClustersMCS
    - input: `out_dir` - percorso della cartella in cui salvare i file PNG
    """
    # Cancella la cartella out/mcs se esiste per eliminare i vecchi risultati
    if os.path.exists(out_dir):
        for file in os.listdir(out_dir):
            os.remove(os.path.join(out_dir, file))

    for cluster_mcs in clusters_mcs:
        DrawClusterMCS(cluster_mcs, out_dir=out_dir)

    PlotClustersMCS(clusters_mcs, out_dir=out_dir)

def DebugMol(mol: Mol):
    """

```

```

    Funzione di debug che stampa informazioni utili sulla molecola, come il
    nome, il numero di atomi e di legami.
    """
    name = GetMolName(mol)
    num_atoms = mol.GetNumAtoms()
    num_bonds = mol.GetNumBonds()
    logger.debug(f"Molecola '{name}': {num_atoms} atomi, {num_bonds} legami")

def DrawMols(mols, out_dir: str = "out/mols"):
    """
    Disegna le molecole e salva il risultato in file PNG nella cartella
    `out_dir`
    """
    for mol in tqdm(mols, desc="Disegnando le molecole"):
        DebugMol(mol)
        DrawMol(mol, out_dir=out_dir)

def DrawMol(
    mol, out_dir: str = "out/mols", name_prefix: str = "mol", legend: str = None
):
    name = GetMolName(mol)

    if mol is None or mol.GetNumAtoms() == 0:
        logger.warning(f"Molecola '{name}' nulla o senza atomi, salto il
disegno")
        return

    def _draw(mol_to_draw):
        return Draw.MolToImage(
            mol_to_draw,
            size=(600, 600),
            legend=legend if legend else name,
        )

    try:
        img = _draw(mol)
    except Exception as e:
        logger.warning(
            f"Prima conversione immagine fallita per '{name}': {e}, provo a
calcolare coordinate 2D e riprovo"
        )
    try:

```

```

        rdDepictor.Compute2DCoords(mol)
        img = _draw(mol)
    except Exception as e2:
        logger.error(f"Errore durante il disegno della molecola '{name}'
dopo Compute2DCoords: {e2}")
        return

    # Se la cartella out non esiste, creala
    if not os.path.exists(out_dir):
        os.makedirs(out_dir)

    # Name potrebbe avere il carattere di separatore di cartella al suo interno
    `/'`
    # Quindi lo dobbiamo safinicare se vogliamo usarlo come nome di file
    safe_name = name.replace("/", "_")
    # Salva l'immagine in out/mol_{name}.png
    img.save(f"{out_dir}/{name_prefix}_{safe_name}.png")

# Funzione che ottiene i murcko scaffold di ogni molecola
# input: mols - contenitore di molecole di cui vogliamo ottenere i murcko
scaffold
# output: lista di murcko scaffold (sempre formato Mol)
def GetScaffoldsFromSDMol(mols):
    return [MurckoScaffold.GetScaffoldForMol(m) for m in mols]

# Funzione che rende i murcko scaffold generici, ignorando anche i tipi di
atomi e di legami
# input: scaffolds - lista di murcko scaffold (sempre formato Mol)
# output: lista di murcko scaffold generici (sempre formato Mol)
def MakeScaffoldsGeneric(scaffolds):
    return [MurckoScaffold.MakeScaffoldGeneric(s) for s in scaffolds]

def GetClustersMCSFromMols(
    mols: list[Mol],
    use_scaffolds: bool = True,
    cluster_count=20,
    random_state=42,
    mapping_strategy: MappingStrategy = MappingStrategy.SAMMON,
    cluster_strategy: ClusteringStrategy = ClusteringStrategy.KMEANS,
    fingerprint_strategy: FingerprintStrategy = FingerprintStrategy.MORGAN,
) -> list[ClusterMCS]:
    """Funzione che ottiene i cluster e i relativi MCS a partire da una lista di

```

molecole

- input: mols - lista di molecole (formato Mol)
- output: lista di ClusterMCS, ovvero una lista di oggetti che rappresentano i cluster e i relativi MCS, con informazioni utili per l'analisi e la visualizzazione

La funzione esegue i seguenti passaggi:

1. Ottiene i fingerprint per ogni molecola
2. Ottiene la matrice di distanze a partire dai fingerprint
3. Utilizza la strategia di clustering selezionata per ottenere i cluster a partire dalla matrice di distanze
4. Per ogni cluster, ottiene l'MCS a partire dalle molecole che appartengono a quel cluster, e salva le informazioni in un oggetto ClusterMCS

"""

Otteniamo le fingerprint per ogni molecola

fingerprints = GetFingerprintFromSDMol(mols, fingerprint_strategy)

Otteniamo la matrice di distanze

compressed_distance_matrix = GetDistanceMatrixFromFingerprints(fingerprints)

expended_distance_matrix = ExpandDistanceMatrix(
 compressed_distance_matrix, len(fingerprints)
)

clustered_mols = GetClustersFromDistanceMatrix(
 mols,
 expended_distance_matrix,
 cluster_count,
 random_state=random_state,
 mapping_strategy=mapping_strategy,
 cluster_strategy=cluster_strategy,
)

if use_scaffolds:

L'MCS trova la più grande sottostruttura comune tra due o più molecole.

Cercare l'MCS su molecole intere potrebbe dare un risultato troppo piccolo o "sporcato" dalle catene laterali.

Usare Bemis-Murcko prima dell'MCS permette di trovare il nucleo comune degli scheletri,

ignorando le variazioni dei sostituenti.

scaffolds = GetScaffoldsFromSDMol(mols)

Se invece vogliamo concentrarci solo sulla topologia dello scheletro,

```

ignorando
    # anche i tipi di atomi e di legami, possiamo usare MakeScaffoldGeneric
    generic_scaffolds = MakeScaffoldsGeneric(scaffolds)

    # otteniamo gli MCS di ogni cluster
    return GetClustersMCS(clustered_mols, scaffolds)
else:
    return GetClustersMCS(clustered_mols, mols)

```

mcs_search.py

```

"""
In questo modulo, eseguiamo due task. A partire dai interessanti MCS dei
cluster di molecole:
1. Troviamo quali compound in `compunds.sdf` contengono quel MCS.
2. Misuriamo quanto ogni molecola del dataset e' simile a quel MCS, usando
Tanimoto similarity tra la fingerprint del MCS e quella di ogni molecola.
"""

import csv
from loguru import logger
from helper import *

def FindClusterMCSByClusterId(cluster_mcscs: list[ClusterMCS], cluster_id: int)
-> ClusterMCS:
    """Trova l'MCS del cluster con id `cluster_id` nella lista
    `cluster_mcscs`."""
    for cluster_mcs in cluster_mcscs:
        if cluster_mcs.cluster_id == cluster_id:
            return cluster_mcs
    return None

def GetInterestingClusterMCS(cluster_mcscs: list[ClusterMCS],
interesting_clusters) -> list[ClusterMCS]:
    """Restituisce gli MCS dei cluster di molecole che sono stati identificati
    come interessanti."""
    interesting_cluster_mcscs = []
    for id in interesting_clusters:

```

```

        cluster_mcs = FindClusterMCSByClusterId(cluster_mcscs, id)
        if cluster_mcs is not None:
            interesting_cluster_mcscs.append(cluster_mcs)
        else:
            logger.warning(f"Non ho trovato l'MCS del cluster con id {id}")
    return interesting_cluster_mcscs

def FindSubstructInMols(substruct: Mol, mols: list[Mol]) -> list[Mol]:
    """Trova quali compound in `mols` contengono (o includono) la sottostruttura
    rappresentata da `substruct`.
    Restituisce la lista di molecole che contengono quella sottostruttura."""
    return [mol for mol in mols if mol.HasSubstructMatch(substruct)]

def GetSimilarityWithMols(mol: Mol, other_mols: list[Mol]) -> list[float]:
    """
    Misura quanto ogni molecola della lista `other_mols` e' simile alla molecola
    rappresentata da `mol`,
    usando Tanimoto similarity tra la fingerprint del MCS e quella di ogni
    molecola.
    In output avremo una lista di numeri in virgola mobile (floating point
    numbers o float)
    e.g. [0.2, 0.02]
    """
    mol_fingerprint = GetFingerprint(mol)
    other_fingerprints = GetFingerprintsFromMols(other_mols)

    return DataStructs.BulkTanimotoSimilarity(
        mol_fingerprint, other_fingerprints
    )

def ProcessSimilarityWithMols(cluster_mcs: ClusterMCS, other_mols: list[Mol],
out_prefix: str = "other_mols"):
    """
    Questa funzione analizza la similarita' tra l'MCS del cluster in
    `cluster_mcs` con tutte le altre
    molecole nella lista `other_mols`.
    Il risultato viene salvato in un file CSV nella cartella "out" il cui nome
    e' `cluster_<id>_<out_prefix>.csv`
    """
    # Analizziamo la similarita' di un MCS di un cluster con le altre molecole
    sims: list[float] = GetSimilarityWithMols(cluster_mcs.mcs_mol, other_mols)

```

```

# Salva la similarita' appena calcolata in un file csv
cluster_id = cluster_mcs.cluster_id
csv_file_path = f"out/cluster_{cluster_id}_vs_{out_prefix}.csv"
with open(csv_file_path, "w") as csv_file:
    csv_writer = csv.writer(csv_file)
    csv_writer.writerow(
        [
            "drug_name",
            "indication",
            f"cluster_{cluster_id}_similarity"
        ]
    )

    for i, other_mol in enumerate(other_mols):
        mol_name = GetMolName(other_mol)
        mol_indication = GetMolProperty(other_mol, "TTD_INDICATION")

        csv_writer.writerow(
            [
                mol_name,
                mol_indication,
                sims[i]
            ]
        )

logger.info(f"Similarita' scritte nel file {csv_file_path}")

```

kovarian_clusters.py

```

# Obiettivo: ottenere la mcs delle molecole dei farmaci del k ovarico
# la mcs deve essere fatta sui cluster dei vari farmaci per ottenere la
migliore
# struttura possibile con un significato chimico e riconoscibile.
# utilizziamo murcko scaffold per ottenere scaffold piu ampi e significativi

from loguru import logger
from rdkit import Chem
from helper import *
import mcs_search

CLUSTER_COUNT = 32
RANDOM_STATE = 10

```

```

INTERESTING_CLUSTER_IDS = [18, 21]

def LoadKOvarianMolecules() -> list[Chem.Mol]:
    """
    Carica le molecole dei farmaci del k ovarico da un file SDF, o se il file
    non esiste,
    li scarica da PubChem usando i nomi dei farmaci da un file CSV.
    Restituisce una lista di oggetti Chem.Mol.
    """
    kovarian_mols = LoadMolecules(
        file_path="data/compounds.sdf",
        fallback_path="data/ttd_drug_disease_ovarian_by_drug.csv",
        # Includiamo le sostanze per aumentare le possibilità di trovare i
farmaci,
        # anche se questo aumenta considerevolmente il tempo di ricerca
        include_substances=True,
    )
    logger.info("Caricate {} molecole".format(len(kovarian_mols)))

    # Filtra le molecole per rimuovere quelle che non sono state caricate
correttamente
    kovarian_mols = GetListFromSDMolSupplier(kovarian_mols)
    logger.info("Dopo il filtraggio, rimangono {}
molecole".format(len(kovarian_mols)))

    return kovarian_mols

def GetClustersMCSFromKOvarianMols(kovarian_mols: list[Chem.Mol]) ->
list[ClusterMCS]:
    """
    Ottiene gli MCS dei cluster a partire dalle molecole del k ovarico.
    Restituisce una lista di ClusterMCS, che contengono l'MCS di ogni cluster e
altre informazioni utili.
    """
    return GetClustersMCSFromMols(
        kovarian_mols,
        use_scaffolds=True,
        cluster_count=CLUSTER_COUNT,
        random_state=RANDOM_STATE,
        mapping_strategy=MappingStrategy.MDS,
        cluster_strategy=ClusteringStrategy.KMEANS,
        fingerprint_strategy=FingerprintStrategy.MORGAN,
    )

```

```

def GetInterestingClusters() -> list[ClusterMCS]:
    """
    Restituisce la lista di ClusterMCS di piccole molecole associate al k
    ovarico
    che sono stati identificati come interessanti in base a criteri arbitrari.
    """
    kovarian_mols = LoadKOvarianMolecules()

    # Ottieni gli MCS dei cluster a partire dalle molecole filtrate
    k_ovarian_cluster_mcscs: list[ClusterMCS] =
GetClustersMCSFromKOvarianMols(kovarian_mols)

    # Seleziona i cluster piu' interessanti
    return mcs_search.GetInterestingClusterMCS(
        k_ovarian_cluster_mcscs, INTERESTING_CLUSTER_IDS
    )

def main():
    InitLogger()

    # Idea:
    # SDF -> Fingerprint -> Matrice di distanze -> Cluster -> MCS per cluster ->
    Draw

    # trovare l'mcs delle molecole in compounds.sdf
    # Questo file e' una collezione di molecole che sono state testate contro il
    k ovarico,
    # e che hanno mostrato una certa efficacia. L'obiettivo e' trovare la
    struttura comune a queste molecole,
    # che potrebbe essere la chiave per capire come funzionano e per progettare
    nuovi farmaci.
    # Se il file non esiste, viene creato scaricando le molecole dal database
    PubChem usando il nome dei farmaci
    # nel file "data/ttd_drug_disease_ovarian_by_drug.csv"
    kovarian_mols = LoadKOvarianMolecules()

    DrawMols(kovarian_mols)
    PrintBiggestMol(kovarian_mols)

    k_ovarian_cluster_mcscs: list[ClusterMCS] =
GetClustersMCSFromKOvarianMols(kovarian_mols)

```

```

DrawClustersMCS(k_ovarian_cluster_mcscs)

interesting_cluster_mcscs = mcs_search.GetInterestingClusterMCS(
    k_ovarian_cluster_mcscs, INTERESTING_CLUSTER_IDS
)

for cluster_mcs in interesting_cluster_mcscs:
    # Cerco la "molecola che rappresenta l'MCS del cluster" nelle molecole
    del dataset
    mols_with_mcs = mcs_search.FindSubstructInMols(
        cluster_mcs.mcs_mol, kovarian_mols
    )
    logger.info(
        f"Trovate {len(mols_with_mcs)} molecole che contengono l'MCS del
cluster {cluster_mcs.cluster_id}"
    )

    mcs_search.ProcessSimilarityWithMols(
        cluster_mcs, kovarian_mols, "kovarian_mols"
    )

# La particolarita' di `if __name__ == "__main__":` e' che permette di eseguire
il codice solo
# quando il file viene eseguito direttamente, e non quando viene importato come
modulo in un altro file.
# In questo modo, possiamo definire la funzione
`FilterApprovedNonKOvarianDrugs` in questo file, e poi
# importarla e usarla in altri file senza eseguire il codice di filtraggio ogni
volta che importiamo questo modulo.
if __name__ == "__main__":
    main()

```

filtra_ttd.py

```

# Obiettivo: filtrare il file "data/ttd_drug_disease_by_drug.csv" per rimuovere
tutte
# le molecole che non sono state approvate o che appartengono alla classe k
ovarico,
# e salvare il risultato in un nuovo file
"data/ttd_approved_non_k_ovarian_drugs.csv"

# Steps:

```

```

# 1. Leggere il file "data/ttd_drug_disease_by_drug.csv" usando pandas
# 2. Filtrare le righe per mantenere solo quelle in cui:
#     - la colonna "approval categories" contiene "approved". E' importante
notare che il
#     valore potrebbe essere una stringa che contiene più categorie di
approvazione
#     (e.g "approved; phase 3"), quindi dobbiamo verificare se "approved" è
presente in quella stringa.
#     - e la colonna "icd11_codes" non è relativa al k ovarico, ovvero non
contiene "2C73"
# 3. Salvare il risultato in un nuovo file
"data/ttd_approved_non_k_ovarian_drugs.csv"

ALL_DRUGS_CSV_FILE_PATH = "data/ttd_drug_disease_by_drug.csv"
APPROVED_NON_K_OVARIAN_CSV_FILE_PATH =
"data/ttd_approved_non_k_ovarian_drugs.csv"

APPROVAL_CATEGORIES_COLUMN_NAME = "approval_categories"
ICD11_CODES_COLUMN_NAME = "icd11_codes"
APPROVED_KEYWORD = "approved"
K_OVARIAN_ICD11_CODE = "2C73"

import pandas as pd

def FilterApprovedNonKOvarianDrugs(input_csv_file_path: str,
output_csv_file_path: str):
    # Leggi il file CSV
    df = pd.read_csv(input_csv_file_path)

    # Filtra le righe
    filtered_df = df[
        # Prendi i farmaci approvati
        df[APPROVAL_CATEGORIES_COLUMN_NAME].str.contains(APPROVED_KEYWORD,
case=False, na=False) &

        # Ma escludi quelli che sono relativi al k ovarico
        ~df[ICD11_CODES_COLUMN_NAME].str.contains(K_OVARIAN_ICD11_CODE,
case=False, na=False)
    ]

    # Salva il risultato in un nuovo file CSV
    filtered_df.to_csv(output_csv_file_path, index=False)

```

```

# La particolarita' di `if __name__ == "__main__":` e' che permette di eseguire
il codice solo
# quando il file viene eseguito direttamente, e non quando viene importato come
modulo in un altro file.
# In questo modo, possiamo definire la funzione
`FilterApprovedNonKOvarianDrugs` in questo file, e poi
# importarla e usarla in altri file senza eseguire il codice di filtraggio ogni
volta che importiamo questo modulo.
if __name__ == "__main__":
    FilterApprovedNonKOvarianDrugs(ALL_DRUGS_CSV_FILE_PATH,
    APPROVED_NON_K_OVARIAN_CSV_FILE_PATH)

```

kovarian_clusters_vs_other_mols.py

```

#obiettivo: confrontare gli scaffold dei cluster con le molecole del dataset
iniziale (con farmaci solo approvati e senza la classe k ovarico),
#per vedere se ci sono molecole che contengono queglii scaffold e se sono simili
a quelli scaffold

import os
import kovarian_clusters
from filtra_ttd import *
from helper import *
from mcs_search import *

# passi da effettuare:
# 1. ripulire il file ttd_drug_disease_by_drug.csv da tutte le molecole che non
sono state approvate o che appartengono alla classe k ovarico

# Se questo file non esiste, creiamolo
if not os.path.exists(APPROVED_NON_K_OVARIAN_CSV_FILE_PATH):
    FilterApprovedNonKOvarianDrugs(ALL_DRUGS_CSV_FILE_PATH,
    APPROVED_NON_K_OVARIAN_CSV_FILE_PATH)
    assert os.path.exists(APPROVED_NON_K_OVARIAN_CSV_FILE_PATH), "Il file
filtrato non è stato creato correttamente."

# 2. Caricare `ttd_approved_non_k_ovarian_drugs.csv` e ottenere la lista di
nomi di molecole
# - A partire da quella lista di nomi di molecole scaricare le MOL da
PubChem e salvarle in un file SDF.

```

```

# - se il file SDF esiste già, caricarlo invece di scaricare le MOL.

APPROVED_NON_K_OVARIAN_SDF_FILE_PATH =
"data/approved_non_k_ovarian_drugs_with_indications.sdf"

supplier = LoadMolecules(
    APPROVED_NON_K_OVARIAN_SDF_FILE_PATH,
    fallback_path=APPROVED_NON_K_OVARIAN_CSV_FILE_PATH,
    # Essendo piu' di 3 mila farmaci, escludiamo la ricerca
    # nel dominio delle substance per evitare di spendere troppo tempo
    include_substances=False
)

other_mols: list[Mol] = GetListFromSDMolSupplier(supplier)

images_out_dir = "out/other_mols"
if not os.path.exists(images_out_dir):
    DrawMols(other_mols, images_out_dir)

logger.info(f"Caricate {len(other_mols)} molecole da
'{APPROVED_NON_K_OVARIAN_SDF_FILE_PATH}'")

# Recuperare gli MCS dei cluster ottenuti in `kovarian_clusters.py`
# e confrontarli con le molecole caricate da `approved_non_k_ovarian_drugs.sdf`
kovarian_intersesting_clusters: list[ClusterMCS] =
kovarian_clusters.GetInterestingClusters()

for interesting_cluster_mcs in kovarian_intersesting_clusters:
    # Vediamo se ogni cluster e' incluso nelle molecole caricate da
    `approved_non_k_ovarian_drugs.sdf`
    other_mols_with_mcs = FindSubstructInMols(interesting_cluster_mcs.mcs_mol,
other_mols)
    logger.info(
        f"Trovate {len(other_mols_with_mcs)} molecole che contengono l'MCS del
cluster {interesting_cluster_mcs.cluster_id} nei farmaci approvati non k
ovarici"
    )
    for mol in other_mols_with_mcs:
        # Per capire quale patologia e' associata a `mol`
        # 1 prendi il nome di mol
        mol_name = GetMolName(mol)
        mol_indication = GetMolProperty(mol, "TTD_INDICATION")
        # 2 cerca il nome di mol nel file `approved_non_k_ovarian_drugs.csv` e
        prendi la patologia associata

```

```
# - carica il file `approved_non_k_ovarian_drugs.csv`
# - trova la riga che nella colonna drug_name contiene il nome di
questa molecola
# - prendi da quella riga il valore della colonna disease_name
indications

logger.info(f"- {mol_name} -> {mol_indication}")

ProcessSimilarityWithMols(interesting_cluster_mcs, other_mols,
"approved_mols")
```